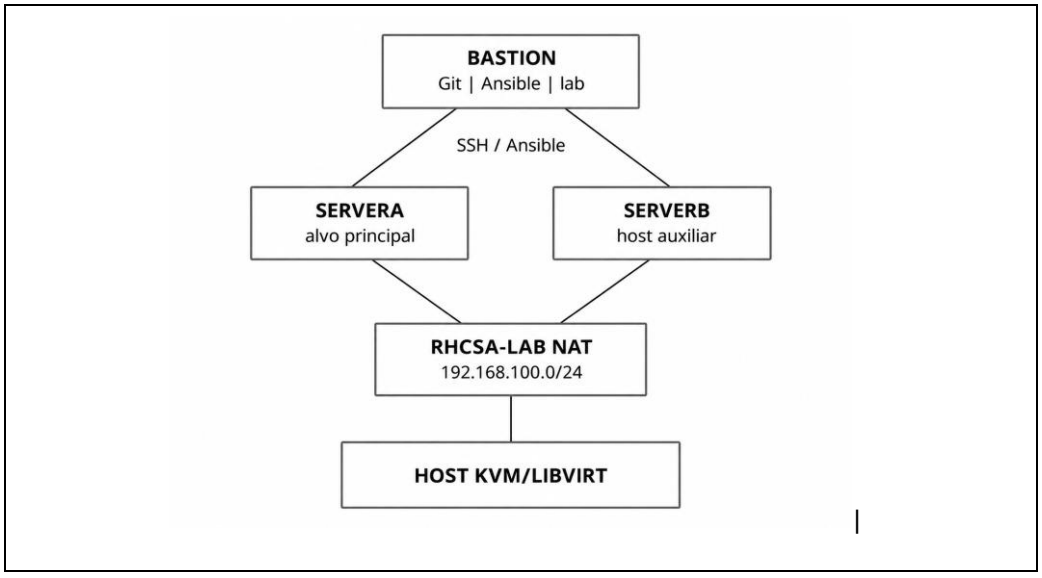
RHCSA EX200 v10 LAB

MANUAL DE IMPLANTAÇÃO, OPERAÇÃO E ESTUDOS

Ambiente hands-on independente para administração Red Hat Enterprise Linux
com foco no RHCSA / EX200



Eduardo Andrade

Edição 1.0 | Base funcional v1.8.4 | Outubro de 2026

Projeto independente • Apache License 2.0

Apresentação

Este manual consolida a arquitetura, a implantação, a operação, a metodologia de estudos, o catálogo de laboratórios, os mecanismos de segurança e a engenharia do projeto RHCSA EX200 v10 Lab. O conteúdo foi organizado a partir do repositório fornecido para esta edição, preservando os procedimentos operacionais e comandos necessários para que outra pessoa consiga reproduzir o ambiente.

O projeto é independente. Ele não é afiliado, patrocinado ou publicado pela Red Hat e não contém questões reais de certificação. Os exercícios são cenários autorais voltados à prática das competências de administração Linux/RHEL.

> ***Observação:*** *O termo "v10" no nome do projeto indica o alvo de estudo baseado em RHEL 10 / EX200. A plataforma local atualmente validada pelo projeto é RHEL 9.8.*

Como utilizar esta apostila

- Para uma instalação nova, siga a Parte II na ordem apresentada.
- Para uso diário, consulte a Parte III e o capítulo do CLI lab.
- Para estudar por domínio, utilize o catálogo de 100 itens da Parte IV.
- Para manutenção do projeto, validação e releases, utilize a Parte V.
- Os procedimentos destrutivos de armazenamento devem respeitar estritamente o contrato de segurança apresentado neste manual.

Sumário

1. Visão geral do projeto 12
2. Arquitetura, rede e armazenamento 13
2.1 Endereçamento 13
2.2 Topologia de discos 14
3. Metodologia, compatibilidade e currículo 15
3.1 Quatro princípios do laboratório 15
3.2 Compatibilidade 15
3.3 Distribuição do currículo 15
4. Roteiro geral de instalação 17
5. Requisitos e arquitetura 18
Plataforma e requisitos 18
Imagens e downloads oficiais 18
Configuração real do hospedeiro 19
Rede isolada de prática 20
Critério para avançar 21
6. Preparação do host KVM/libvirt 22
2.1 Caminho principal — Ubuntu Desktop 24.04 LTS 22
2.2 Caminho alternativo — RHEL 9 / Rocky Linux 9 23
2.3 Cockpit — administração e consultas rápidas 24
2.4 Etapas comuns aos hospedeiros 25
2.5 Preparar o armazenamento 26
2.6 Criar a rede NAT do laboratório 27
2.7 Preparar a origem das VMs 28
2.8 Validação final do host 29
2.9 Critério para avançar 29
7. Criação das máquinas virtuais 31
3.1 Método A — instalação manual a partir da ISO do RHEL 31
3.2 Método B — implantação a partir de uma imagem-base 31
Qual método escolher? 31
3.1 — Método A: instalação manual a partir da ISO do RHEL 32
3.1.1 Resultado esperado 32
3.1.2 Onde executar 32
3.1.3 Pré-requisitos 32
3.1.4 Verificar se os nomes das VMs estão disponíveis 33
3.1.5 Criar a VM bastion 33
3.1.6 Criar a VM servera 34

3.1.7 Criar a VM serverb 34
3.1.8 Confirmar as três VMs 35
3.1.9 Abrir o console de instalação 35
3.1.10 Instalar o RHEL no bastion 36
3.1.11 Instalar o RHEL no servera 36
3.1.12 Instalar o RHEL no serverb 36
3.1.13 Confirmar que as VMs inicializam pelo disco 37
3.1.14 Validar os discos dentro das VMs 37
3.1.15 Validar os discos pelo host KVM 38
3.1.16 Validar as interfaces de rede 38
3.1.17 Problemas comuns 38
3.1.18 Critério para concluir o Método A 39
3.2 — Método B: implantação a partir da imagem-base 40
3.2.1 Resultado esperado 40
3.2.2 Onde executar 40
3.2.3 Pré-requisitos 41
3.2.4 Sobre a imagem-base 41
3.2.5 Download da imagem 42
3.2.6 Verificar o SHA-256 42
3.2.7 Verificar a estrutura QCOW2 42
3.2.8 Preservar uma cópia-mestre 43
3.2.9 Definir os caminhos utilizados pelo procedimento 43
3.2.10 Criar os discos independentes das três VMs 44
3.2.11 Validar os três discos de sistema 44
3.2.12 Criar os discos adicionais do servera 45
3.2.13 Criar os discos adicionais do serverb 45
3.2.14 Ajustar contexto de segurança quando aplicável 45
3.2.15 Criar a VM bastion 46
3.2.16 Criar a VM servera 46
3.2.17 Criar a VM serverb 47
3.2.18 Confirmar as três máquinas virtuais 47
3.2.19 Primeiro boot 47
3.2.20 Identificar o endereço DHCP temporário 48
3.2.21 Validar a identidade de cada sistema 48
3.2.22 Validar as chaves SSH do host 49
3.2.23 Verificar o estado do Red Hat Subscription Management 49
3.2.24 Registrar cada VM individualmente, quando necessário 49
3.2.25 Validar os discos dentro do bastion 50
3.2.26 Validar os discos dentro do servera 50

3.2.27 Validar os discos dentro do serverb 50
3.2.28 Não alterar a imagem-base depois da implantação 51
3.2.29 Verificar que não existem backing files 51
3.2.30 Problemas comuns 51
3.2.31 Como a imagem-base é preparada 53
3.2.32 Segurança da imagem-base 54
3.2.33 Compatibilidade com a versão atual do laboratório 54
3.2.34 Critério para concluir o Método B 55
8. Configuração inicial das VMs 56
4.1 Resultado esperado 56
4.2 Antes de alterar a rede 56
4.3 Identificar o perfil de rede correto 57
4.4 Configurar o bastion 57
4.5 Configurar o servera 58
4.6 Configurar o serverb 59
4.7 Configurar resolução local de nomes 59
4.8 Validar identidades individuais das VMs 60
4.9 Validar o usuário student 60
4.10 Habilitar SSH 61
4.11 Manter SELinux habilitado 61
4.12 Habilitar e configurar o firewalld 61
4.13 Escolher a origem dos pacotes RHEL 62
4.14 Opção A — Red Hat Subscription Management 62
4.15 Opção B — repositório local pela ISO/DVD 63
4.16 Instalar o conjunto básico de pacotes 64
4.17 Validar conectividade com o gateway 65
4.18 Validar comunicação entre as VMs 65
4.19 Validar a topologia dos discos 65
4.20 Validação consolidada por VM 66
4.21 Valores esperados 66
4.22 Problemas comuns 67
4.23 Critério para avançar 68
9. Preparação do bastion, Git e SSH 70
5.1 Resultado esperado 70
5.2 Confirmar que está no bastion 71
5.3 Validar rede e resolução de nomes 71
5.4 Confirmar a origem de pacotes 72
5.5 Instalar as dependências do bastion 72
5.6 Validar as dependências 72

5.7 Confirmar que o diretório do projeto ainda não existe 73
5.8 Clonar o repositório público 73
5.9 Validar a cópia do projeto 73
5.10 Conferir a estrutura essencial do repositório 74
5.11 Separar a chave do laboratório das demais chaves SSH 74
5.12 Criar a chave SSH dedicada 75
5.13 Conferir a fingerprint da chave do bastion 75
5.14 Validar as fingerprints de servera e serverb antes do primeiro acesso 76
5.15 Copiar a chave pública para servera 76
5.16 Copiar a chave pública para serverb 76
5.17 Validar o authorized_keys nos hosts gerenciados 76
5.18 Criar a configuração SSH do bastion 77
5.19 Entender por que IdentitiesOnly yes é utilizado 77
5.20 Validar acesso SSH explícito 78
5.21 Validar acesso pelo arquivo ~/.ssh/config 78
5.22 Validar SSH não interativo 78
5.23 Verificar as host keys armazenadas 79
5.24 O que fazer se uma host key mudar 79
5.25 Se a chave utilizar passphrase 80
5.26 Instalar o comando lab 80
5.27 Atualizar o PATH 80
5.28 Validar o CLI 81
5.29 Não copiar credenciais para o repositório 81
5.30 Atualizar uma instalação existente do projeto 81
5.31 Uso do GitHub para estudantes e mantenedores 82
5.32 Validação consolidada 82
5.33 Problemas comuns 83
5.34 Critério para avançar 85
10. Configuração e validação do Ansible 86
6.1 Arquitetura do plano de controle 86
6.2 Confirmar o ponto de execução 87
6.3 Confirmar as ferramentas necessárias 87
6.4 Trabalhar a partir do diretório Ansible 88
6.5 Conferir o ansible.cfg 88
6.6 Confirmar qual configuração o Ansible está utilizando 89
6.7 Conferir o inventário 90
6.8 Validar a sintaxe e a estrutura do inventário 90
6.9 Confirmar a chave privada referenciada pelo inventário 91
6.10 Confirmar SSH antes de testar Ansible 91

6.11 Confirmar as host keys conhecidas 92
6.12 Preparar a elevação de privilégios da automação 92
6.13 Validar sudo não interativo a partir do bastion 93
6.14 Primeiro teste Ansible: grupo control 94
6.15 Testar os hosts gerenciados 94
6.16 Testar todos os nós 95
6.17 Confirmar a identidade dos hosts pelo Ansible 95
6.18 Confirmar o usuário remoto 95
6.19 Validar become pelo Ansible 95
6.20 Verificar sintaxe dos playbooks principais 96
6.21 Validar o baseline de sistema operacional 96
6.22 Inspecionar as dependências definidas pelo projeto 97
6.23 Preparar as dependências do laboratório 97
6.24 Validar o conjunto de dependências 98
6.25 Executar novamente o teste de conectividade 98
6.26 Validar o CLI lab 99
6.27 Entender o que lab status valida 99
6.28 Validar o catálogo de laboratórios 100
6.29 Verificação consolidada em sequência 100
6.30 Resultado mínimo esperado 101
6.31 Problemas comuns 101
6.32 Critério para avançar 105
11. Baseline e snapshots 106
7.1 Entender a diferença entre lab finish e snapshot 106
7.2 Não criar o baseline com laboratório ativo 107
7.3 Validar o ambiente antes do snapshot 107
7.4 Registrar a versão do projeto utilizada no baseline 108
7.5 Por que o bastion não é o alvo principal do snapshot 109
7.6 Ir para o host KVM 109
7.7 Inspecionar a topologia de discos antes do snapshot 109
7.8 Confirmar o formato dos discos 110
7.9 Verificar snapshots existentes 110
7.10 Desligar os nós gerenciados de forma limpa 111
7.11 Confirmar que os nós realmente desligaram 111
7.12 Criar o snapshot de servera 111
7.13 Criar o snapshot de serverb 112
7.14 Inspecionar o XML dos snapshots 112
7.15 Opcional: snapshot do bastion 113
7.16 Reiniciar as VMs 113

7.17 Aguardar o sistema operacional ficar disponível 114
7.18 Revalidar o ambiente depois do snapshot 114
Restauração 114
7.19 Quando utilizar o snapshot 114
7.20 Preservar dados antes de restaurar 115
7.21 Desligar antes da restauração 115
7.22 Restaurar servera 115
7.23 Restaurar serverb 116
7.24 Restaurar os dois nós 116
7.25 Revalidar SSH após restauração 116
7.26 Revalidar o Ansible após restauração 116
7.27 Sincronizar o estado do CLI depois de uma recuperação manual 117
7.28 Revalidar o baseline completo após restauração 118
Manutenção dos snapshots 118
7.29 Não confundir snapshot com backup 118
7.30 Antes de copiar um QCOW2 118
7.31 Inspecionar snapshots periodicamente 118
7.32 Verificar espaço em disco 119
7.33 Remover um snapshot antigo 119
7.34 Atualizar o baseline após uma nova versão do projeto 119
7.35 Estratégia de nomes recomendada 120
7.36 Problemas comuns 120
7.37 Critério para concluir a etapa 122
12. Primeiro exercício 123
8.1 Fluxo normal de um laboratório 123
8.2 Confirmar que o ambiente está pronto 124
8.3 Consultar o catálogo 124
8.4 Conhecer o exercício antes de iniciá-lo 125
8.5 O que o primeiro exercício pratica 125
8.6 Iniciar o exercício 126
8.7 O que acontece durante `lab start` 126
8.8 Confirmar o laboratório ativo 126
8.9 Conectar ao alvo 127
8.10 Ir para o diretório do exercício 127
8.11 Ler o enunciado novamente, se necessário 128
8.12 Resolver manualmente 128
8.13 Não alterar o cenário fora do solicitado 128
8.14 Sair de servera antes da avaliação 128
8.15 Avaliar o exercício 129

8.16 Como interpretar a avaliação 129
8.17 O que o grader verifica em obj01-01 129
8.18 Corrigir uma tentativa incompleta 130
8.19 Não usar lab start novamente para corrigir uma tentativa 130
8.20 Registrar mentalmente o motivo de cada erro 130
8.21 Finalizar o exercício 131
8.22 Não executar finish antes de terminar a prática 131
8.23 Confirmar a limpeza 131
8.24 O que acontece se lab finish falhar 132
8.25 Um exercício por vez 132
8.26 Status ready e outros estados do catálogo 132
8.27 Como estudar com cada laboratório 133
8.28 Use documentação local antes de procurar respostas prontas 133
8.29 Soluções de referência 134
8.30 Compatibilidade RHEL 9.8 e RHEL 10 134
8.31 O projeto não simula uma prova oficial 134
8.32 Primeira sessão de estudo recomendada 135
8.33 Antes de iniciar o segundo exercício 135
8.34 Critério para concluir a instalação do laboratório 135
13. CLI lab e fluxo operacional 138
13.1 Fluxo de uma sessão 138
14. Estratégia de estudo e avaliação por estado final 139
14.1 Rotina recomendada 139
15. Catálogo completo dos 100 itens 141
Objective 01 — Essential Tools 141
Objective 02 — Users & Groups 141
Objective 03 — RPM, DNF & Repositories 142
Objective 04 — Simple Shell Scripts 142
Objective 05 — Processes, Scheduling, Tuning & systemd 142
Objective 06 — NetworkManager, IP Networking & Name Resolution 143
Objective 07 — Scheduling, Temporary Files, Logging & Time 143
Objective 08 — Partitions, Filesystems, Swap & LVM 144
Objective 09 — Boot, GRUB & Recovery 144
Objective 10 — SELinux & firewalld 145
Objective 11 — NFS & AutoFS 145
Objective 12 — Comprehensive EX200 Challenges 146
15.1 Totais operacionais 146
16. Boot, recuperação manual e segurança de armazenamento 147
16.1 Dois exercícios deliberadamente manuais 147

16.2 Contrato de segurança do Objective 08 147
17. Estrutura do repositório e contrato de um lab 149
17.1 Contrato de um laboratório ready 149
17.2 Separação de responsabilidades 149
18. Release gate, integração de referência e manutenção 150
18.1 Gate estático 150
18.2 Integração de referência 150
18.3 Checklist de release 150
18.4 Fluxo Git recomendado para manutenção documental 150
19. Releases e roadmap 151
19.1 Estado do roadmap 151
20. Referência rápida 154
20.1 Comandos do estudante 154
20.2 Comandos de diagnóstico do ambiente 154
20.3 Valores esperados de rede 154
20.4 Downloads e repositório 154
21. Autoria, apoio, licença e marcas 155
21.1 Autor e perfis profissionais 155
21.2 Apoie o projeto 155
21.3 Independência e marcas 155
21.4 Licença 155
Apêndice A — Arquivo LICENSE (Apache License 2.0) 156

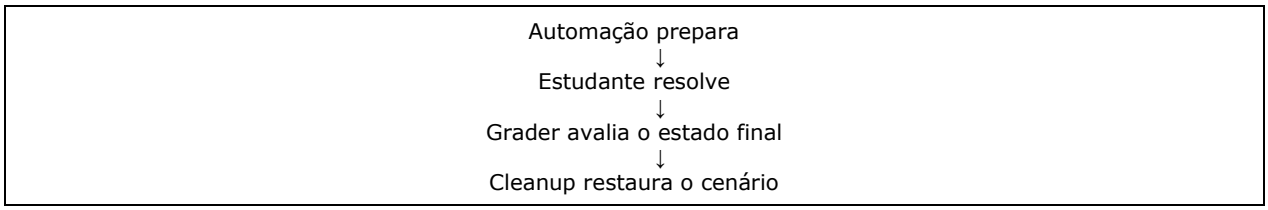
PARTE I

Visão geral, arquitetura e princípios

RHCSA EX200 v10 Lab

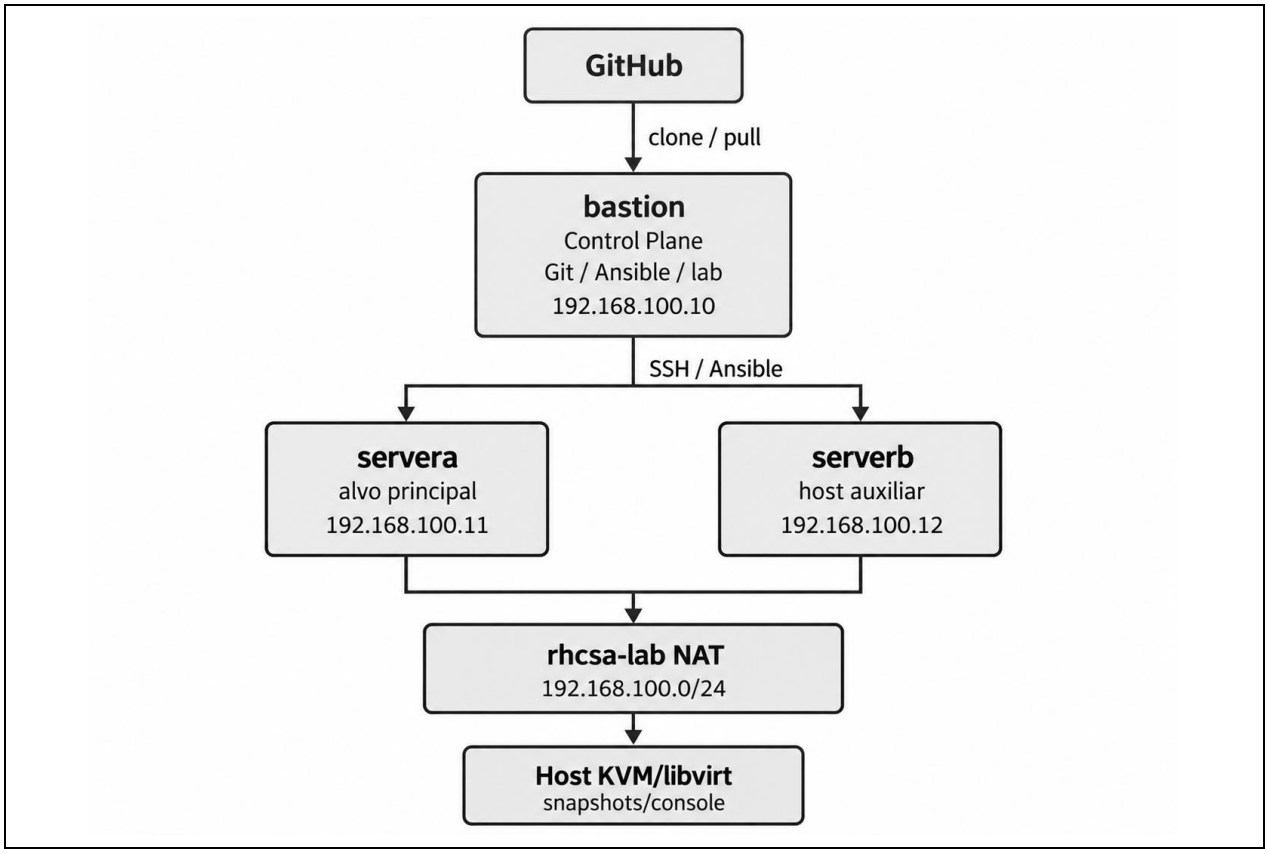
1. Visão geral do projeto

O RHCSA EX200 v10 Lab é um ambiente local, repetível e orientado a estado final para prática de administração Linux/RHEL. O laboratório utiliza três máquinas virtuais: `bastion`, `servera` e `serverb`. A automação prepara o cenário, valida dependências, executa graders e restaura o ambiente; a resolução das tarefas permanece manual.



Item	Estado
Currículo	Concluído
Objetivos	12
Itens de aprendizagem	100
Labs automatizados e avaliáveis em runtime	98
Drills manuais por console	2
CLI lab	Implementado
Setup / grade / cleanup	Implementados
Release gate	Implementado
Regression runner de referência	Implementado
Plataforma local	RHEL 9.8
Alvo de estudo	RHEL 10 / RHCSA EX200
Virtualização	KVM/libvirt
Licença	Apache License 2.0
Base estável	v1.8.4

2. Arquitetura, rede e armazenamento



Componente	Responsabilidade
bastion	Git, Ansible, CLI lab, graders, estado dos exercícios e automação
servera	Principal máquina de prática do estudante
serverb	NFS, AutoFS, SSH, rede, serviços auxiliares e cenários distribuídos
Host KVM	Criação das VMs, discos, console, snapshots e recuperação out-of-band

2.1 Endereçamento

Nó	IPv4	Papel
bastion	192.168.100.10	Plano de controle
servera	192.168.100.11	Alvo principal
serverb	192.168.100.12	Host auxiliar
Gateway libvirt	192.168.100.1	NAT / saída da rede
Domínio local	lab.test	Resolução local

2.2 Topologia de discos

VM	Disco	Tamanho	Uso
bastion	vda	30 GiB	Sistema operacional
servera	vda	30 GiB	Sistema operacional — protegido
servera	vdb	5 GiB	Scratch
servera	vdc	5 GiB	Scratch
servera	vdd	3 GiB	Scratch
servera	vde	2 GiB	Scratch
serverb	vda	30 GiB	Sistema operacional — protegido
serverb	vdb	5 GiB	Scratch
serverb	vdc	3 GiB	Scratch

3. Metodologia, compatibilidade e currículo

3.1 Quatro princípios do laboratório

- O estudante resolve manualmente: Ansible prepara e restaura o ambiente, mas não executa a tarefa do aluno.
- O grader avalia o estado final: uma solução válida não depende de sequência única de comandos.
- O cleanup faz parte do contrato: `lab finish` deve restaurar o cenário previsto.
- O disco do sistema é protegido: exercícios destrutivos de storage são restritos aos discos scratch autorizados.

3.2 Compatibilidade

Nível	Significado
exact	Execução equivalente no ambiente local e no alvo de estudo
concept-and-state	Conceito e estado final alinhados; podem existir diferenças operacionais entre versões
manual-console-rhel10-aligned	Prática manual por console alinhada ao alvo RHEL 10

Observação: *A matriz efetiva dos 100 itens nesta edição contém 52 labs exact, 46 concept-and-state e 2 exercícios manual-console-rhel10-aligned.*

3.3 Distribuição do currículo

Objective	Tema	Itens
01	Essential Tools	10
02	Users & Groups	8
03	RPM, DNF & Repositories	8
04	Simple Shell Scripts	8
05	Processes, Scheduling, Tuning & systemd	10
06	Networking / NetworkManager	8
07	Scheduling, Temporary Files, Logging & Time	8
08	Partitions, Filesystems, Swap & LVM	12
09	Boot, GRUB & Recovery	7
10	SELinux & firewalld	9
11	NFS & AutoFS	6
12	Comprehensive EX200 Challenges	6
Total		100

PARTE II

Implantação completa do laboratório

RHCSA EX200 v10 Lab

4. Roteiro geral de instalação

A implantação segue oito etapas. Os capítulos seguintes incorporam o conteúdo operacional completo da documentação `docs/setup/`, consolidado em formato de apostila.

Etapa	Conteúdo
01	Requisitos e arquitetura
02	Preparação do host KVM/libvirt
03	Criação das VMs — Método A ou Método B
04	Configuração inicial das VMs
05	Bastion, Git e SSH
06	Ansible e validação
07	Baseline e snapshots
08	Primeiro exercício

Observação: *Os procedimentos do Método A e do Método B convergem para a mesma topologia final. Depois da criação das VMs, a configuração segue o mesmo caminho.*

5. Requisitos e arquitetura

Índice · Home · Próximo

Este roteiro começa com um host Linux e termina com o primeiro exercício executado pelo estudante.

A construção das máquinas virtuais pode ser realizada por dois métodos:

- **Método A — instalação manual:** criação das VMs e instalação do RHEL a partir da mídia ISO oficial;
- **Método B — imagem-base:** implantação rápida a partir de uma imagem QCOW2 previamente preparada.

Os dois métodos convergem para a mesma arquitetura final. O Ansible entra posteriormente, no `bastion`, para preparar, restaurar e avaliar os cenários dos laboratórios.

Plataforma e requisitos

Plataforma e requisitos

A base atual do projeto utiliza **RHEL 9.8 nas três VMs** e toma **RHEL 10 como referência de estudo e classificação das competências**.

O playbook `ansible/playbooks/validate.yml` da versão atual valida a base RHEL utilizada pelo projeto. A adoção de RHEL 10 como sistema operacional local exige revisão prévia dessa validação e da compatibilidade dos exercícios; não é suficiente apenas substituir a mídia de instalação.

O hospedeiro utilizado originalmente no desenvolvimento do projeto é **Ubuntu Desktop 24.04.5 LTS (Noble Numbat)** com KVM/libvirt. O guia também contempla sistemas da família RHEL como hospedeiros KVM, mantendo a distribuição do host independente da distribuição instalada nas VMs.

Independentemente do hospedeiro escolhido, a arquitetura lógica do laboratório permanece a mesma.

Imagens e downloads oficiais

Uso	Distribuição / imagem	Link oficial
Host Ubuntu	Ubuntu Desktop 24.04 LTS, ISO AMD64	Imagens da série 24.04
Host alternativo	Rocky Linux 9.8 x86_64, DVD com ambiente gráfico	Diretório oficial de ISOs 9.8
Host alternativo, mídia completa	Rocky Linux 9.8 DVD x86_64	Baixar ISO DVD
VMs da base atual	RHEL 9.8, Binary DVD x86_64	Red Hat Developer · Portal de downloads RHEL

Escolha da origem das VMs

A mídia necessária depende do método escolhido na etapa 03.

Método	Origem	Necessidade
Método A	ISO Binary DVD x86_64 do RHEL 9.8	Obrigatória
Método B	Imagem-base rhel-rhcsa-base-v1.qcow2	Obrigatória
Método B	Arquivo rhel-rhcsa-base-v1.qcow2.sha256	Recomendado para validação de integridade

No **Método A**, cada VM é criada e instalada individualmente a partir da mídia oficial do RHEL.

No **Método B**, a imagem-base é utilizada somente como origem para a criação de discos independentes de `bastion`, `servera` e `serverb`. A imagem original não deve ser utilizada diretamente como disco ativo de uma VM.

O repositório Git não armazena ISOs nem imagens QCOW2 do Red Hat Enterprise Linux.

O padrão deste laboratório é **hospedeiro Desktop, com interface gráfica**. Use Ubuntu Desktop; na alternativa Rocky Linux, instale pela mídia DVD e selecione um ambiente com interface gráfica no instalador. Isso permite usar navegador, virt-manager e consoles localmente. Esse padrão é do hospedeiro; as VMs RHEL continuam com instalação mínima conforme a etapa 3. Use a ISO da série 24.04 disponível na página oficial, sem confundir a atualização da mídia com a versão exata do host original.

O download RHEL depende do acesso autorizado à conta Red Hat. No portal, selecione RHEL 9.8 e a mídia **Binary DVD x86_64** para reproduzir a base. Links diretos autenticados/temporários da ISO não são publicados no projeto. A ISO Boot depende de uma fonte de pacotes durante a instalação; ela não substitui a DVD no procedimento de repositório local.

Confira checksums nas páginas oficiais: SHA256SUMS do Ubuntu e CHECKSUM do Rocky 9.8. Compare o arquivo exato baixado e, para conferir a origem, valide também a assinatura com a chave oficial da distribuição.

Rocky como host ou como VM?

Como hospedeiro: pode executar KVM/libvirt com as VMs RHEL do projeto; siga o caminho Rocky na etapa 2. Essa alternativa está documentada, mas não foi validada neste projeto durante a geração do pacote.

Como sistema das VMs: Rocky Linux 9.8 é uma opção para adaptar os exercícios gerais de Linux, mas não é uma substituição automática no projeto atual. O playbook `ansible/playbooks/validate.yml` exige `ansible_distribution == "RedHat"`; com Rocky ele falhará. Também é necessário revisar repositórios, chaves GPG, pacotes e classificações de compatibilidade antes de considerar essa variante validada. Não use a configuração de repositórios RHEL/DVD como se fosse Rocky. A base de referência continua RHEL 9.8 nas VMs.

Configuração real do hospedeiro

Item	Registro disponível
Equipamento	ThinkPad T430 (identificação informada no terminal do autor)
Sistema	Ubuntu Desktop 24.04.5 LTS (Noble Numbat), x86_64
Virtualização	KVM/QEMU gerenciado por libvirt
Processador	Intel Core i5-3320M, frequência nominal 2,60 GHz; máxima reportada 3,30 GHz
Topologia de CPU	1 socket, 2 núcleos físicos, 2 threads por núcleo: 4 CPUs lógicas
Virtualização da CPU	Intel VT-x (vmx no resultado de lscpu)
Memória RAM	16 GiB reportados por free -h
Swap	4,0 GiB
Disco físico	SSD Kingston SA400S37240G, capacidade comercial 240 GB; 223,6 GiB exibidos por lsblk
Tipo de armazenamento	Não rotacional (ROTA=0 no SSD)
Rede das VMs	192.168.100.0/24; gateway 192.168.100.1
Discos virtuais dos alvos	Sistema 30 GiB; discos extras conforme tabela abaixo

Configuração registrada a partir da coleta fornecida pelo autor em 02/10/2026. Os 4 processadores lógicos correspondem a **2 núcleos físicos com Hyper-Threading**, não a 4 núcleos físicos. A coleta identifica VT-x; a validação operacional de KVM continua na etapa 2.

Os valores momentâneos de memória usada/disponível não são requisitos do projeto e não foram tratados como capacidade fixa. A coleta não informa o espaço livre do filesystem nem confirma o diretório físico dos discos das VMs. Antes de uma nova instalação, confira `df -h /var/lib/libvirt/images` (ou o caminho real do pool).

A tabela de recursos abaixo é **dimensionamento sugerido para uma nova instalação**, não a alocação medida das VMs atuais. As três VMs propostas somam 7 GiB de RAM e 6 vCPUs; vCPUs são uma alocação compartilhada sobre as 4 CPUs lógicas do host. O desempenho depende da carga simultânea e dos recursos consumidos pelo Ubuntu.

Para conferir a configuração de outro hospedeiro, execute nele:

```
cat /etc/os-release
lscpu
free -h
lsblk -d -o NAME,SIZE,MODEL,ROTA
sudo virsh -c qemu:///system net-list --all
sudo virsh -c qemu:///system pool-list --all
```

Não execute a coleta dentro de bastion: ela mostraria os recursos da VM. `ROTA=0` normalmente indica armazenamento não rotacional; `ROTA=1` indica rotacional conforme exposto pelo dispositivo.

Dimensionamento inicial sugerido, não um mínimo certificado:

Máquina	vCPU	RAM	Disco do sistema	Discos extras
bastion	2	2 GiB	30 GiB	nenhum
servera	2	3 GiB	30 GiB	5 + 5 + 3 + 2 GiB
serverb	2	2 GiB	30 GiB	5 + 3 GiB

Reserve memória para o host; 16 GiB de RAM é uma configuração inicial confortável. Reserve pelo menos 120 GiB livres para sistemas, ISO e crescimento dos snapshots. A necessidade real varia com exercícios e retenção. SSD é preferível.

Pré-requisitos para iniciar a instalação

Independentemente do método escolhido:

- CPU x86_64 com AMD-V ou Intel VT-x habilitado no firmware;
- host Linux com KVM/QEMU e libvirt;
- acesso administrativo ao hospedeiro;
- espaço em disco suficiente para as VMs, imagens e snapshots;
- conexão para obtenção do projeto e das dependências;
- faixa de rede disponível para o laboratório.

Para o **Método A**, também é necessária uma mídia ISO DVD do RHEL obtida por meio de acesso autorizado à Red Hat.

Para o **Método B**, é necessária uma imagem-base QCOW2 compatível com a versão documentada pelo projeto e, preferencialmente, seu respectivo arquivo de checksum SHA-256.

ISOs, credenciais Red Hat e outros materiais proprietários não são armazenados no repositório Git.

Rede isolada de prática

Elemento	Endereço	Papel
Rede libvirt rhcsa-lab	192.168.100.0/24	NAT do laboratório

Elemento	Endereço	Papel
Interface do host virbr-rhcsa	192.168.100.1	gateway/DNS das VMs
bastion	192.168.100.10	estação do aluno e automação
servera	192.168.100.11	alvo principal
serverb	192.168.100.12	alvo auxiliar

Domínio usado no roteiro: lab.test. Verifique se 192.168.100.0/24 conflita com LAN, VPN ou outra rede libvirt. Se conflitar, escolha outra faixa e atualize rede, VMs, resolução de nomes e inventário juntos.

O host KVM executa virsh e cria as VMs. O bastion executa Git, Ansible e lab. As tarefas dos exercícios são resolvidas manualmente em servera/serverb. Não é necessário clonar o projeto no host para seguir as etapas 2 a 4.

Critério para avançar

Antes de seguir para a preparação do host KVM/libvirt, confirme:

```
[ ] virtualização por hardware disponível
[ ] KVM/libvirt poderá ser utilizado no hospedeiro
[ ] memória e armazenamento suficientes para as três VMs
[ ] faixa 192.168.100.0/24 sem conflito com LAN, VPN ou outra rede libvirt
[ ] Método A ou Método B escolhido
[ ] origem necessária para o método escolhido disponível

[Índice](README.md) · [Home](../../README.md) · [Próximo](02-host-kvm.md)
```

6. Preparação do host KVM/libvirt

Índice · Home · Anterior · Próximo

Onde executar: terminal do host KVM, com privilégios administrativos via `sudo`.

Esta etapa prepara o hospedeiro que executará as máquinas virtuais do laboratório. O objetivo é deixar KVM/QEMU, libvirt, armazenamento, rede virtual e ferramentas de administração prontos antes da criação de `bastion`, `servera` e `serverb`.

A preparação desta página é comum aos dois métodos de implantação:

- **Método A — instalação manual:** criação das VMs e instalação do RHEL a partir da mídia ISO;
- **Método B — imagem-base:** criação das VMs a partir de uma imagem QCOW2 previamente preparada.

2.1 Caminho principal — Ubuntu Desktop 24.04 LTS

Parta de um Ubuntu Desktop 24.04 LTS já instalado.

2.1.1 Instalar KVM, libvirt e ferramentas de administração

Atualize o índice de pacotes:

```
sudo apt update
```

Instale os componentes necessários:

```
sudo apt install -y \
  qemu-kvm \
  qemu-utils \
  libvirt-daemon-system \
  libvirt-clients \
  virtinst \
  virt-manager \
  virt-viewer \
  cpu-checker \
  libosinfo-bin \
  cockpit \
  cockpit-machines
```

Os principais componentes utilizados pelo projeto são:

Componente	Finalidade
qemu-kvm	Virtualização acelerada por hardware
qemu-utils	Ferramentas para manipulação e validação de imagens, incluindo qemu-img
libvirt-daemon-system	Serviços de virtualização gerenciados pelo libvirt
libvirt-clients	Ferramentas de linha de comando, incluindo virsh
virtinst	Ferramentas de criação de VMs, incluindo virt-install
virt-manager	Administração gráfica das VMs
virt-viewer	Acesso gráfico ao console das VMs
cpu-checker	Validação de suporte à virtualização
libosinfo-bin	Informações de sistemas operacionais utilizadas pelo virt-install
cockpit	Administração web do hospedeiro
cockpit-machines	Gerenciamento de máquinas virtuais pelo Cockpit

2.1.2 Habilitar os serviços

```
sudo systemctl enable --now libvirtd
sudo systemctl enable --now cockpit.socket
```

2.1.3 Adicionar o usuário aos grupos de virtualização

```
sudo usermod -aG kvm,libvirt "$USER"
```

Encerre a sessão do usuário e entre novamente para que a associação aos grupos seja atualizada.

O restante deste guia utiliza `sudo` nos comandos administrativos, portanto a atualização dos grupos não impede a continuidade da preparação.

2.1.4 Validar virtualização por hardware

Execute:

```
kvm-ok
```

Depois:

```
sudo virt-host-validate qemu
```

Confirme também o acesso ao libvirt:

```
sudo virsh -c qemu:///system list --all
```

Se a virtualização não estiver disponível, confirme no firmware da máquina se Intel VT-x ou AMD-V está habilitado.

Falhas reportadas por `virt-host-validate` devem ser investigadas antes da criação das VMs.

Referência oficial: libvirt no Ubuntu.

2.2 Caminho alternativo — RHEL 9 / Rocky Linux 9

Os comandos desta seção são destinados a hospedeiros da família RHEL.

Não execute os comandos `dnf` desta seção em um host Ubuntu.

2.2.1 Validar suporte à virtualização

Execute:

```
lscpu
```

Confirme também a presença do dispositivo KVM:

```
ls -l /dev/kvm
```

Se `/dev/kvm` não existir, verifique:

- se Intel VT-x ou AMD-V está habilitado no firmware;
- se o processador suporta virtualização;
- se `kvm_intel` ou `kvm_amd` está carregado.

Exemplos:

```
lsmod | grep '^kvm'
grep -Eo 'vmx|svm' /proc/cpuinfo | head
```

2.2.2 Instalar as ferramentas

```
sudo dnf install -y \
  qemu-kvm \
  libvirt \
  virt-install \
  virt-viewer \
  cockpit \
  cockpit-machines
```

O virt-manager é opcional quando o hospedeiro é administrado principalmente por Cockpit ou linha de comando.

Para instalá-lo:

```
sudo dnf install -y virt-manager
```

2.2.3 Habilitar os serviços do libvirt

Em instalações que utilizam os daemons modulares do libvirt, habilite os sockets necessários:

```
for drv in qemu network nodedev nwfilter secret storage interface; do
  sudo systemctl enable --now virt${drv}d{,-ro,-admin}.socket
done
```

Habilite também o Cockpit:

```
sudo systemctl enable --now cockpit.socket
```

> Dependendo da distribuição e da forma como o libvirt foi empacotado, *libvirtd* ainda pode estar disponível. O objetivo desta etapa é garantir que a conexão *qemu:///system* esteja funcional, independentemente do modelo de daemon utilizado pelo hospedeiro.

2.2.4 Validar o hospedeiro

Execute:

```
sudo virt-host-validate qemu
```

Depois:

```
sudo virsh -c qemu:///system list --all
```

Falhas (FAIL) devem ser investigadas antes da criação das VMs.

Avisos (WARN) devem ser avaliados conforme a funcionalidade envolvida. IOMMU, por exemplo, não é requisito deste laboratório, pois o projeto não utiliza passthrough de dispositivos.

2.3 Cockpit — administração e consultas rápidas

O Cockpit e o módulo cockpit-machines fazem parte da preparação do **host KVM**.

Não é necessário instalar o Cockpit nas três VMs apenas para administrar o hospedeiro e visualizar suas máquinas virtuais.

2.3.1 Confirmar o serviço

```
systemctl is-active cockpit.socket
```

Consulte detalhes:

```
systemctl status cockpit.socket --no-pager
```

2.3.2 Acessar a interface web

A interface web do Cockpit utiliza, por padrão, a porta TCP 9090.

No próprio hospedeiro:

```
https://localhost:9090
```

A partir de outra estação da rede:

```
https://ENDERECO_DO_HOST:9090
```

Entre com uma conta autorizada do **hospedeiro KVM**, e não com a conta `student` das VMs do laboratório.

Uma instalação com certificado local pode apresentar um aviso de certificado no primeiro acesso. Confirme que o endereço acessado corresponde ao seu hospedeiro antes de prosseguir.

O Cockpit pode ser utilizado para consultar:

- CPU e memória;
- serviços;
- logs;
- interfaces de rede;
- armazenamento;
- máquinas virtuais.

O módulo `cockpit-machines` adiciona a seção **Máquinas virtuais**.

Para ações administrativas, utilize a elevação de privilégios oferecida pela interface.

Para administração remota, a liberação da porta 9090 deve respeitar a política de firewall e segurança da rede em que o hospedeiro está instalado.

O roteiro não exige exposição do Cockpit à Internet.

Referências:

- Cockpit — instalação
- Cockpit — máquinas virtuais

2.3.3 Cockpit em Ubuntu via backports

No Ubuntu LTS, também é possível utilizar versões disponibilizadas pelos backports oficiais.

Com `noble-backports` habilitado:

```
sudo apt install -t noble-backports cockpit cockpit-machines
```

Essa é uma opção de atualização.

A instalação inicial descrita anteriormente utiliza os repositórios já habilitados no sistema.

2.4 Etapas comuns aos hospedeiros

A partir deste ponto, o procedimento é o mesmo independentemente de o hospedeiro utilizar Ubuntu, RHEL ou Rocky Linux.

Antes de criar recursos, inspecione o ambiente existente:

```
ip route
sudo virsh -c qemu:///system net-list --all
sudo virsh -c qemu:///system pool-list --all
```

O projeto utiliza como padrão:

Recurso	Valor
Rede libvirt	rhcsa-lab
Bridge virtual	virbr-rhcsa
Rede IPv4	192.168.100.0/24
Gateway	192.168.100.1
Domínio	lab.test
Pool principal	default
Diretório dos discos	/var/lib/libvirt/images

Antes de criar a rede, confirme que `192.168.100.0/24` não conflita com:

- rede local;
- VPN;
- outra bridge;
- outra rede libvirt;
- redes utilizadas por containers.

2.5 Preparar o armazenamento

Liste os pools existentes:

```
sudo virsh -c qemu:///system pool-list --all
```

2.5.1 Se o pool ***default*** já existir

Se estiver ativo, não faça alterações.

Se estiver inativo:

```
sudo virsh -c qemu:///system pool-start default
```

Para habilitar inicialização automática:

```
sudo virsh -c qemu:///system pool-autostart default
```

2.5.2 Se o pool ***default*** não existir

Crie o diretório:

```
sudo mkdir -p /var/lib/libvirt/images
```

Defina o pool:

```
sudo virsh -c qemu:///system \
  pool-define-as default dir \
  --target /var/lib/libvirt/images
```

Inicie:

```
sudo virsh -c qemu:///system pool-start default
```

Habilite autostart:

```
sudo virsh -c qemu:///system pool-autostart default
```

Valide:

```
sudo virsh -c qemu:///system pool-info default
```

Este projeto utiliza discos QCOW2 armazenados nesse pool.

Em hospedeiros RHEL/Rocky, mantenha SELinux habilitado.

Em Ubuntu, preserve as proteções AppArmor utilizadas pelo libvirt.

Não desabilite mecanismos de segurança para fazer o laboratório funcionar.

2.6 Criar a rede NAT do laboratório

Antes de criar a rede, confira novamente as rotas:

```
ip route
```

Confira as redes do libvirt:

```
sudo virsh -c qemu:///system net-list --all
```

A rede padrão deste projeto será:

```
Nome:       rhcsa-lab
Bridge:     virbr-rhcsa
Rede:       192.168.100.0/24
Gateway:    192.168.100.1
Domínio:    lab.test
DHCP:       192.168.100.100 a 192.168.100.199
```

Os endereços fixos das VMs ficam fora da faixa DHCP:

```
bastion   192.168.100.10
servera   192.168.100.11
serverb   192.168.100.12
```

2.6.1 Criar o XML da rede

Crie /tmp/rhcsa-lab.xml:

```
cat > /tmp/rhcsa-lab.xml <<'XML'
<network>
  <name>rhcsa-lab</name>
  <forward mode='nat'/>
  <bridge name='virbr-rhcsa' stp='on' delay='0'/>
  <domain name='lab.test'/>
  <ip address='192.168.100.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.100.100' end='192.168.100.199'/>
    </dhcp>
  </ip>
</network>
XML
```

Defina a rede:

```
sudo virsh -c qemu:///system net-define /tmp/rhcsa-lab.xml
```

Inicie:

```
sudo virsh -c qemu:///system net-start rhcsa-lab
```

Habilite autostart:

```
sudo virsh -c qemu:///system net-autostart rhcsa-lab
```

2.6.2 Validar a rede

Consulte:

```
sudo virsh -c qemu:///system net-info rhcsa-lab
```

Confira o XML efetivamente aplicado:

```
sudo virsh -c qemu:///system net-dumpxml rhcsa-lab
```

Valide a bridge:

```
ip addr show virbr-rhcsa
```

Ela deve possuir:

```
192.168.100.1/24
```

2.6.3 Se a rede já existir

Não redefina automaticamente uma rede existente.

Primeiro consulte:

```
sudo virsh -c qemu:///system net-dumpxml rhcsa-lab
```

Reutilize-a somente se corresponder ao plano de endereçamento deste projeto.

2.7 Preparar a origem das VMs

A origem necessária depende do método escolhido na etapa seguinte.

2.7.1 Método A — instalação manual

O Método A utiliza a mídia ISO oficial do RHEL.

Se desejar seguir esse método, disponibilize a ISO no hospedeiro.

O roteiro utiliza como referência:

```
/var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso
```

Esse caminho é apenas uma convenção do projeto e pode ser adaptado.

Confira:

```
ls -lh /var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso
```

Valide a integridade da ISO utilizando o checksum fornecido pela origem oficial.

A mídia ISO não é distribuída por este repositório.

2.7.2 Método B — imagem-base

O Método B não exige a ISO para criar `bastion`, `servera` e `serverb`.

A imagem QCOW2 e seu checksum serão preparados e validados na página:

3.2 — Método B: implantação a partir da imagem-base

Não copie a imagem para o pool nem crie as VMs nesta etapa.

A página do Método B executa esse procedimento de forma controlada.

2.8 Validação final do host

Antes de prosseguir, faça uma última conferência.

2.8.1 Libvirt

```
sudo virsh -c qemu:///system list --all
```

2.8.2 Pool

```
sudo virsh -c qemu:///system pool-info default
```

O pool deve aparecer como ativo.

2.8.3 Rede

```
sudo virsh -c qemu:///system net-info rhcsa-lab
```

A rede deve estar:

```
Active: yes
Autostart: yes
```

2.8.4 Bridge

```
ip -br addr show virbr-rhcsa
```

A bridge deve possuir o endereço:

```
192.168.100.1/24
```

2.8.5 Ferramentas

Confirme:

```
command -v virsh
command -v virt-install
command -v qemu-img
```

Se utilizar interface gráfica:

```
command -v virt-viewer
```

2.9 Critério para avançar

Antes de prosseguir para a criação das máquinas virtuais, confirme:

```
[ ] virtualização por hardware validada
[ ] acesso qemu:///system funcionando
[ ] KVM/QEMU instalado
[ ] libvirt operacional
[ ] virt-install disponível
[ ] qemu-img disponível

[ ] pool default ativo
[ ] diretório /var/lib/libvirt/images disponível

[ ] rede rhcsa-lab ativa
[ ] bridge virbr-rhcsa criada
[ ] gateway 192.168.100.1 configurado
[ ] faixa 192.168.100.0/24 sem conflito

[ ] Cockpit funcional, se escolhido para administração

[ ] Método A ou Método B definido
```

Se utilizar o **Método A**:

```
[ ] ISO oficial do RHEL disponível
[ ] integridade da ISO validada
```

Se utilizar o **Método B**:

```
[ ] host preparado para receber e validar a imagem-base
```

Com o hospedeiro preparado, prossiga para:

03 — Criação das máquinas virtuais

Índice · Home · Anterior · Próximo

7. Criação das máquinas virtuais

Índice · Home · Anterior · Próximo

Nesta etapa são criadas as três máquinas virtuais que formam o ambiente do laboratório:

Máquina	Função principal	RAM	vCPU	Disco do sistema
bastion	Estação de controle, Git, Ansible e comando lab	2 GiB	2	30 GiB
servera	Alvo principal dos exercícios	3 GiB	2	30 GiB
serverb	Alvo auxiliar dos exercícios	2 GiB	2	30 GiB

O projeto oferece dois métodos de implantação. Escolha apenas um deles.

3.1 Método A — instalação manual a partir da ISO do RHEL

Neste método, cada máquina virtual é criada individualmente e o Red Hat Enterprise Linux é instalado a partir da mídia oficial.

É o caminho recomendado para quem deseja praticar também:

- criação das VMs;
- instalação do RHEL;
- seleção do disco de instalação;
- configuração inicial do sistema;
- preparação completa do ambiente desde o início.

Seguir o Método A — instalação manual

3.2 Método B — implantação a partir de uma imagem-base

Neste método, uma imagem QCOW2 previamente preparada é utilizada como ponto de partida para as três máquinas virtuais.

O objetivo é reduzir o tempo necessário para montar o ambiente e permitir que o estudante chegue mais rapidamente aos laboratórios RHCSA.

Mesmo utilizando a imagem-base, cada VM será individualizada antes do uso, recebendo:

- disco próprio;
- identidade própria;
- hostname próprio;
- endereço IP próprio;
- endereço MAC próprio;
- novas chaves SSH do host;
- registro Red Hat individual, quando aplicável.

Seguir o Método B — imagem-base/template

Qual método escolher?

Situação	Método recomendado
Primeira montagem do laboratório	Método A
Deseja aprender também a instalação do RHEL	Método A
Deseja reproduzir todo o processo manualmente	Método A

Situação	Método recomendado
Já conhece instalação de RHEL	Método B
Precisa reconstruir rapidamente o laboratório	Método B
Novo host KVM para demonstrações ou estudos	Método B

Os dois métodos devem resultar na mesma arquitetura lógica:

```
          Host KVM/libvirt
                 |
              rhcsa-lab
            192.168.100.0/24
                 |
      +----------+----------+
      |          |          |
   bastion    servera    serverb
192.168.100.10  192.168.100.11  192.168.100.12
```

3.1 — Método A: instalação manual a partir da ISO do RHEL

Criação das VMs · Índice · Home · Anterior · Próximo

Este método cria as três máquinas virtuais do laboratório individualmente e instala o Red Hat Enterprise Linux a partir da mídia ISO.

É o método recomendado para quem deseja praticar também a criação das VMs, a instalação do sistema operacional e a identificação dos discos utilizados pelo laboratório.

3.1.1 Resultado esperado

Ao final desta etapa, o host KVM deverá possuir:

VM	RAM	vCPU	Disco do sistema	Discos adicionais
bastion	2 GiB	2	30 GiB	nenhum
servera	3 GiB	2	30 GiB	5 GiB, 5 GiB, 3 GiB e 2 GiB
serverb	2 GiB	2	30 GiB	5 GiB e 3 GiB

Os discos adicionais não devem ser particionados durante a instalação do RHEL. Eles serão utilizados posteriormente pelos exercícios de armazenamento.

3.1.2 Onde executar

Local: host KVM/libvirt.

Os comandos desta página devem ser executados no sistema que hospeda as máquinas virtuais, e não dentro de `bastion`, `servera` ou `serverb`.

Confirme o host antes de continuar:

```
hostnamectl
```

3.1.3 Pré-requisitos

Antes de criar as máquinas, confirme que:

- KVM/libvirt está instalado e operacional;
- o serviço libvirt está ativo;
- a rede `rhcsa-lab` foi criada na etapa anterior;
- o pool de armazenamento `default` está disponível;
- a ISO do RHEL está armazenada no host;
- não existem VMs antigas utilizando os nomes `bastion`, `servera` ou `serverb`.

Confirme o libvirt:

```
sudo virsh -c qemu:///system list --all
```

Confirme as redes:

```
sudo virsh -c qemu:///system net-list --all
```

A rede do laboratório deverá aparecer como ativa:

```
Name       State    Autostart   Persistent
----------------------------------------------
rhcsa-lab  active   yes         yes
```

Confirme o pool de armazenamento:

```
sudo virsh -c qemu:///system pool-list --all
```

Confirme a presença da ISO:

```
ls -lh /var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso
```

> O nome e o caminho da ISO podem ser diferentes no seu host. Se necessário, ajuste o parâmetro *--cdrom* nos comandos desta página.

3.1.4 Verificar se os nomes das VMs estão disponíveis

Execute:

```
sudo virsh -c qemu:///system list --all
```

Não prossiga se já existirem domínios chamados:

```
bastion
servera
serverb
```

Esta documentação considera uma instalação nova.

3.1.5 Criar a VM bastion

A VM `bastion` será a estação de controle do laboratório.

Ela hospedará posteriormente o repositório Git, Ansible e o comando `lab`.

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name bastion \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
```

```
  --disk pool=default,size=30,format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --cdrom /var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confirme a criação:

```
sudo virsh -c qemu:///system list --all
```

Verifique o disco associado:

```
sudo virsh -c qemu:///system domblklist bastion
```

A VM deve possuir um disco principal de 30 GiB.

3.1.6 Criar a VM servera

`servera` é o principal alvo dos exercícios RHCSA.

Além do disco do sistema, ela recebe quatro discos adicionais para os laboratórios de particionamento, LVM, sistemas de arquivos e swap.

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name servera \
  --memory 3072 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk pool=default,size=30,format=qcow2,bus=virtio \
  --disk pool=default,size=5,format=qcow2,bus=virtio \
  --disk pool=default,size=5,format=qcow2,bus=virtio \
  --disk pool=default,size=3,format=qcow2,bus=virtio \
  --disk pool=default,size=2,format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --cdrom /var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confira:

```
sudo virsh -c qemu:///system domblklist servera
```

Depois da instalação, a expectativa é que o RHEL reconheça os discos VirtIO aproximadamente como:

```
/dev/vda   30 GiB   disco do sistema
/dev/vdb    5 GiB   laboratório
/dev/vdc    5 GiB   laboratório
/dev/vdd    3 GiB   laboratório
/dev/vde    2 GiB   laboratório
```

A ordem deve ser validada dentro da VM com `lsblk`.

3.1.7 Criar a VM serverb

`serverb` funciona como segundo host do laboratório e fornece recursos utilizados por exercícios que envolvem comunicação entre sistemas.

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name serverb \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk pool=default,size=30,format=qcow2,bus=virtio \
  --disk pool=default,size=5,format=qcow2,bus=virtio \
  --disk pool=default,size=3,format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --cdrom /var/lib/libvirt/images/rhel-9.8-x86_64-dvd.iso \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confira:

```
sudo virsh -c qemu:///system domblklist serverb
```

Depois da instalação, a expectativa é:

```
/dev/vda   30 GiB   disco do sistema
/dev/vdb    5 GiB   laboratório
/dev/vdc    3 GiB   laboratório
```

3.1.8 Confirmar as três VMs

Execute:

```
sudo virsh -c qemu:///system list --all
```

As três máquinas devem aparecer:

```
bastion
servera
serverb
```

Consulte também as interfaces virtuais:

```
sudo virsh -c qemu:///system domiflist bastion
sudo virsh -c qemu:///system domiflist servera
sudo virsh -c qemu:///system domiflist serverb
```

Cada VM deve possuir sua própria interface e endereço MAC.

3.1.9 Abrir o console de instalação

Os comandos `virt-install` utilizam:

```
--graphics vnc,listen=127.0.0.1
--noautoconsole
```

Isso significa que a VM é criada sem prender o terminal ao instalador.

Em um host com ambiente gráfico, uma opção é utilizar:

```
virt-manager
```

Também é possível utilizar `virt-viewer`:

```
sudo virt-viewer --connect qemu:///system bastion
```

Para as demais:

```
sudo virt-viewer --connect qemu:///system servera
sudo virt-viewer --connect qemu:///system serverb
```

Se o host KVM for administrado remotamente, utilize uma forma de acesso gráfico compatível com sua infraestrutura.

3.1.10 Instalar o RHEL no bastion

No instalador do RHEL, configure o sistema normalmente.

Para o disco de instalação, utilize somente o disco de **30 GiB**.

No bastion não existem discos adicionais, portanto a seleção é direta.

Defina o hostname:

```
bastion.lab.test
```

Crie o usuário:

```
student
```

A senha local utilizada no seu laboratório não deve ser publicada no repositório Git.

Conclua a instalação e reinicie a VM.

3.1.11 Instalar o RHEL no servera

Este ponto exige mais atenção.

servera possui cinco discos:

```
30 GiB
5 GiB
5 GiB
3 GiB
2 GiB
```

No instalador, selecione **somente o disco de 30 GiB** como destino da instalação.

Não crie partições, LVM ou sistemas de arquivos nos quatro discos adicionais.

Eles devem permanecer intactos para os exercícios.

Defina o hostname:

```
servera.lab.test
```

Crie também o usuário:

```
student
```

Conclua a instalação e reinicie a VM.

3.1.12 Instalar o RHEL no serverb

serverb possui:

```
30 GiB
5 GiB
3 GiB
```

Novamente, utilize **somente o disco de 30 GiB** para instalar o sistema operacional.

Os discos de 5 GiB e 3 GiB devem permanecer sem uso.

Defina:

```
serverb.lab.test
```

Crie o usuário:

```
student
```

Conclua a instalação e reinicie.

3.1.13 Confirmar que as VMs inicializam pelo disco

Depois da instalação, confirme:

```
sudo virsh -c qemu:///system list --all
```

Se necessário, inicie uma VM manualmente:

```
sudo virsh -c qemu:///system start bastion
sudo virsh -c qemu:///system start servera
sudo virsh -c qemu:///system start serverb
```

As máquinas devem inicializar normalmente pelo sistema instalado.

3.1.14 Validar os discos dentro das VMs

Entre em cada VM pelo console e execute:

```
lsblk
```

No bastion, deve existir apenas o disco de sistema:

```
vda    30G
```

No servera, devem aparecer:

```
vda    30G
vdb     5G
vdc     5G
vdd     3G
vde     2G
```

No serverb:

```
vda    30G
vdb     5G
vdc     3G
```

O ponto mais importante é confirmar que os discos adicionais não possuem partições ou sistemas de arquivos criados durante a instalação.

Uma verificação mais detalhada pode ser feita com:

```
lsblk -f
```

3.1.15 Validar os discos pelo host KVM

No host:

```
sudo virsh -c qemu:///system domblklist bastion
```

Depois:

```
sudo virsh -c qemu:///system domblklist servera
```

E:

```
sudo virsh -c qemu:///system domblklist serverb
```

Isso permite conferir quais arquivos de disco estão associados a cada domínio.

3.1.16 Validar as interfaces de rede

No host KVM:

```
sudo virsh -c qemu:///system domiflist bastion
sudo virsh -c qemu:///system domiflist servera
sudo virsh -c qemu:///system domiflist serverb
```

Confirme que todas as interfaces estão associadas à rede:

```
rhcsa-lab
```

Cada VM deve possuir um endereço MAC diferente.

3.1.17 Problemas comuns

A VM já existe

Exemplo:

```
ERROR: Domain name 'servera' already exists
```

Confirme:

```
sudo virsh -c qemu:///system list --all
```

Não sobrescreva uma VM existente sem saber se ela contém dados que precisam ser preservados.

A ISO não foi encontrada

Confirme o caminho:

```
ls -lh /var/lib/libvirt/images/
```

Ajuste `--cdrom` para o caminho real da mídia.

A rede ***rhcsa-lab*** não existe

Confira:

```
sudo virsh -c qemu:///system net-list --all
```

Se ela não existir, retorne à etapa:

02 — Preparação do host KVM/libvirt

A VM não inicializa

Consulte:

```
sudo virsh -c qemu:///system list --all
```

Tente iniciar:

```
sudo virsh -c qemu:///system start NOME_DA_VM
```

Consulte detalhes:

```
sudo virsh -c qemu:///system dominfo NOME_DA_VM
```

O servera não possui todos os discos

Confira no host:

```
sudo virsh -c qemu:///system domblklist servera
```

E dentro da VM:

```
lsblk
```

Não avance para os laboratórios de armazenamento enquanto a topologia de discos não estiver correta.

3.1.18 Critério para concluir o Método A

Antes de avançar, confirme:

```
[ ] bastion existe no libvirt
[ ] servera existe no libvirt
[ ] serverb existe no libvirt

[ ] as três VMs inicializam pelo disco do sistema
[ ] o RHEL está instalado nas três VMs
[ ] o usuário student existe nas três VMs

[ ] bastion possui disco de sistema de 30 GiB

[ ] servera possui:
   30 GiB
    5 GiB
    5 GiB
    3 GiB
    2 GiB

[ ] serverb possui:
   30 GiB
    5 GiB
    3 GiB

[ ] somente o disco de sistema foi utilizado durante a instalação
[ ] os discos adicionais permanecem disponíveis para os laboratórios
[ ] cada VM possui sua própria interface de rede
```

Se todos os itens estiverem corretos, o Método A está concluído.

Continue para:

04 — Configuração inicial das VMs

Criação das VMs · Índice · Home · Anterior · PróximoS

3.2 — Método B: implantação a partir da imagem-base

Criação das VMs · Índice · Home · Anterior · Próximo

Este método cria o ambiente do laboratório a partir de uma imagem-base QCOW2 previamente preparada.

O objetivo é reduzir o tempo necessário para reconstruir o ambiente, mantendo as três máquinas virtuais independentes e preservando a arquitetura utilizada pelos laboratórios.

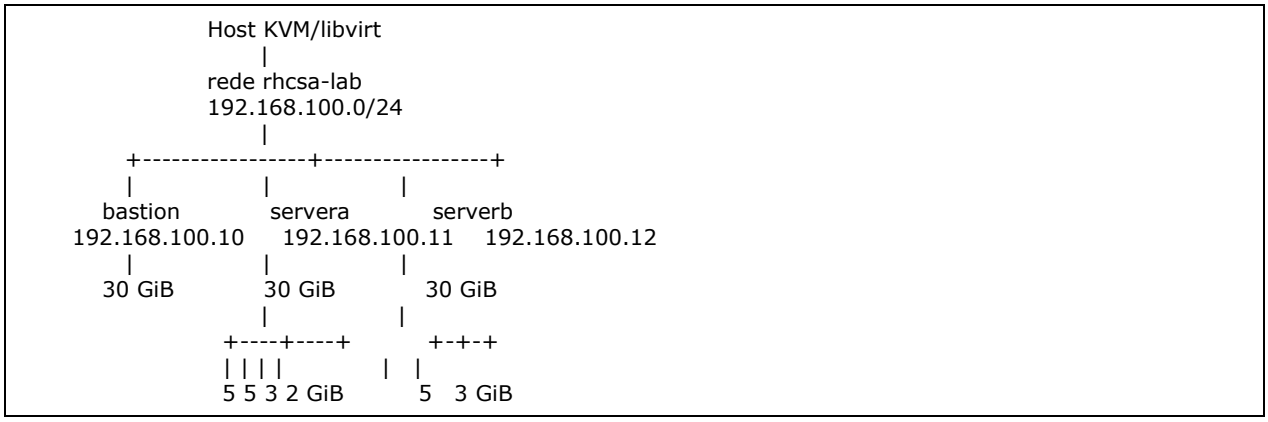
O uso da imagem-base não elimina as etapas de individualização das VMs. Cada máquina deverá possuir disco próprio, endereço MAC próprio, identidade de máquina própria e chaves SSH próprias.

3.2.1 Resultado esperado

Ao final deste método, o host KVM deverá possuir:

VM	RAM	vCPU	Disco do sistema	Discos adicionais
bastion	2 GiB	2	30 GiB	nenhum
servera	3 GiB	2	30 GiB	5 GiB, 5 GiB, 3 GiB e 2 GiB
serverb	2 GiB	2	30 GiB	5 GiB e 3 GiB

A arquitetura final será:



Os endereços IP definitivos e os hostnames serão configurados na etapa 04.

3.2.2 Onde executar

Local: host KVM/libvirt.

Os comandos desta página devem ser executados no hospedeiro das máquinas virtuais.

Não execute os comandos de manipulação de discos QCOW2 dentro de `bastion`, `servera` ou `serverb`.

Confirme o host:

```
hostnamectl
```

Confirme o libvirt:

```
sudo virsh -c qemu:///system list --all
```

3.2.3 Pré-requisitos

Antes de utilizar a imagem-base, confirme que a etapa de preparação do host foi concluída.

Devem estar disponíveis:

```
KVM/QEMU
libvirt
virt-install
qemu-img
rede rhcsa-lab
pool de armazenamento default
```

Confirme a rede:

```
sudo virsh -c qemu:///system net-list --all
```

Resultado esperado:

```
Name       State    Autostart   Persistent
----------------------------------------------
rhcsa-lab  active   yes         yes
```

Confirme o pool:

```
sudo virsh -c qemu:///system pool-list --all
```

O pool default deve estar ativo.

Confira também se os nomes das VMs estão disponíveis:

```
sudo virsh -c qemu:///system list --all
```

Não prossiga se já existirem máquinas chamadas:

```
bastion
servera
serverb
```

Este procedimento considera uma implantação nova.

3.2.4 Sobre a imagem-base

A imagem utilizada neste método é:

```
rhel-rhcsa-base-v1.qcow2
```

Ela possui um sistema RHEL previamente instalado e preparado para servir como origem para novas VMs.

O arquivo original deve ser tratado como **somente leitura**.

Nunca utilize a imagem-base diretamente como disco de uma VM de laboratório.

O fluxo correto é:

```
rhel-rhcsa-base-v1.qcow2
        |
        +---- cópia independente ----> bastion.qcow2
        |
        +---- cópia independente ----> servera.qcow2
        |
        +---- cópia independente ----> serverb.qcow2
```

As três VMs serão, portanto, independentes da imagem original.

3.2.5 Download da imagem

A distribuição pública da imagem-base deve observar os termos aplicáveis ao Red Hat Enterprise Linux.

A imagem-base e o arquivo de verificação SHA-256 estão disponíveis na pasta pública do projeto no Google Drive:

Baixar imagem-base do laboratório

```
rhel-rhcsa-base-v1.qcow2
rhel-rhcsa-base-v1.qcow2.sha256
```

Localize os dois arquivos no host KVM.

Exemplo:

```
cd ~/Downloads
ls -lh rhel-rhcsa-base-v1.qcow2*
```

Resultado esperado:

```
rhel-rhcsa-base-v1.qcow2
rhel-rhcsa-base-v1.qcow2.sha256
```

Não prossiga antes de validar a integridade da imagem.

3.2.6 Verificar o SHA-256

Objetivo: confirmar que o arquivo baixado corresponde à imagem publicada.

No diretório em que os dois arquivos foram armazenados:

```
cd ~/Downloads
```

Execute:

```
sha256sum -c rhel-rhcsa-base-v1.qcow2.sha256
```

Resultado esperado:

```
rhel-rhcsa-base-v1.qcow2: OK
```

Se aparecer:

```
FAILED
```

ou qualquer erro de checksum, não utilize a imagem.

Faça novamente o download e repita a verificação.

O checksum protege contra corrupção acidental do arquivo e também permite detectar se o arquivo baixado é diferente daquele utilizado para gerar a assinatura publicada pelo projeto.

3.2.7 Verificar a estrutura QCOW2

Além do checksum, valide a estrutura interna da imagem:

```
qemu-img check ~/Downloads/rhel-rhcsa-base-v1.qcow2
```

Resultado esperado:

```
No errors were found on the image.
```

Confira também as informações da imagem:

```
qemu-img info ~/Downloads/rhel-rhcsa-base-v1.qcow2
```

A imagem utilizada neste projeto possui tamanho virtual de aproximadamente:

```
30 GiB
```

O tamanho efetivamente ocupado no disco pode ser consideravelmente menor devido ao formato QCOW2 e à compressão.

3.2.8 Preservar uma cópia-mestre

É recomendável separar a imagem-base dos discos utilizados pelas VMs.

Crie um diretório específico:

```
sudo mkdir -p /var/lib/libvirt/base-images
```

Copie a imagem:

```
sudo cp \
  ~/Downloads/rhel-rhcsa-base-v1.qcow2 \
  /var/lib/libvirt/base-images/
```

Proteja a cópia contra alterações acidentais:

```
sudo chmod 0444 \
  /var/lib/libvirt/base-images/rhel-rhcsa-base-v1.qcow2
```

Confira:

```
ls -lh \
  /var/lib/libvirt/base-images/rhel-rhcsa-base-v1.qcow2
```

A partir deste ponto, utilize:

```
/var/lib/libvirt/base-images/rhel-rhcsa-base-v1.qcow2
```

como origem das novas VMs.

Não inicialize uma VM diretamente a partir desse arquivo.

3.2.9 Definir os caminhos utilizados pelo procedimento

Para reduzir erros de digitação, defina:

```
BASE_IMAGE=/var/lib/libvirt/base-images/rhel-rhcsa-base-v1.qcow2
IMAGE_DIR=/var/lib/libvirt/images
```

Confira:

```
echo "$BASE_IMAGE"
echo "$IMAGE_DIR"
```

Valide novamente:

```
sudo qemu-img check "$BASE_IMAGE"
```

3.2.10 Criar os discos independentes das três VMs

Neste projeto são utilizadas cópias completas e independentes.

Não são utilizados backing files ou linked clones neste procedimento.

Essa decisão torna cada VM independente da imagem-base e simplifica cópias, backups, snapshots e movimentação do laboratório entre hospedeiros.

Crie o disco do bastion:

```
sudo qemu-img convert \
  -p \
  -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/bastion.qcow2"
```

Crie o disco do servera:

```
sudo qemu-img convert \
  -p \
  -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/servera.qcow2"
```

Crie o disco do serverb:

```
sudo qemu-img convert \
  -p \
  -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/serverb.qcow2"
```

Atualize o pool:

```
sudo virsh -c qemu:///system pool-refresh default
```

3.2.11 Validar os três discos de sistema

Execute:

```
sudo qemu-img check "$IMAGE_DIR/bastion.qcow2"
sudo qemu-img check "$IMAGE_DIR/servera.qcow2"
sudo qemu-img check "$IMAGE_DIR/serverb.qcow2"
```

Os três comandos devem retornar:

```
No errors were found on the image.
```

Confira o tamanho virtual:

```
sudo qemu-img info "$IMAGE_DIR/bastion.qcow2"
sudo qemu-img info "$IMAGE_DIR/servera.qcow2"
sudo qemu-img info "$IMAGE_DIR/serverb.qcow2"
```

Cada disco deve apresentar aproximadamente:

```
virtual size: 30 GiB
```

3.2.12 Criar os discos adicionais do servera

Os exercícios de armazenamento utilizam discos separados do sistema operacional.

Crie:

```
sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/servera-vdb.qcow2" \
  5G

sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/servera-vdc.qcow2" \
  5G

sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/servera-vdd.qcow2" \
  3G

sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/servera-vde.qcow2" \
  2G
```

Esses discos devem permanecer vazios.

Não crie partições ou sistemas de arquivos neles.

3.2.13 Criar os discos adicionais do serverb

Execute:

```
sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/serverb-vdb.qcow2" \
  5G

sudo qemu-img create \
  -f qcow2 \
  "$IMAGE_DIR/serverb-vdc.qcow2" \
  3G
```

3.2.14 Ajustar contexto de segurança quando aplicável

Em hospedeiros RHEL, Rocky Linux ou outras distribuições que utilizem SELinux, restaure os contextos dos arquivos:

```
if command -v restorecon >/dev/null 2>&1; then
  sudo restorecon -RFv /var/lib/libvirt/images
fi
```

Não desabilite SELinux para fazer o laboratório funcionar.

Da mesma forma, em hospedeiros Ubuntu, não desabilite o AppArmor do libvirt.

> Não force *chown qemu:qemu* de forma indiscriminada. O usuário utilizado pelo QEMU varia entre distribuições. Deixe o libvirt gerenciar as permissões e os rótulos sempre que possível.

3.2.15 Criar a VM bastion

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name bastion \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/bastion.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confirme:

```
sudo virsh -c qemu:///system list --all
```

Confira o disco:

```
sudo virsh -c qemu:///system domblklist bastion
```

Confira a interface:

```
sudo virsh -c qemu:///system domiflist bastion
```

3.2.16 Criar a VM servera

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name servera \
  --memory 3072 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/servera.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdc.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdd.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vde.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confira:

```
sudo virsh -c qemu:///system domblklist servera
```

A ordem esperada é:

```
vda   disco do sistema   30 GiB
vdb   laboratório         5 GiB
vdc   laboratório         5 GiB
vdd   laboratório         3 GiB
vde   laboratório         2 GiB
```

3.2.17 Criar a VM serverb

Execute:

```
sudo virt-install \
  --connect qemu:///system \
  --name serverb \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/serverb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/serverb-vdb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/serverb-vdc.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Confira:

```
sudo virsh -c qemu:///system domblklist serverb
```

A ordem esperada é:

```
vda   disco do sistema   30 GiB
vdb   laboratório         5 GiB
vdc   laboratório         3 GiB
```

3.2.18 Confirmar as três máquinas virtuais

Execute:

```
sudo virsh -c qemu:///system list --all
```

Devem existir:

```
bastion
servera
serverb
```

Confira as interfaces:

```
sudo virsh -c qemu:///system domiflist bastion
sudo virsh -c qemu:///system domiflist servera
sudo virsh -c qemu:///system domiflist serverb
```

Cada interface deve possuir seu próprio endereço MAC.

Não configure o mesmo endereço MAC manualmente em duas VMs.

Quando o parâmetro `mac=` não é especificado, o libvirt gera endereços individuais para os domínios.

3.2.19 Primeiro boot

A imagem-base foi preparada para gerar novas identidades durante o primeiro boot.

Inicialize inicialmente uma VM de cada vez para facilitar a validação.

Comece pelo `bastion`:

```
sudo virsh -c qemu:///system start bastion
```

Confirme:

```
sudo virsh -c qemu:///system list
```

Depois faça o mesmo com:

```
sudo virsh -c qemu:///system start servera
sudo virsh -c qemu:///system start serverb
```

Se o virt-install já tiver deixado alguma VM em execução, não é necessário iniciá-la novamente.

3.2.20 Identificar o endereço DHCP temporário

Antes da configuração dos IPs definitivos, as VMs podem receber endereços do DHCP da rede rhcsa-lab.

Consulte:

```
sudo virsh -c qemu:///system net-dhcp-leases rhcsa-lab
```

Também é possível consultar individualmente:

```
sudo virsh -c qemu:///system domifaddr bastion
sudo virsh -c qemu:///system domifaddr servera
sudo virsh -c qemu:///system domifaddr serverb
```

Esses endereços são temporários.

Os endereços definitivos serão:

```
bastion   192.168.100.10/24
servera   192.168.100.11/24
serverb   192.168.100.12/24
```

Eles serão configurados na etapa 04.

3.2.21 Validar a identidade de cada sistema

A imagem-base não deve transportar a mesma identidade de máquina para todos os clones.

Dentro de cada VM, execute:

```
cat /etc/machine-id
```

Anote o valor de cada máquina.

Os três valores devem ser diferentes.

Exemplo conceitual:

```
bastion   <machine-id A>
servera   <machine-id B>
serverb   <machine-id C>
```

Nunca devem ser:

```
A = B = C
```

Se duas ou mais VMs apresentarem o mesmo machine-id, interrompa a implantação e não avance para os laboratórios.

3.2.22 Validar as chaves SSH do host

Dentro de cada VM:

```
sudo ls -l /etc/ssh/ssh_host_*
```

As chaves devem existir após o primeiro boot.

Consulte a impressão digital da chave ED25519:

```
sudo ssh-keygen \
  -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Repita nas três máquinas.

As fingerprints devem ser diferentes.

Isso garante que as VMs não compartilham a mesma identidade SSH.

Se as chaves não existirem:

```
sudo ssh-keygen -A
sudo systemctl restart sshd
```

Depois repita a validação.

3.2.23 Verificar o estado do Red Hat Subscription Management

A imagem-base não deve transportar uma identidade ativa de assinatura pertencente ao sistema utilizado para criá-la.

Dentro de cada VM:

```
sudo subscription-manager identity
```

Antes do registro individual, não deve existir uma identidade RHSM herdada da imagem-base.

Se uma identidade válida do sistema original aparecer, não considere a imagem corretamente sanitizada.

Não publique nem replique essa imagem até corrigir o processo de preparação.

3.2.24 Registrar cada VM individualmente, quando necessário

O laboratório pode utilizar:

- repositórios autorizados através do Red Hat Subscription Management; ou
- a mídia DVD local, conforme descrito na etapa 04.

Se optar pelo Red Hat Subscription Management, registre **cada VM individualmente**.

Não registre a imagem-base.

Exemplo interativo:

```
sudo subscription-manager register
```

Após o registro:

```
sudo subscription-manager identity
```

Cada VM deve possuir sua própria identidade.

Não inclua nomes de usuário, senhas, tokens ou activation keys no repositório Git.

Se o laboratório utilizar exclusivamente o repositório local da ISO/DVD, prossiga para a etapa 04 sem registrar as VMs neste momento.

3.2.25 Validar os discos dentro do bastion

No bastion:

```
lsblk
```

Resultado esperado:

```
vda    30G
```

Confira também:

```
lsblk -f
```

3.2.26 Validar os discos dentro do servera

No servera:

```
lsblk
```

Esperado:

```
vda    30G
vdb     5G
vdc     5G
vdd     3G
vde     2G
```

Os discos:

```
vdb
vdc
vdd
vde
```

devem permanecer disponíveis para os laboratórios.

Confira:

```
lsblk -f
```

Eles não devem conter sistemas de arquivos criados pela implantação.

3.2.27 Validar os discos dentro do serverb

No serverb:

```
lsblk
```

Esperado:

```
vda    30G
vdb     5G
vdc     3G
```

Confira:

```
lsblk -f
```

Os discos adicionais devem permanecer sem uso.

3.2.28 Não alterar a imagem-base depois da implantação

Depois que as cópias forem criadas, a imagem:

```
/var/lib/libvirt/base-images/rhel-rhcsa-base-v1.qcow2
```

não participa da execução das VMs.

Essa é uma característica intencional do projeto.

As VMs utilizam:

```
/var/lib/libvirt/images/bastion.qcow2
/var/lib/libvirt/images/servera.qcow2
/var/lib/libvirt/images/serverb.qcow2
```

Por isso, alterações futuras em uma VM não modificam a imagem-base.

Também significa que a imagem-base pode ser utilizada novamente para reconstruir o laboratório.

3.2.29 Verificar que não existem backing files

O Método B utiliza discos completos e independentes.

Confira:

```
sudo qemu-img info --backing-chain \
  /var/lib/libvirt/images/bastion.qcow2
```

Repita:

```
sudo qemu-img info --backing-chain \
  /var/lib/libvirt/images/servera.qcow2

sudo qemu-img info --backing-chain \
  /var/lib/libvirt/images/serverb.qcow2
```

Cada comando deve apresentar somente o próprio disco.

Não deve existir dependência da imagem:

```
rhel-rhcsa-base-v1.qcow2
```

3.2.30 Problemas comuns

A imagem falhou na verificação SHA-256

Não utilize o arquivo.

Remova a cópia incorreta e realize novamente o download.

Depois:

```
sha256sum -c rhel-rhcsa-base-v1.qcow2.sha256
```

Só prossiga quando o resultado for:

```
OK
```

qemu-img check encontrou erros

Não tente construir as VMs a partir de uma imagem estruturalmente inconsistente.

Utilize outra cópia da imagem-base.

A VM já existe

Confira:

```
sudo virsh -c qemu:///system list --all
```

Não sobrescreva uma VM existente sem verificar se ela contém dados ou snapshots que precisam ser preservados.

A rede ***rhcsa-lab*** não existe

Confira:

```
sudo virsh -c qemu:///system net-list --all
```

Retorne para:

02 — Preparação do host KVM/libvirt

Erro de permissão ao acessar o QCOW2

Confira:

```
ls -lhZ /var/lib/libvirt/images/
```

Em host com SELinux:

```
sudo restorecon -RFv /var/lib/libvirt/images
```

Não desabilite SELinux para contornar o problema.

Em Ubuntu, consulte também os logs do libvirt e as políticas AppArmor antes de modificar permissões manualmente.

O host não suporta SPICE

Este procedimento utiliza VNC:

```
--graphics vnc,listen=127.0.0.1
```

Não é necessário utilizar SPICE para executar o laboratório.

virsh domifaddr não mostra o endereço

Consulte os leases diretamente:

```
sudo virsh -c qemu:///system net-dhcp-leases rhcsa-lab
```

Também é possível acessar o console da VM através de `virt-manager`, `virt-viewer` ou Cockpit.

As SSH host keys são iguais

Não avance.

A imagem ou os clones não foram individualizados corretamente.

Confira:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

em cada VM.

O ***machine-id*** é igual em mais de uma VM

Não avance.

O arquivo:

```
/etc/machine-id
```

deve representar uma identidade individual por máquina.

Uma imagem destinada a clonagem deve ser preparada antes da geração das cópias.

O Subscription Manager mostra a identidade da VM original

Não considere essa imagem apta para distribuição ou clonagem.

A identidade RHSM da máquina utilizada para construir o template precisa ser removida antes da publicação da imagem-base.

3.2.31 Como a imagem-base é preparada

Esta seção documenta o processo utilizado pelo mantenedor do projeto para construir uma imagem destinada à clonagem.

Ela não precisa ser repetida por quem apenas utiliza uma imagem-base já validada.

Antes da sanitização, o sistema utilizado como origem deve ser desregistrado:

```
sudo subscription-manager unregister
sudo subscription-manager clean
```

Quando aplicável, remova também conexões adicionais associadas à máquina original.

A VM é então desligada.

No host KVM, a sanitização pode ser realizada com virt-sysprep.

O procedimento utilizado na preparação desta imagem inclui:

```
sudo virt-sysprep \
  -a /var/lib/libvirt/images/rhel-rhcsa-sanitize.qcow2 \
  --operations \
bash-history,dhcp-client-state,logfiles,machine-id,net-hostname,ssh-hostkeys,ssh-userdir,tmp-files,utmp
```

As operações utilizadas removem informações que não devem ser replicadas para as novas VMs, incluindo:

```
histórico de shell
estado antigo de DHCP
logs da máquina de origem
machine-id
hostname da máquina de origem
SSH host keys
dados SSH dos usuários
arquivos temporários
registros de login
```

Após o virt-sysprep, a imagem não deve ser inicializada novamente antes da geração da versão distribuível.

Valide:

```
sudo qemu-img check \
  /var/lib/libvirt/images/rhel-rhcsa-sanitize.qcow2
```

A versão final pode ser produzida com:

```
sudo qemu-img convert \
  -p \
  -O qcow2 \
  -c \
  /var/lib/libvirt/images/rhel-rhcsa-sanitize.qcow2 \
  rhel-rhcsa-base-v1.qcow2
```

Em seguida:

```
qemu-img check rhel-rhcsa-base-v1.qcow2
```

E:

```
sha256sum rhel-rhcsa-base-v1.qcow2 \
  > rhel-rhcsa-base-v1.qcow2.sha256
```

A imagem e o arquivo `.sha256` formam o conjunto mínimo necessário para uma distribuição verificável.

3.2.32 Segurança da imagem-base

Antes da publicação de uma nova versão da imagem, o mantenedor deve confirmar que ela não contém:

```
chaves SSH privadas pessoais
authorized_keys pessoais
tokens de GitHub
credenciais de nuvem
senhas armazenadas em arquivos de configuração
activation keys
tokens RHSM
identidade de assinatura da VM de origem
histórico contendo segredos
arquivos pessoais
dados de outros projetos
```

Arquivos de imagem RHEL e mídias ISO não devem ser adicionados ao Git.

O Git deve armazenar apenas documentação, automação e código do projeto.

3.2.33 Compatibilidade com a versão atual do laboratório

A imagem-base descrita nesta documentação utiliza:

```
RHEL 9.8
```

como sistema operacional local.

O projeto utiliza RHEL 10 como referência de conteúdo para estudo e classificação das competências.

A utilização de RHEL 9.8 localmente não significa que todos os comportamentos sejam idênticos ao RHEL 10.

Os exercícios do projeto devem continuar respeitando a classificação de compatibilidade documentada no repositório.

3.2.34 Critério para concluir o Método B

Antes de prosseguir, confirme:

```
[ ] checksum SHA-256 da imagem validado
[ ] qemu-img check sem erros

[ ] imagem-base preservada e não utilizada diretamente por uma VM

[ ] bastion possui disco independente
[ ] servera possui disco independente
[ ] serverb possui disco independente

[ ] não existem backing files entre os discos das VMs e a imagem-base

[ ] bastion existe no libvirt
[ ] servera existe no libvirt
[ ] serverb existe no libvirt

[ ] cada VM possui endereço MAC diferente
[ ] cada VM possui machine-id diferente
[ ] cada VM possui SSH host keys diferentes

[ ] bastion possui disco de sistema de 30 GiB

[ ] servera possui:
   30 GiB
    5 GiB
    5 GiB
    3 GiB
    2 GiB

[ ] serverb possui:
   30 GiB
    5 GiB
    3 GiB

[ ] discos adicionais permanecem sem uso
[ ] nenhuma identidade RHSM antiga foi herdada da imagem-base
[ ] nenhuma credencial pessoal foi incorporada à imagem
```

Se todos os itens forem atendidos, o Método B está concluído.

A próxima etapa configura os nomes e os endereços IP definitivos das três VMs.

Continue para:

04 — Configuração inicial das VMs

Criação das VMs · Índice · Home · Anterior · Próximo

8. Configuração inicial das VMs

Índice · Home · Anterior · Próximo

Onde executar: principalmente no console de cada VM, como `student` com `sudo`.

Esta etapa é comum aos dois métodos de criação das máquinas virtuais:

- **Método A — instalação manual a partir da ISO do RHEL**;
- **Método B — implantação a partir da imagem-base QCOW2**.

Os dois métodos devem chegar aqui com as três VMs existentes no libvirt:

```
bastion
servera
serverb
```

O objetivo desta etapa é transformar essas VMs em um ambiente consistente, com identidade, rede, resolução de nomes, SSH, firewall, SELinux e acesso a pacotes corretamente configurados.

4.1 Resultado esperado

Ao final desta etapa:

VM	Hostname	IPv4
bastion	bastion.lab.test	192.168.100.10/24
servera	servera.lab.test	192.168.100.11/24
serverb	serverb.lab.test	192.168.100.12/24

Parâmetros comuns:

```
Gateway:    192.168.100.1
DNS:       192.168.100.1
Domínio:    lab.test
Rede:      192.168.100.0/24
```

Também devem estar funcionando:

```
student
sudo
sshd
firewalld
SELinux Enforcing
dnf
resolução de nomes entre as VMs
```

4.2 Antes de alterar a rede

A mudança de DHCP para endereço estático pode interromper uma sessão SSH em andamento.

Por isso, faça a configuração inicial de rede preferencialmente pelo:

- console da VM no Cockpit;
- virt-manager;
- virt-viewer;
- outro console local fornecido pelo libvirt.

Evite depender exclusivamente do SSH enquanto altera o perfil de rede.

Em cada VM, confirme o estado atual:

```
hostnamectl
ip -br address
ip route
nmcli device status
nmcli connection show
```

4.3 Identificar o perfil de rede correto

Não suponha que a interface seja `eth0` nem que o perfil tenha um nome específico.

Primeiro identifique:

```
nmcli -f NAME,UUID,TYPE,DEVICE connection show
```

Exemplo:

```
NAME            UUID                                  TYPE      DEVICE
Wired connection 1  xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  ethernet  enp1s0
```

O nome da interface pode variar entre instalações.

O importante é identificar o **perfil ativo ligado à interface conectada à rede rhcsa-lab**.

Nos exemplos abaixo, substitua:

```
NOME_DA_CONEXAO
```

pelo nome real mostrado por `nmcli`.

4.4 Configurar o bastion

Entre no console do `bastion`.

4.4.1 Definir o hostname

```
sudo hostnamectl set-hostname bastion.lab.test
```

Confira:

```
hostnamectl
```

4.4.2 Identificar o perfil de rede

```
nmcli connection show
```

Defina uma variável com o nome real do perfil.

Exemplo:

```
LAB_CONNECTION='Wired connection 1'
```

Confirme:

```
printf '%s\n' "$LAB_CONNECTION"
```

4.4.3 Configurar IPv4 estático

```
sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.10/24 \
```

```
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes
```

Reative o perfil:

```
sudo nmcli connection up "$LAB_CONNECTION"
```

4.4.4 Validar

```
hostname -f
ip -4 -br address
ip route
```

O endereço esperado é:

```
192.168.100.10/24
```

4.5 Configurar o servera

Entre no console do servera.

4.5.1 Definir o hostname

```
sudo hostnamectl set-hostname servera.lab.test
```

4.5.2 Identificar o perfil

```
nmcli connection show
```

Defina a variável com o nome real.

Exemplo:

```
LAB_CONNECTION='Wired connection 1'
```

4.5.3 Configurar IPv4 estático

```
sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.11/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes
```

Reative:

```
sudo nmcli connection up "$LAB_CONNECTION"
```

4.5.4 Validar

```
hostname -f
ip -4 -br address
ip route
```

O endereço esperado é:

```
192.168.100.11/24
```

4.6 Configurar o serverb

Entre no console do serverb.

4.6.1 Definir o hostname

```
sudo hostnamectl set-hostname serverb.lab.test
```

4.6.2 Identificar o perfil

```
nmcli connection show
```

Defina a variável com o nome real.

Exemplo:

```
LAB_CONNECTION='Wired connection 1'
```

4.6.3 Configurar IPv4 estático

```
sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.12/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes
```

Reative:

```
sudo nmcli connection up "$LAB_CONNECTION"
```

4.6.4 Validar

```
hostname -f
ip -4 -br address
ip route
```

O endereço esperado é:

```
192.168.100.12/24
```

4.7 Configurar resolução local de nomes

Mesmo com o DNS fornecido pelo libvirt, este laboratório mantém uma tabela local explícita para tornar a resolução entre as VMs previsível.

Em **cada uma das três VMs**, edite:

```
sudoedit /etc/hosts
```

Preserve as entradas de localhost.

Adicione uma única vez:

```
192.168.100.10 bastion.lab.test bastion
192.168.100.11 servera.lab.test servera
192.168.100.12 serverb.lab.test serverb
```

Não duplique essas entradas.

4.7.1 Validar a resolução

Em cada VM:

```
getent hosts bastion
getent hosts servera
getent hosts serverb
```

Exemplo esperado:

```
192.168.100.11  servera.lab.test servera
```

Também confira o FQDN local:

```
hostname -f
```

4.8 Validar identidades individuais das VMs

Esta validação é especialmente importante quando o ambiente foi criado pelo **Método B**.

Em cada VM:

```
cat /etc/machine-id
```

Os valores de bastion, servera e serverb devem ser diferentes.

Não avance se duas VMs apresentarem o mesmo machine-id.

4.8.1 Validar as SSH host keys

Em cada VM:

```
sudo ls -l /etc/ssh/ssh_host_*
```

Confira a fingerprint da chave ED25519:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

As três VMs devem possuir fingerprints diferentes.

Se uma VM não possuir SSH host keys:

```
sudo ssh-keygen -A
```

Depois:

```
sudo systemctl restart sshd
```

4.9 Validar o usuário **student**

O projeto utiliza o usuário:

```
student
```

em todas as VMs.

Confira:

```
id student
```

O usuário deve existir e pertencer ao grupo administrativo wheel.

Exemplo:

```
groups=...,wheel
```

Se o usuário ainda não existir:

```
sudo useradd -m -G wheel student
```

Defina uma senha local:

```
sudo passwd student
```

Se o usuário já existir, não o recrie.

Confira o acesso ao sudo:

```
sudo -v
```

A configuração NOPASSWD necessária para a automação dos laboratórios será tratada posteriormente, na etapa de validação do Ansible.

Não publique senhas locais no repositório Git.

4.10 Habilitar SSH

Em cada VM:

```
sudo systemctl enable --now sshd
```

Confira:

```
systemctl is-active sshd
```

Resultado esperado:

```
active
```

Confira também:

```
ss -lnt | grep ':22'
```

4.11 Manter SELinux habilitado

Este laboratório utiliza SELinux como parte dos exercícios RHCSA.

Confira:

```
getenforce
```

Resultado esperado:

```
Enforcing
```

Se o resultado for Permissive ou Disabled, investigue a configuração antes de continuar.

O laboratório não deve ser preparado desabilitando SELinux para contornar problemas.

Confira também:

```
sestatus
```

4.12 Habilitar e configurar o firewalld

Em cada VM:

```
sudo systemctl enable --now firewalld
```

Confira:

```
systemctl is-active firewalld
```

Autorize SSH:

```
sudo firewall-cmd --permanent --add-service=ssh
```

Recarregue:

```
sudo firewall-cmd --reload
```

Valide:

```
sudo firewall-cmd --list-services
```

O serviço ssh deve aparecer na zona efetivamente utilizada pela interface.

Confira a zona ativa:

```
sudo firewall-cmd --get-active-zones
```

Não desabilite o firewall para simplificar o laboratório.

4.13 Escolher a origem dos pacotes RHEL

As três VMs precisam de uma origem funcional de pacotes para que as etapas seguintes possam instalar dependências.

Utilize **uma fonte coerente com sua instalação**.

As opções documentadas são:

1. Red Hat Subscription Management;
2. mídia DVD local compatível com a versão instalada.

Não misture repositórios de versões diferentes do RHEL.

4.14 Opção A — Red Hat Subscription Management

Use esta opção quando as VMs puderem ser registradas individualmente por meio de uma assinatura ou acesso autorizado.

4.14.1 Verificar o estado atual

Em cada VM:

```
sudo subscription-manager identity
```

Se o sistema ainda não estiver registrado, o comando informará que não existe identidade válida.

4.14.2 Registrar a VM

Execute de forma interativa:

```
sudo subscription-manager register
```

Não armazene usuário, senha, activation key ou token em arquivos versionados pelo projeto.

4.14.3 Confirmar a identidade

```
sudo subscription-manager identity
```

Se estiver utilizando o Método B, cada VM deve possuir sua própria identidade.

Nunca registre a imagem-base que será utilizada como template.

4.14.4 Conferir repositórios

```
sudo dnf repolist
```

Confirme que há repositórios adequados à versão instalada antes de prosseguir.

4.15 Opção B — repositório local pela ISO/DVD

Esta opção utiliza a mídia DVD correspondente à mesma versão do RHEL instalada na VM.

4.15.1 Método A

Quando as VMs foram instaladas pela ISO, o CD-ROM virtual pode já estar associado à máquina.

Confira:

```
lsblk
```

O dispositivo costuma aparecer como:

```
sr0
```

mas o nome deve ser validado no seu sistema.

4.15.2 Método B

As VMs criadas pela imagem-base não precisam ter um CD-ROM virtual conectado.

Se você optar pelo repositório local, anexe uma ISO oficial e compatível à VM antes de continuar.

Essa ação deve ser feita no **host KVM**, pelo Cockpit, `virt-manager` ou libvirt.

Depois retorne ao console da VM e confirme:

```
lsblk
```

4.15.3 Montar a mídia

Em cada VM que utilizar o DVD:

```
sudo mkdir -p /mnt/rhel-dvd
```

Monte somente leitura:

```
sudo mount -o ro /dev/sr0 /mnt/rhel-dvd
```

Valide a estrutura:

```
ls /mnt/rhel-dvd/BaseOS/repodata
ls /mnt/rhel-dvd/AppStream/repodata
```

Os dois caminhos devem existir.

4.15.4 Configurar montagem persistente

Após confirmar que `/dev/sr0` é realmente o dispositivo correto, adicione uma única entrada a `/etc/fstab`:

```
/dev/sr0 /mnt/rhel-dvd iso9660 ro,nofail 0 0
```

Não duplique a entrada.

Teste:

```
sudo umount /mnt/rhel-dvd
sudo mount -a
```

Confirme:

```
mount | grep '/mnt/rhel-dvd'
```

4.15.5 Criar os repositórios locais

Crie:

```
sudoedit /etc/yum.repos.d/rhcsa-dvd.repo
```

Conteúdo:

```
[rhcsa-dvd-baseos]
name=RHCSA DVD BaseOS
baseurl=file:///mnt/rhel-dvd/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

[rhcsa-dvd-appstream]
name=RHCSA DVD AppStream
baseurl=file:///mnt/rhel-dvd/AppStream
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

Confira se a chave indicada existe:

```
ls -l /etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

Atualize o cache:

```
sudo dnf clean all

sudo dnf repolist
```

Os repositórios locais devem aparecer habilitados.

> O DVD fornece os pacotes existentes na mídia. Ele não substitui os repositórios de atualização da Red Hat.

4.16 Instalar o conjunto básico de pacotes

Com uma fonte de pacotes funcional, instale em cada VM:

```
sudo dnf install -y \
  openssh-server \
  openssh-clients \
  python3 \
  tar \
  gzip \
  bzip2 \
  xz \
  unzip \
  zip
```

Valide:

```
python3 --version
ssh -V
tar --version | head -1
```

O python3 é necessário para a execução dos módulos Ansible nas VMs gerenciadas.

4.17 Validar conectividade com o gateway

Em cada VM:

```
ping -c 2 192.168.100.1
```

A resposta do gateway confirma conectividade básica com a bridge da rede rhcsa-lab.

Um ping bem-sucedido, porém, não substitui os testes de resolução de nomes e SSH.

4.18 Validar comunicação entre as VMs

No bastion:

```
getent hosts servera
getent hosts serverb
```

Teste conectividade IP:

```
ping -c 2 servera
ping -c 2 serverb
```

Teste a porta SSH:

```
ssh -o ConnectTimeout=5 student@servera
```

Na primeira conexão, confira a fingerprint do host antes de aceitar.

Saia:

```
exit
```

Repita:

```
ssh -o ConnectTimeout=5 student@serverb
```

Saia:

```
exit
```

A autenticação automatizada por chave será configurada na etapa seguinte.

4.19 Validar a topologia dos discos

Os discos adicionais fazem parte dos exercícios e não devem ser utilizados durante a preparação inicial.

4.19.1 bastion

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Esperado:

```
vda   30G
```

4.19.2 servera

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Esperado:

```
vda   30G
vdb    5G
vdc    5G
vdd    3G
vde    2G
```

Os discos vdb, vdc, vdd e vde devem estar disponíveis para os laboratórios.

4.19.3 serverb

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Esperado:

```
vda   30G
vdb    5G
vdc    3G
```

Os discos adicionais não devem estar montados nem conter estruturas criadas durante a preparação do ambiente.

4.20 Validação consolidada por VM

Em cada máquina, execute:

```
hostname -f
ip -4 -br address
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-services
dnf repolist
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

4.21 Valores esperados

bastion

```
Hostname:  bastion.lab.test
IPv4:      192.168.100.10/24
Gateway:   192.168.100.1
SELinux:   Enforcing
SSH:       active
Firewall:  active
```

servera

```
Hostname:  servera.lab.test
IPv4:      192.168.100.11/24
Gateway:   192.168.100.1
SELinux:   Enforcing
SSH:       active
Firewall:  active
```

serverb

```
Hostname:  serverb.lab.test
IPv4:      192.168.100.12/24
Gateway:   192.168.100.1
SELinux:   Enforcing
SSH:       active
Firewall:  active
```

4.22 Problemas comuns

nmcli connection up interrompeu o acesso

Isso pode acontecer ao trocar DHCP por endereço estático.

Use o console da VM para validar:

```
ip -4 -br address
ip route
```

Depois tente novamente o acesso pelo novo endereço.

hostname -f não mostra ***lab.test***

Confira:

```
hostnamectl
```

Depois:

```
cat /etc/hosts
```

Garanta que a entrada da própria VM contém o FQDN correto.

Uma VM não resolve ***servera*** ou ***serverb***

Confira:

```
getent hosts servera
```

Depois:

```
cat /etc/hosts
```

Verifique também:

```
cat /etc/resolv.conf
```

SSH não responde

Confira:

```
systemctl status sshd --no-pager
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-services
```

```
ss -lnt | grep ':22'
```

Não desabilite o firewall como solução.

dnf não encontra pacotes

Confira:

```
sudo dnf repolist
```

Se estiver utilizando DVD:

```
mount | grep '/mnt/rhel-dvd'
```

Confira:

```
ls /mnt/rhel-dvd/BaseOS/repodata
ls /mnt/rhel-dvd/AppStream/repodata
```

Se estiver utilizando RHSM:

```
sudo subscription-manager identity
```

Não combine repositórios de versões diferentes do RHEL.

SELinux não está Enforcing

Confira:

```
getenforce
sestatus
```

Não avance simplesmente colocando o sistema em modo permissivo.

Os discos extras possuem partições ou sistemas de arquivos

Não utilize esses discos em outras tarefas antes dos laboratórios.

Confira:

```
lsblk -f
```

Se a topologia estiver diferente da documentação, investigue antes de criar o baseline.

4.23 Critério para avançar

Antes de continuar para a preparação do bastion, confirme:

```
[ ] bastion.lab.test configurado
[ ] servera.lab.test configurado
[ ] serverb.lab.test configurado

[ ] bastion usa 192.168.100.10/24
[ ] servera usa 192.168.100.11/24
[ ] serverb usa 192.168.100.12/24

[ ] gateway 192.168.100.1 configurado nas três VMs
[ ] resolução bastion/servera/serverb funcionando

[ ] usuário student existe nas três VMs
[ ] student possui acesso administrativo via sudo

[ ] machine-id é diferente nas três VMs
[ ] SSH host keys são diferentes nas três VMs

[ ] sshd está ativo
```

```
[ ] firewalld está ativo
[ ] serviço SSH está autorizado no firewall
[ ] SELinux está Enforcing

[ ] existe uma fonte funcional de pacotes
[ ] python3 está instalado

[ ] discos adicionais de servera permanecem disponíveis
[ ] discos adicionais de serverb permanecem disponíveis

[ ] bastion alcança servera pela rede
[ ] bastion alcança serverb pela rede
```

A autenticação SSH por chave e a clonagem do projeto serão configuradas na etapa seguinte.

Continue para:

05 — Preparação do bastion e clonagem

Índice · Home · Anterior · Próximo

9. Preparação do bastion, Git e SSH

Índice · Home · Anterior · Próximo

Onde executar: bastion, conectado como usuário student, exceto quando o texto indicar explicitamente outro local.

Esta etapa transforma o bastion na estação de controle do laboratório.

Ao final, o bastion deverá possuir:

- Git;
- Ansible Core;
- Python 3 e PyYAML;
- cliente OpenSSH;
- uma cópia limpa do projeto;
- uma chave SSH dedicada ao laboratório;
- acesso SSH não interativo a servera e serverb;
- o comando lab instalado no perfil do usuário student.

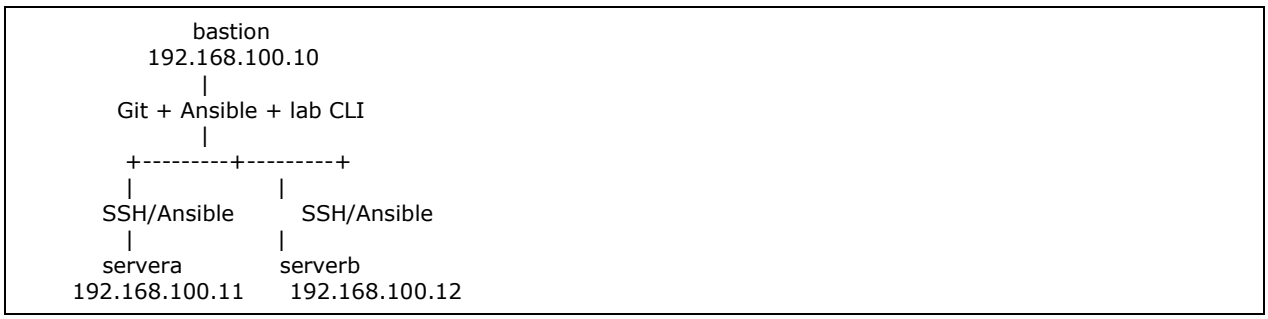
> Não execute a clonagem como *root*. O projeto foi estruturado para permanecer em */home/student/rhcsa-ex200-v10-lab*.

5.1 Resultado esperado

A estrutura principal deverá ficar semelhante a:

```
/home/student/
├── .local/
│   └── bin/
│       └── lab -> /home/student/rhcsa-ex200-v10-lab/bin/lab
├── .ssh/
│   ├── config
│   ├── id_ed25519_rhcsa_lab
│   ├── id_ed25519_rhcsa_lab.pub
│   └── known_hosts
└── rhcsa-ex200-v10-lab/
    ├── ansible/
    ├── bin/
    ├── bootstrap/
    ├── docs/
    ├── labctl/
    ├── labs/
    └── tests/
```

O fluxo de administração será:



5.2 Confirmar que está no bastion

Antes de instalar qualquer componente:

```
hostname -f
```

Resultado esperado:

```
bastion.lab.test
```

Confirme também o usuário:

```
id
```

O usuário esperado é:

```
student
```

Confira o diretório pessoal:

```
printf '%s\n' "$HOME"
```

Resultado esperado:

```
/home/student
```

Se esses valores não corresponderem ao ambiente esperado, não continue até identificar em qual máquina e usuário a sessão está sendo executada.

5.3 Validar rede e resolução de nomes

Antes de instalar dependências ou configurar SSH, confirme que o `bastion` alcança os dois hosts gerenciados.

Resolva os nomes:

```
getent hosts servera
getent hosts serverb
```

Resultados esperados:

```
192.168.100.11  servera.lab.test servera
192.168.100.12  serverb.lab.test serverb
```

Teste a conectividade:

```
ping -c 2 servera
ping -c 2 serverb
```

A ausência de resposta ICMP não deve ser confundida com uma falha de SSH, mas neste laboratório os testes devem ser investigados antes de avançar.

Confira a porta 22:

```
timeout 5 bash -c '</dev/tcp/servera/22' && echo "servera: SSH acessível"
timeout 5 bash -c '</dev/tcp/serverb/22' && echo "serverb: SSH acessível"
```

Se algum teste falhar, retorne à etapa:

04 — Configuração inicial das VMs

5.4 Confirmar a origem de pacotes

Antes de instalar as ferramentas do `bastion`, confirme que o DNF possui uma origem funcional:

```
sudo dnf repolist
```

O comando deve apresentar repositórios habilitados, provenientes de:

- Red Hat Subscription Management; ou
- mídia DVD local configurada na etapa anterior.

Não avance se o `dnf` estiver sem uma origem válida de pacotes.

5.5 Instalar as dependências do bastion

Instale:

```
sudo dnf install -y \
  git \
  ansible-core \
  python3 \
  python3-pyyaml \
  openssh-clients
```

A instalação fornece as principais ferramentas utilizadas pelo projeto.

Pacote	**Finalidade**
git	Clonar e atualizar o repositório
ansible-core	Preparar, restaurar e validar os cenários
python3	Executar o CLI e módulos Ansible
python3-pyyaml	Ler os arquivos YAML utilizados pelo projeto
openssh-clients	Acesso SSH aos hosts gerenciados

5.6 Validar as dependências

Confira o Git:

```
git --version
```

Confira o Ansible:

```
ansible --version
```

Confira o Python:

```
python3 --version
```

Valide o módulo PyYAML:

```
python3 -c 'import yaml; print("PyYAML OK")'
```

Resultado esperado:

```
PyYAML OK
```

Confira o cliente SSH:

```
ssh -V
```

Se algum comando não estiver disponível, resolva a instalação antes de continuar.

5.7 Confirmar que o diretório do projeto ainda não existe

O caminho esperado pelo projeto é:

```
/home/student/rhcsa-ex200-v10-lab
```

Confira:

```
ls -ld ~/rhcsa-ex200-v10-lab 2>/dev/null || true
```

Se o diretório não existir

Prossiga para a clonagem.

Se o diretório já existir

Não execute `git clone` por cima dele.

Entre no diretório:

```
cd ~/rhcsa-ex200-v10-lab
```

Confira:

```
git status
git remote -v
git log -1 --oneline --decorate
```

Se houver alterações locais, preserve-as antes de qualquer atualização.

Este guia não orienta apagar alterações locais automaticamente.

5.8 Clonar o repositório público

Se o projeto ainda não existir no `bastion`, execute:

```
cd ~
```

Clone via HTTPS:

```
git clone https://github.com/eduardoandradej/rhcsa-ex200-v10-lab.git
```

Entre no diretório:

```
cd ~/rhcsa-ex200-v10-lab
```

O clone público via HTTPS não exige credenciais do GitHub para leitura.

Credenciais de GitHub somente são necessárias para operações de escrita, como `push`, e não fazem parte da preparação normal do estudante.

5.9 Validar a cópia do projeto

Confira o diretório atual:

```
pwd
```

Resultado esperado:

```
/home/student/rhcsa-ex200-v10-lab
```

Confira o repositório remoto:

```
git remote -v
```

A origem deve apontar para:

```
https://github.com/eduardoandradej/rhcsa-ex200-v10-lab.git
```

Confira a branch:

```
git branch --show-current
```

Confira o último commit:

```
git log -1 --oneline --decorate
```

Confira o estado da árvore:

```
git status
```

Em uma clonagem nova, o esperado é:

```
nothing to commit, working tree clean
```

Também é útil identificar a versão ou tag associada ao commit atual:

```
git describe --tags --always
```

> A documentação do projeto evolui com o repositório. Quando estiver reproduzindo um ambiente específico, registre o commit ou a tag utilizada.

5.10 Conferir a estrutura essencial do repositório

No diretório do projeto:

```
find . -maxdepth 2 -type d \
  -not -path './.git*' \
  | sort
```

Confira especificamente:

```
test -d ansible && echo "ansible: OK"
test -d bootstrap && echo "bootstrap: OK"
test -d labctl && echo "labctl: OK"
test -d labs && echo "labs: OK"
test -d tests && echo "tests: OK"
test -f bin/lab && echo "bin/lab: OK"
```

Todos os itens devem existir.

Se algum deles estiver ausente em uma clonagem nova, confirme se o repositório correto foi clonado e se a operação terminou sem erros.

5.11 Separar a chave do laboratório das demais chaves SSH

O projeto utiliza uma chave SSH dedicada para o tráfego entre:

```
bastion -> servera
bastion -> serverb
```

O arquivo utilizado pelo inventário atual é:

```
/home/student/.ssh/id_ed25519_rhcsa_lab
```

Não reutilize uma chave pessoal do GitHub, de produção ou de outro ambiente.

Crie o diretório SSH, caso ainda não exista:

```
mkdir -p ~/.ssh
```

Ajuste a permissão:

```
chmod 700 ~/.ssh
```

5.12 Criar a chave SSH dedicada

Primeiro confira se a chave já existe:

```
ls -l ~/.ssh/id_ed25519_rhcsa_lab* 2>/dev/null || true
```

Se **não existir**, crie:

```
ssh-keygen \
  -t ed25519 \
  -f ~/.ssh/id_ed25519_rhcsa_lab \
  -C rhcsa-lab
```

O ssh-keygen solicitará uma passphrase.

Para um laboratório automatizado, o usuário pode optar por uma chave sem passphrase ou utilizar ssh-agent.

O projeto não armazena nem distribui a chave privada.

Permissões esperadas

Ajuste:

```
chmod 600 ~/.ssh/id_ed25519_rhcsa_lab
chmod 644 ~/.ssh/id_ed25519_rhcsa_lab.pub
```

Confira:

```
ls -l ~/.ssh/id_ed25519_rhcsa_lab*
```

A chave privada deve ser legível apenas pelo proprietário.

5.13 Conferir a fingerprint da chave do bastion

Execute:

```
ssh-keygen -lf ~/.ssh/id_ed25519_rhcsa_lab.pub
```

Esse valor identifica a chave pública utilizada pelo bastion para autenticação nos hosts gerenciados.

Ele não é a mesma coisa que a fingerprint da **host key** de servera ou serverb.

5.14 Validar as fingerprints de servera e serverb antes do primeiro acesso

Na primeira conexão SSH, o cliente perguntará se a identidade do host deve ser aceita.

Não aceite automaticamente sem saber qual máquina está sendo acessada.

Em servera

Pelo console de servera, execute:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Anote a fingerprint.

Em serverb

Pelo console de serverb:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Anote a fingerprint.

As duas fingerprints devem ser diferentes.

Esses valores serão comparados com o aviso apresentado pelo cliente SSH no bastion.

5.15 Copiar a chave pública para servera

De volta ao bastion:

```
ssh-copy-id \
  -i ~/.ssh/id_ed25519_rhcsa_lab.pub \
  student@servera
```

Na primeira conexão, compare a fingerprint apresentada pelo SSH com a fingerprint obtida diretamente no console de servera.

Somente depois de confirmar a identidade do host, aceite a chave.

O comando solicitará a senha atual do usuário student em servera.

Ele copia **somente a chave pública**.

A chave privada permanece no bastion.

5.16 Copiar a chave pública para serverb

Execute:

```
ssh-copy-id \
  -i ~/.ssh/id_ed25519_rhcsa_lab.pub \
  student@serverb
```

Novamente, confira a fingerprint de serverb antes de aceitar a host key.

5.17 Validar o **authorized_keys** nos hosts gerenciados

Em servera, pelo console ou SSH:

```
ls -ld ~/.ssh

ls -l ~/.ssh/authorized_keys
```

Permissões recomendadas:

```
~/.ssh                 700
~/.ssh/authorized_keys  600
```

Se necessário:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Repita em serverb.

5.18 Criar a configuração SSH do bastion

No bastion, crie ou edite:

```
nano ~/.ssh/config
```

Adicione:

```
Host servera
   HostName 192.168.100.11
   User student
   IdentityFile ~/.ssh/id_ed25519_rhcsa_lab
   IdentitiesOnly yes

Host serverb
   HostName 192.168.100.12
   User student
   IdentityFile ~/.ssh/id_ed25519_rhcsa_lab
   IdentitiesOnly yes
```

Salve o arquivo.

Ajuste:

```
chmod 600 ~/.ssh/config
```

Confira:

```
ls -l ~/.ssh/config
```

5.19 Entender por que **IdentitiesOnly yes** é utilizado

Um usuário pode possuir várias chaves carregadas no ssh-agent ou no diretório ~/.ssh.

A diretiva:

```
IdentitiesOnly yes
```

faz o cliente SSH utilizar explicitamente a chave configurada para o laboratório, reduzindo tentativas com identidades que pertencem a outros ambientes.

Isso ajuda a manter o laboratório isolado de:

- chaves pessoais;
- credenciais de GitHub;
- chaves corporativas;
- outros ambientes de estudo.

5.20 Validar acesso SSH explícito

Antes de depender do arquivo ~/.ssh/config, teste a chave diretamente.

Para servera:

```
ssh \
  -i ~/.ssh/id_ed25519_rhcsa_lab \
  student@servera \
  hostname -f
```

Resultado esperado:

```
servera.lab.test
```

Para serverb:

```
ssh \
  -i ~/.ssh/id_ed25519_rhcsa_lab \
  student@serverb \
  hostname -f
```

Resultado esperado:

```
serverb.lab.test
```

5.21 Validar acesso pelo arquivo **~/.ssh/config**

Agora teste:

```
ssh servera hostname -f
```

Resultado esperado:

```
servera.lab.test
```

Depois:

```
ssh serverb hostname -f
```

Resultado esperado:

```
serverb.lab.test
```

5.22 Validar SSH não interativo

O Ansible e o CLI do laboratório precisam conectar sem solicitar senha de login.

Teste:

```
ssh \
  -o BatchMode=yes \
  -o ConnectTimeout=5 \
  servera \
  hostname -f
```

Depois:

```
ssh \
  -o BatchMode=yes \
  -o ConnectTimeout=5 \
  serverb \
  hostname -f
```

Os dois comandos devem terminar sem prompt de senha.

Resultados esperados:

```
servera.lab.test
```

e:

```
serverb.lab.test
```

Se `BatchMode=yes` falhar, não avance para o Ansible.

5.23 Verificar as host keys armazenadas

Depois das primeiras conexões:

```
ssh-keygen -F servera
ssh-keygen -F serverb
```

Também pode conferir:

```
ls -l ~/.ssh/known_hosts
```

O projeto mantém verificação de host key habilitada no Ansible.

Por isso, `known_hosts` deve refletir as identidades corretas de `servera` e `serverb`.

5.24 O que fazer se uma host key mudar

Uma mudança de host key pode acontecer legitimamente quando:

- uma VM é reinstalada;
- uma VM é recriada a partir de template;
- um snapshot anterior é restaurado;
- as SSH host keys são regeneradas.

Também pode indicar que o cliente está alcançando um sistema diferente do esperado.

Por isso, não remova o aviso indiscriminadamente.

Primeiro confirme a nova fingerprint diretamente no console da VM:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Somente depois de confirmar a identidade, remova a entrada antiga.

Exemplo para `servera`:

```
ssh-keygen -R servera
```

Se houver uma entrada pelo endereço IP:

```
ssh-keygen -R 192.168.100.11
```

Depois faça uma nova conexão e valide a fingerprint novamente.

Repita o procedimento correspondente para `serverb`, se necessário.

5.25 Se a chave utilizar passphrase

Caso a chave dedicada tenha sido criada com passphrase, utilize `ssh-agent`.

Inicie o agente, se necessário:

```
eval "$(ssh-agent -s)"
```

Carregue a chave:

```
ssh-add ~/.ssh/id_ed25519_rhcsa_lab
```

Confira:

```
ssh-add -l
```

Depois repita:

```
ssh -o BatchMode=yes servera hostname -f
ssh -o BatchMode=yes serverb hostname -f
```

O agente vale para a sessão em que foi iniciado.

5.26 Instalar o comando **lab**

Entre na raiz do projeto:

```
cd ~/rhcsa-ex200-v10-lab
```

Execute o instalador fornecido pelo próprio repositório:

```
bash bootstrap/10-install-lab-cli.sh
```

O script valida a presença de:

```
python3
ansible
ssh
PyYAML
```

e cria um link para o CLI em:

```
~/.local/bin/lab
```

5.27 Atualizar o **PATH**

O instalador acrescenta ao `~/.bashrc`, quando necessário:

```
export PATH="$HOME/.local/bin:$PATH"
```

Recarregue o perfil:

```
source ~/.bashrc
```

Confira:

```
command -v lab
```

Resultado esperado:

```
/home/student/.local/bin/lab
```

Confira o link:

```
readlink -f ~/.local/bin/lab
```

Ele deve resolver para:

```
/home/student/rhcsa-ex200-v10-lab/bin/lab
```

5.28 Validar o CLI

Confira a versão:

```
lab --version
```

Liste os laboratórios:

```
lab list
```

O comando deve exibir o catálogo disponível no repositório sem erro de Python, YAML ou PATH.

A validação completa do ambiente será executada na próxima etapa.

5.29 Não copiar credenciais para o repositório

O diretório do projeto não deve receber:

```
chaves SSH privadas
authorized_keys pessoais
tokens do GitHub
senhas
activation keys
arquivos de assinatura Red Hat
credenciais de nuvem
arquivos .env com segredos
```

A chave privada do laboratório deve permanecer somente em:

```
/home/student/.ssh/id_ed25519_rhcsa_lab
```

Nunca copie essa chave para dentro de:

```
/home/student/rhcsa-ex200-v10-lab
```

Antes de qualquer contribuição futura para o Git, confira os arquivos que serão adicionados ao commit.

5.30 Atualizar uma instalação existente do projeto

Esta subseção só se aplica quando o repositório já foi clonado anteriormente.

Entre no diretório:

```
cd ~/rhcsa-ex200-v10-lab
```

Confira alterações locais:

```
git status
```

Se a árvore estiver limpa:

```
git fetch --tags origin
```

Confira:

```
git log --oneline --decorate HEAD..origin/main
```

Se desejar atualizar a branch main e não houver alterações locais:

```
git switch main
git pull --ff-only
```

O uso de --ff-only impede que um pull crie um merge inesperado.

Se existirem alterações locais, não execute comandos de atualização destrutivos. Preserve ou versione o trabalho antes de sincronizar.

5.31 Uso do GitHub para estudantes e mantenedores

Para **utilizar os laboratórios**, basta o clone público via HTTPS:

```
https://github.com/eduardoandradej/rhcsa-ex200-v10-lab.git
```

Não é necessário possuir credenciais de escrita no GitHub.

Operações como:

```
git push
criação de branches remotas
pull requests
manutenção do projeto
```

fazem parte do fluxo de contribuição e não são requisito para executar os laboratórios.

As credenciais utilizadas para contribuir com o GitHub devem permanecer separadas da chave:

```
id_ed25519_rhcsa_lab
```

que existe exclusivamente para comunicação entre as VMs do laboratório.

5.32 Validação consolidada

No bastion, execute:

```
hostname -f
```

Esperado:

```
bastion.lab.test
```

Confira Git:

```
git --version
```

Confira Ansible:

```
ansible --version
```

Confira Python e PyYAML:

```
python3 -c 'import sys, yaml; print(sys.version.split()[0]); print("PyYAML OK")'
```

Confira o projeto:

```
cd ~/rhcsa-ex200-v10-lab
git status
git log -1 --oneline --decorate
```

Confira SSH:

```
ssh -o BatchMode=yes servera hostname -f
ssh -o BatchMode=yes serverb hostname -f
```

Confira o CLI:

```
lab --version
lab list
```

5.33 Problemas comuns

git clone retorna que o diretório já existe

Não clone por cima.

Confira:

```
cd ~/rhcsa-ex200-v10-lab
git status
git remote -v
```

Decida se a cópia existente deve ser atualizada ou preservada.

dnf não encontra ***ansible-core***

Confira:

```
sudo dnf repolist
```

Retorne à configuração de repositórios da etapa 04.

Não substitua o problema instalando pacotes de fontes aleatórias.

python3 -c 'import yaml' falha

Confira:

```
rpm -q python3-pyyaml
```

Instale, se necessário:

```
sudo dnf install -y python3-pyyaml
```

ssh-copy-id solicita senha repetidamente

Confirme no host de destino:

```
id student
systemctl is-active sshd
```

Confira também se autenticação compatível com o primeiro provisionamento está disponível.

Se necessário, use o console da VM para investigar antes de alterar a configuração do SSH.

Permission denied (publickey,password)

No bastion:

```
ls -l ~/.ssh/id_ed25519_rhcsa_lab*
```

Confirme a chave configurada:

```
ssh -G servera | grep -i '^identityfile'
```

No destino:

```
ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
```

Host key verification failed

Não desabilite a verificação de host key.

Primeiro confirme a fingerprint no console do servidor.

Depois corrija somente a entrada correspondente em `known_hosts`.

ssh -o BatchMode=yes solicita interação ou falha

Isso indica que o acesso ainda não está pronto para Ansible.

Teste detalhadamente:

```
ssh -vvv servera
```

Use a saída para identificar:

- chave selecionada;
- arquivo de configuração utilizado;
- host key;
- método de autenticação.

lab: command not found

Confira:

```
ls -l ~/.local/bin/lab
```

Depois:

```
printf '%s\n' "$PATH" | tr ':' '\n'
```

Recarregue:

```
source ~/.bashrc
```

Confira novamente:

```
command -v lab
```

lab apresenta erro de PyYAML

Valide:

```
python3 -c 'import yaml; print(yaml.__version__)'
```

Se falhar:

```
sudo dnf install -y python3-pyyaml
```

O clone foi feito como ***root***

Não continue utilizando uma cópia em `/root`.

O projeto é esperado em:

```
/home/student/rhcsa-ex200-v10-lab
```

Faça a clonagem como student.

5.34 Critério para avançar

Antes de seguir para a validação do Ansible, confirme:

```
[ ] sessão executada em bastion.lab.test
[ ] usuário atual é student

[ ] Git instalado
[ ] Ansible Core instalado
[ ] Python 3 instalado
[ ] PyYAML funcionando
[ ] cliente OpenSSH instalado

[ ] projeto clonado em /home/student/rhcsa-ex200-v10-lab
[ ] origin aponta para o repositório correto
[ ] working tree está em estado conhecido

[ ] chave ~/.ssh/id_ed25519_rhcsa_lab existe
[ ] chave privada possui permissão restrita
[ ] chave pública foi instalada em servera
[ ] chave pública foi instalada em serverb

[ ] fingerprint de servera foi validada
[ ] fingerprint de serverb foi validada
[ ] host keys corretas estão em known_hosts

[ ] ssh servera funciona
[ ] ssh serverb funciona
[ ] BatchMode funciona sem senha em servera
[ ] BatchMode funciona sem senha em serverb

[ ] comando lab está em ~/.local/bin
[ ] lab --version funciona
[ ] lab list funciona
```

Com esses requisitos atendidos, prossiga para:

06 — Configuração e validação do Ansible

Índice · Home · Anterior · Próximo

10. Configuração e validação do Ansible

Índice · Home · Anterior · Próximo

Onde executar: bastion, conectado como usuário student, exceto quando o texto indicar explicitamente uma das VMs gerenciadas.

Esta etapa valida o plano de controle do laboratório antes da criação do baseline.

O objetivo não é apenas confirmar que o comando ansible está instalado. Antes de avançar, é necessário comprovar que:

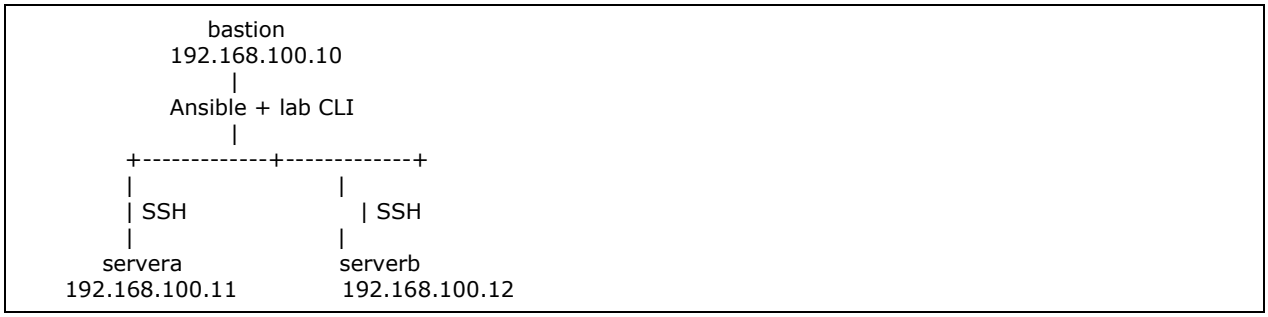
- o inventário do projeto está sendo carregado;
- bastion, servera e serverb são identificados corretamente;
- o SSH por chave funciona para os hosts gerenciados;
- a verificação de host keys permanece habilitada;
- o Python está disponível nos destinos;
- a elevação de privilégios utilizada pela automação funciona sem interação;
- as três VMs correspondem ao baseline RHEL 9 utilizado pela versão atual do laboratório;
- as dependências necessárias aos exercícios estão disponíveis;
- o CLI lab consegue consultar o ambiente.

Não crie snapshots de baseline de um ambiente que ainda apresente falhas nesta etapa.

6.1 Arquitetura do plano de controle

O Ansible é executado no bastion.

A arquitetura utilizada pelo projeto é:



No inventário:

```
control
└── bastion

managed
├── servera
└── serverb
```

O bastion utiliza conexão local do Ansible.

servera e serverb utilizam SSH com o usuário:

```
student
```

e a chave:

```
/home/student/.ssh/id_ed25519_rhcsa_lab
```

6.2 Confirmar o ponto de execução

Antes de continuar:

```
hostname -f
```

Resultado esperado:

```
bastion.lab.test
```

Confira o usuário:

```
id -un
```

Resultado esperado:

```
student
```

Entre na raiz do projeto:

```
cd ~/rhcsa-ex200-v10-lab
```

Confira:

```
pwd
```

Resultado esperado:

```
/home/student/rhcsa-ex200-v10-lab
```

Valide o estado do repositório:

```
git status
```

Registre também a versão atual:

```
git describe --tags --always
```

6.3 Confirmar as ferramentas necessárias

Execute:

```
command -v ansible
command -v ansible-playbook
command -v ansible-inventory
command -v ssh
command -v git
command -v python3
```

Todos os comandos devem retornar um caminho.

Confira as versões:

```
ansible --version

python3 --version
```

Valide PyYAML:

```
python3 -c 'import yaml; print("PyYAML OK")'
```

Resultado esperado:

```
PyYAML OK
```

Se alguma dependência estiver ausente, retorne à etapa:

05 — Preparação do bastion e clonagem

6.4 Trabalhar a partir do diretório Ansible

O arquivo ansible.cfg do projeto utiliza caminhos relativos.

Por isso, os comandos Ansible desta etapa devem ser executados a partir de:

```
/home/student/rhcsa-ex200-v10-lab/ansible
```

Entre no diretório:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Confira:

```
pwd
```

Resultado esperado:

```
/home/student/rhcsa-ex200-v10-lab/ansible
```

Liste os principais arquivos:

```
find . -maxdepth 2 -type f | sort
```

6.5 Conferir o **ansible.cfg**

Exiba:

```
cat ansible.cfg
```

A versão atual do projeto utiliza:

```
[defaults]
inventory = ./inventory/hosts.yml
host_key_checking = True
retry_files_enabled = False
interpreter_python = auto_silent
forks = 10

[privilege_escalation]
become = False
```

6.5.1 ***inventory***

```
inventory = ./inventory/hosts.yml
```

Define o inventário padrão do projeto.

Por utilizar um caminho relativo, reforça a necessidade de executar os comandos a partir do diretório ansible/.

6.5.2 ***host_key_checking***

```
host_key_checking = True
```

A verificação de identidade SSH dos hosts permanece habilitada.

Não altere esta opção para False como forma de contornar erros de SSH.

Quando houver mudança legítima de host key, confirme a nova fingerprint diretamente no console da VM antes de atualizar known_hosts.

6.5.3 ***interpreter_python***

```
interpreter_python = auto_silent
```

Permite ao Ansible identificar automaticamente o interpretador Python apropriado nos hosts.

6.5.4 ***forks***

```
forks = 10
```

Define o limite de execuções paralelas.

O laboratório possui apenas três nós, portanto esse valor não implica a existência de dez VMs.

6.5.5 ***become***

```
become = False
```

A elevação de privilégios não é habilitada globalmente.

Os playbooks que precisam de privilégios administrativos declaram explicitamente:

```
become: true
```

Essa separação é intencional.

6.6 Confirmar qual configuração o Ansible está utilizando

Execute:

```
ansible --version
```

Localize na saída a linha:

```
config file = ...
```

Ela deve apontar para o ansible.cfg do projeto, por exemplo:

```
/home/student/rhcsa-ex200-v10-lab/ansible/ansible.cfg
```

Uma validação objetiva pode ser feita com:

```
ansible-config dump --only-changed
```

Confirme especialmente:

```
DEFAULT_HOST_LIST
HOST_KEY_CHECKING
INTERPRETER_PYTHON
```

Se o Ansible estiver utilizando outro arquivo de configuração, confirme o diretório atual e se existe uma variável ANSIBLE_CONFIG sobrescrevendo o projeto:

```
printf '%s\n' "${ANSIBLE_CONFIG:-<não definida>}"
```

6.7 Conferir o inventário

Exiba:

```
cat inventory/hosts.yml
```

A versão atual utiliza:

```
---
all:
  children:

    control:
      hosts:
        bastion:
          ansible_connection: local
          ansible_python_interpreter: /usr/bin/python3

    managed:
      hosts:

        servera:
          ansible_host: 192.168.100.11

        serverb:
          ansible_host: 192.168.100.12

  vars:
    ansible_user: student
    ansible_ssh_private_key_file: /home/student/.ssh/id_ed25519_rhcsa_lab
```

O inventário pressupõe a arquitetura padrão documentada pelo projeto:

Host	Grupo	Endereço
bastion	control	conexão local
servera	managed	192.168.100.11
serverb	managed	192.168.100.12

Se você seguiu a documentação padrão, não é necessário alterar esses endereços.

6.8 Validar a sintaxe e a estrutura do inventário

Execute:

```
ansible-inventory --graph
```

A estrutura deve representar:

```
@all:
  |--@control:
  |  |--bastion
  |--@managed:
  |  |--servera
  |  |--serverb
```

A apresentação exata pode variar conforme a versão do Ansible, mas os três hosts e os dois grupos precisam estar presentes.

Consulte também:

```
ansible-inventory --list
```

Para validar apenas servera:

```
ansible-inventory --host servera
```

Para serverb:

```
ansible-inventory --host serverb
```

Confira se os endereços correspondem a:

```
servera = 192.168.100.11
serverb = 192.168.100.12
```

6.9 Confirmar a chave privada referenciada pelo inventário

O inventário utiliza:

```
/home/student/.ssh/id_ed25519_rhcsa_lab
```

Confira:

```
ls -l /home/student/.ssh/id_ed25519_rhcsa_lab
```

Valide as permissões:

```
stat -c '%a %U:%G %n' \
  /home/student/.ssh/id_ed25519_rhcsa_lab
```

O arquivo deve pertencer a student e possuir permissão restrita.

Uma configuração típica é:

```
600 student:student /home/student/.ssh/id_ed25519_rhcsa_lab
```

Não copie a chave privada para dentro do repositório.

6.10 Confirmar SSH antes de testar Ansible

O Ansible depende do SSH para servera e serverb.

Teste primeiro o mecanismo diretamente:

```
ssh -o BatchMode=yes servera hostname -f
```

Resultado esperado:

```
servera.lab.test
```

Depois:

```
ssh -o BatchMode=yes serverb hostname -f
```

Resultado esperado:

```
serverb.lab.test
```

Se um desses comandos falhar, corrija o SSH antes de diagnosticar o Ansible.

6.11 Confirmar as host keys conhecidas

Confira:

```
ssh-keygen -F servera
ssh-keygen -F serverb
```

Como host_key_checking está habilitado, as identidades corretas devem estar presentes em:

```
/home/student/.ssh/known_hosts
```

Se uma host key tiver mudado porque uma VM foi recriada, valide a nova fingerprint pelo console da VM antes de substituir a entrada.

Não use:

```
StrictHostKeyChecking=no
```

como correção permanente para o laboratório.

6.12 Preparar a elevação de privilégios da automação

Vários playbooks de preparação e limpeza utilizam:

```
become: true
```

O runner atual do projeto executa esses playbooks sem solicitar senha de sudo.

Por isso, nas VMs **descartáveis e dedicadas ao laboratório**, o usuário student deve poder elevar privilégios sem interação.

Essa configuração não representa uma recomendação para servidores de produção.

6.12.1 Configurar no bastion

No bastion:

```
sudo visudo -f /etc/sudoers.d/rhcsa-lab-student
```

Adicione:

```
student ALL=(ALL) NOPASSWD: ALL
```

Salve e saia.

Ajuste:

```
sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
```

Valide:

```
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student
```

Resultado esperado:

```
/etc/sudoers.d/rhcsa-lab-student: parsed OK
```

Teste sem prompt:

```
sudo -n true
```

O comando deve terminar com código 0 e sem solicitar senha.

Confira:

```
echo $?
```

Resultado esperado:

```
0
```

6.12.2 Configurar em servera

Acesse:

```
ssh servera
```

Crie:

```
sudo visudo -f /etc/sudoers.d/rhcsa-lab-student
```

Conteúdo:

```
student ALL=(ALL) NOPASSWD: ALL
```

Depois:

```
sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student
sudo -n true
```

Saia:

```
exit
```

6.12.3 Configurar em serverb

Acesse:

```
ssh serverb
```

Repita:

```
sudo visudo -f /etc/sudoers.d/rhcsa-lab-student
```

Conteúdo:

```
student ALL=(ALL) NOPASSWD: ALL
```

Valide:

```
sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student
sudo -n true
```

Saia:

```
exit
```

6.13 Validar **sudo** não interativo a partir do bastion

No bastion, execute:

```
sudo -n true && echo "bastion: sudo OK"
```

Depois:

```
ssh servera 'sudo -n true' && echo "servera: sudo OK"
ssh serverb 'sudo -n true' && echo "serverb: sudo OK"
```

Resultados esperados:

```
bastion: sudo OK
servera: sudo OK
serverb: sudo OK
```

Não prossiga se algum host solicitar senha.

6.14 Primeiro teste Ansible: grupo control

Ainda em:

```
~/rhcsa-ex200-v10-lab/ansible
```

execute:

```
ansible control -m ansible.builtin.ping
```

O bastion deve retornar sucesso.

O módulo:

```
ansible.builtin.ping
```

não executa ICMP.

Ele testa se o Ansible consegue executar código Python no host e receber uma resposta válida.

6.15 Testar os hosts gerenciados

Execute:

```
ansible managed -m ansible.builtin.ping
```

O resultado esperado para os dois hosts contém:

```
SUCCESS
```

e:

```
"ping": "pong"
```

Exemplo conceitual:

```
servera | SUCCESS => {
   "changed": false,
   "ping": "pong"
}

serverb | SUCCESS => {
   "changed": false,
   "ping": "pong"
}
```

Se servera ou serverb estiver UNREACHABLE, resolva o problema antes de prosseguir.

6.16 Testar todos os nós

Execute:

```
ansible all -m ansible.builtin.ping
```

Os três hosts devem responder:

```
bastion
servera
serverb
```

com sucesso.

Esse teste valida:

- inventário;
- conexão local com bastion;
- SSH com os hosts gerenciados;
- chave privada;
- host keys;
- usuário remoto;
- disponibilidade do Python.

6.17 Confirmar a identidade dos hosts pelo Ansible

Execute:

```
ansible all \
  -m ansible.builtin.command \
  -a 'hostname -f'
```

Resultados esperados:

```
bastion.lab.test
servera.lab.test
serverb.lab.test
```

Cada nome deve corresponder ao host do inventário que executou o comando.

Se servera retornar serverb.lab.test, por exemplo, interrompa a preparação e corrija a identidade da VM.

6.18 Confirmar o usuário remoto

Nos hosts gerenciados:

```
ansible managed \
  -m ansible.builtin.command \
  -a 'id -un'
```

Resultado esperado:

```
student
```

6.19 Validar **become** pelo Ansible

Teste o grupo gerenciado:

```
ansible managed \
  --become \
  -m ansible.builtin.command \
  -a 'id -u'
```

Resultado esperado nos dois hosts:

```
0
```

Teste o bastion:

```
ansible control \
  --become \
  -m ansible.builtin.command \
  -a 'id -u'
```

Resultado esperado:

```
0
```

Se aparecer:

```
Missing sudo password
```

ou uma falha de sudo, corrija a configuração NOPASSWD antes de continuar.

6.20 Verificar sintaxe dos playbooks principais

Antes da execução, valide os playbooks desta etapa.

Execute:

```
ansible-playbook \
  --syntax-check \
  playbooks/validate.yml
```

Depois:

```
ansible-playbook \
  --syntax-check \
  playbooks/00-bootstrap-dependencies.yml
```

E:

```
ansible-playbook \
  --syntax-check \
  playbooks/00-validate-dependencies.yml
```

Os três comandos devem terminar sem erro de sintaxe.

6.21 Validar o baseline de sistema operacional

Execute:

```
ansible-playbook playbooks/validate.yml
```

O playbook atual coleta fatos de todos os hosts e valida duas condições:

```
ansible_distribution == "RedHat"
ansible_distribution_major_version == "9"
```

Portanto, o baseline local atual exige:

```
Red Hat Enterprise Linux 9
```

nas três VMs.

A saída de sucesso apresenta informações como:

```
inventory
fqdn
ipv4
python
```

e uma mensagem semelhante a:

```
RedHat 9.x OK
```

Importante sobre RHEL 10

O projeto utiliza RHEL 10 como referência para o objetivo de certificação e classificação de compatibilidade dos exercícios, mas o baseline local desta versão do repositório continua sendo RHEL 9.

Não altere `validate.yml` apenas para fazer uma VM incompatível passar na validação.

Uma futura migração do laboratório local para RHEL 10 deve incluir revisão dos exercícios, dependências e validações.

6.22 Inspecionar as dependências definidas pelo projeto

Exiba:

```
cat vars/lab_dependencies.yml
```

Esse arquivo divide as dependências em:

```
control_plane_packages
managed_baseline_packages
control_plane_required_commands
managed_required_commands
```

A lista existe para que a infraestrutura necessária aos exercícios seja preparada antes do baseline.

Isso evita que o estudante precise instalar, durante um exercício, um pacote que deveria fazer parte da infraestrutura do próprio laboratório.

6.23 Preparar as dependências do laboratório

Antes desta etapa, confirme novamente que o `bastion` possui uma fonte funcional de pacotes:

```
sudo dnf repolist
```

O bootstrap utiliza os repositórios disponíveis no `bastion`.

Execute:

```
ansible-playbook \
  playbooks/00-bootstrap-dependencies.yml
```

O que esse playbook faz

No grupo `control`, ele:

3. instala as dependências do plano de controle;

4. recria:

```
/var/tmp/rhcsa-baseline-rpms
```

5. utiliza `dnf download --resolve --alldeps`;
6. constrói um conjunto local de RPMs necessários aos hosts gerenciados.

Depois, no grupo `managed`, ele:

7. cria um diretório temporário em cada host;
8. copia o conjunto de RPMs a partir do `bastion`;
9. instala os pacotes com os repositórios remotos desabilitados;
10. remove o diretório temporário das VMs gerenciadas.

Essa estratégia torna o `bastion` a fonte de preparação das dependências do laboratório.

Requisito importante

O `bastion` precisa possuir repositórios capazes de fornecer os pacotes e suas dependências.

Se a tarefa de download falhar, corrija a origem de pacotes do `bastion` antes de prosseguir.

6.24 Validar o conjunto de dependências

Depois do bootstrap:

```
ansible-playbook \
  playbooks/00-validate-dependencies.yml
```

O playbook verifica comandos necessários tanto no plano de controle quanto nos hosts gerenciados.

Entre os recursos validados estão ferramentas relacionadas a:

```
Git
SSH
arquivos e compressão
ACLs
atributos estendidos
SELinux
particionamento
LVM
XFS
ext4
NFS
AutoFS
chrony
firewalld
NetworkManager
Apache HTTP Server
```

Não substitua essa validação por uma checagem manual de apenas dois ou três comandos.

O playbook é a referência executável do baseline de dependências do projeto.

6.25 Executar novamente o teste de conectividade

Depois da instalação das dependências:

```
ansible all -m ansible.builtin.ping
```

Depois:

```
ansible all \
  -m ansible.builtin.command \
  -a 'hostname -f'
```

Isso confirma que a preparação não interrompeu a comunicação com nenhum nó.

6.26 Validar o CLI **lab**

Volte para a raiz:

```
cd ~/rhcsa-ex200-v10-lab
```

Confira a versão:

```
lab --version
```

Depois:

```
lab status
```

O comando consulta o inventário do projeto e executa `ansible.builtin.ping` nos nós.

Ele também verifica a presença local de:

```
ansible
ssh
git
python3
```

O resultado esperado deve mostrar:

```
Control Plane
  bastion      ...    ONLINE

Managed Nodes
  servera      ...    ONLINE
  serverb      ...    ONLINE
```

A seção de plataforma mostra o baseline local e o alvo de certificação:

```
Local       RHEL 9.8
Exam target RHEL 10
```

A seção de ferramentas deve indicar:

```
ansible    OK
ssh        OK
git        OK
python3    OK
```

Ao final:

```
Environment: READY
```

6.27 Entender o que **lab status** valida

`lab status` é uma verificação rápida de saúde do ambiente.

Ele confirma principalmente:

- leitura do inventário;
- conectividade Ansible;
- Python nos nós;
- disponibilidade das ferramentas locais essenciais.

Porém:

```
Environment: READY
```

não substitui:

```
ansible-playbook playbooks/validate.yml
```

nem:

```
ansible-playbook playbooks/00-validate-dependencies.yml
```

O primeiro valida o baseline de sistema operacional.

O segundo valida as dependências necessárias aos laboratórios.

Para considerar o ambiente pronto para o snapshot, **as três verificações devem estar corretas**.

6.28 Validar o catálogo de laboratórios

Execute:

```
lab list
```

O catálogo deve ser carregado sem erros de Python ou YAML.

Isso confirma que o CLI consegue descobrir os laboratórios presentes no repositório.

Não é necessário iniciar um exercício nesta etapa.

O primeiro laboratório será tratado depois da criação do baseline.

6.29 Verificação consolidada em sequência

Para uma conferência final, execute a partir do bastion.

Entre no diretório Ansible:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Valide o inventário:

```
ansible-inventory --graph
```

Valide conectividade:

```
ansible all -m ansible.builtin.ping
```

Valide identidades:

```
ansible all \
  -m ansible.builtin.command \
  -a 'hostname -f'
```

Valide privilégios:

```
ansible all \
  --become \
  -m ansible.builtin.command \
  -a 'id -u'
```

Valide o sistema operacional:

```
ansible-playbook playbooks/validate.yml
```

Valide dependências:

```
ansible-playbook playbooks/00-validate-dependencies.yml
```

Volte à raiz:

```
cd ~/rhcsa-ex200-v10-lab
```

Valide o CLI:

```
lab status
```

E o catálogo:

```
lab list
```

6.30 Resultado mínimo esperado

O ambiente só deve seguir para o baseline quando:

```
bastion   ONLINE
servera   ONLINE
serverb   ONLINE
```

e:

```
Ansible ping       PASS
Hostname           correto
sudo/become        funcional
RHEL baseline      PASS
Dependências       PASS
lab status         READY
lab list           funcional
```

6.31 Problemas comuns

ansible-inventory não encontra hosts

Confirme o diretório:

```
pwd
```

Esperado:

```
/home/student/rhcsa-ex200-v10-lab/ansible
```

Confira:

```
cat ansible.cfg
```

E:

```
cat inventory/hosts.yml
```

Valide:

```
ansible-inventory --graph
```

---

ansible --version mostra outro arquivo de configuração

Confira:

```
ansible --version
```

Depois:

```
printf '%s\n' "${ANSIBLE_CONFIG:-<não definida>}"
```

Entre novamente no diretório:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Não altere o ansible.cfg do projeto apenas para esconder uma configuração externa incorreta.

UNREACHABLE ou timeout

Teste primeiro a rede:

```
ping -c 2 servera
```

Depois:

```
ssh -o BatchMode=yes servera hostname -f
```

Confira:

```
ip route
```

No host de destino, verifique:

```
systemctl is-active sshd
```

e:

```
sudo firewall-cmd --list-services
```

Investigue rede, resolução de nomes e SSH antes de modificar o inventário.

Permission denied

Confira a chave:

```
ls -l ~/.ssh/id_ed25519_rhcsa_lab
```

Confira a configuração efetiva:

```
ssh -G servera | grep -Ei '^(user|hostname|identityfile)'
```

Teste:

```
ssh -vvv servera
```

No destino:

```
ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
```

Host key verification failed

Não desabilite a verificação de host key.

Confirme a fingerprint no console do host de destino:

```
sudo ssh-keygen \
  -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Somente depois de confirmar a identidade, remova uma entrada antiga do `known_hosts`.

---

ansible.builtin.ping falha por Python

O módulo Ansible `ping` depende de Python no destino.

Confirme diretamente:

```
ssh servera 'python3 --version'
ssh serverb 'python3 --version'
```

Se estiver ausente, retorne à etapa 04 e corrija o baseline da VM.

Missing sudo password

Isso significa que um playbook com `become: true` não conseguiu elevar privilégios sem interação.

Teste:

```
ssh servera 'sudo -n true'
ssh serverb 'sudo -n true'
```

E localmente:

```
sudo -n true
```

Confira:

```
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student
```

Não utilize `--ask-become-pass` como solução permanente para o runner desta versão do projeto.

---

validate.yml rejeita o sistema operacional

O baseline atual exige:

```
ansible_distribution == RedHat
ansible_distribution_major_version == 9
```

Confira:

```
ansible all \
  -m ansible.builtin.setup \
  -a 'filter=ansible_distribution*'
```

Não modifique o playbook apenas para transformar uma plataforma não validada em uma plataforma suportada.

O bootstrap de dependências falha no ***dnf download***

Confira no bastion:

```
sudo dnf repolist
```

Verifique:

```
command -v dnf
```

O playbook instala dnf-plugins-core no plano de controle, que fornece os recursos utilizados na preparação do conjunto de RPMs.

Confirme espaço:

```
df -h /var/tmp
```

Se o repositório local/DVD não possuir os pacotes requeridos, utilize uma fonte de pacotes compatível com o baseline.

A validação de dependências informa ***MISSING***

Leia o nome do comando apontado pelo playbook.

Consulte:

```
cat vars/lab_dependencies.yml
```

O arquivo define o baseline requerido.

Não remova um comando da lista apenas para fazer a validação passar sem entender qual exercício depende dele.

---

lab status mostra ***DEGRADED***

Execute manualmente:

```
cd ~/rhcsa-ex200-v10-lab/ansible
ansible all -m ansible.builtin.ping
```

Depois confira:

```
command -v ansible
command -v ssh
command -v git
command -v python3
```

lab status depende desses elementos.

---

lab status mostra READY, mas ***validate.yml*** falha

Isso é possível.

lab status verifica conectividade e ferramentas essenciais.

validate.yml verifica o baseline de sistema operacional.

Resolva a falha do playbook antes de criar snapshots.

6.32 Critério para avançar

Antes de seguir para a criação do baseline e dos snapshots, confirme:

```
[ ] execução feita a partir de bastion.lab.test
[ ] usuário student

[ ] ansible.cfg correto
[ ] inventário inventory/hosts.yml carregado
[ ] grupo control contém bastion
[ ] grupo managed contém servera e serverb

[ ] host_key_checking permanece habilitado
[ ] chave SSH dedicada existe e possui permissão correta
[ ] known_hosts contém as identidades validadas

[ ] SSH BatchMode funciona para servera
[ ] SSH BatchMode funciona para serverb

[ ] sudo -n funciona no bastion
[ ] sudo -n funciona no servera
[ ] sudo -n funciona no serverb

[ ] ansible control -m ping funciona
[ ] ansible managed -m ping funciona
[ ] ansible all -m ping funciona

[ ] hostnames retornados pelo Ansible estão corretos
[ ] become retorna UID 0 nos nós necessários

[ ] playbooks principais passam no syntax-check
[ ] playbooks/validate.yml passa
[ ] baseline local confirmado como RHEL 9

[ ] bootstrap de dependências foi concluído
[ ] playbooks/00-validate-dependencies.yml passa

[ ] lab --version funciona
[ ] lab status mostra Environment: READY
[ ] lab list carrega o catálogo
```

Com todos os itens atendidos, o ambiente está tecnicamente pronto para gerar um baseline consistente.

Continue para:

07 — Baseline e snapshots

Índice · Home · Anterior · Próximo

11. Baseline e snapshots

Índice · Home · Anterior · Próximo

Onde executar: primeiro no bastion, depois no **host KVM/libvirt**.

Esta etapa cria um ponto de recuperação consistente antes do início dos exercícios.

O laboratório foi projetado com uma separação clara de responsabilidades:

```
bastion
└── plano de controle estável
    ├── Git
    ├── Ansible
    ├── CLI lab
    └── estado de execução dos exercícios

servera / serverb
└── nós descartáveis de prática
    ├── alterações dos exercícios
    ├── serviços
    ├── armazenamento
    ├── rede
    └── cenários de troubleshooting
```

Por isso, o baseline recomendado desta versão é:

- manter o bastion estável e versionado pelo Git;
- criar snapshots de servera e serverb;
- utilizar lab finish como mecanismo normal de limpeza;
- utilizar snapshots como mecanismo de recuperação quando a limpeza normal não for suficiente.

> Snapshot não substitui backup. Ele é um checkpoint local do laboratório.

7.1 Entender a diferença entre **lab finish** e snapshot

O comando:

```
lab finish
```

é o mecanismo normal de encerramento de um exercício.

Ele executa o arquivo:

```
finish.yml
```

do laboratório ativo.

Quando a limpeza termina com sucesso, o CLI remove o estado ativo do exercício e apresenta:

```
LAB FINISHED
```

Se o playbook de limpeza falhar, o estado ativo **não é apagado**.

Isso é intencional: uma falha de cleanup não deve ser escondida.

Um snapshot tem outra função.

Ele permite retornar uma VM a um ponto anterior quando, por exemplo:

- um exercício alterou profundamente o sistema;
- a VM não inicializa corretamente;
- houve erro manual fora do escopo do exercício;
- a limpeza normal não consegue restaurar o nó;

- um exercício de armazenamento deixou o disco em estado inesperado;
- um cenário de recuperação precisa ser repetido do zero.

Fluxo normal:

```
lab start
   ↓
resolver exercício
   ↓
lab grade
   ↓
lab finish
   ↓
próximo exercício
```

Fluxo de contingência:

```
lab finish falhou
      ↓
diagnosticar
      ↓
restaurar snapshot, se necessário
      ↓
revalidar ambiente
      ↓
sincronizar estado do CLI
```

7.2 Não criar o baseline com laboratório ativo

No bastion:

```
cd ~/rhcsa-ex200-v10-lab
```

Execute:

```
lab status
```

A seção:

```
Lab State
```

deve mostrar:

```
Active     none
```

Se existir um exercício ativo, finalize-o:

```
lab finish
```

ou, explicitamente:

```
lab finish ID_DO_EXERCICIO
```

Depois:

```
lab status
```

Não crie snapshots de baseline enquanto o CLI indicar um exercício ativo.

7.3 Validar o ambiente antes do snapshot

Ainda no bastion, confirme o plano de controle:

```
lab status
```

O ambiente deve apresentar:

```
bastion   ONLINE
servera   ONLINE
serverb   ONLINE
```

e:

```
Environment: READY
```

Entre no diretório Ansible:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Teste:

```
ansible all -m ansible.builtin.ping
```

Valide o baseline de sistema operacional:

```
ansible-playbook playbooks/validate.yml
```

Valide as dependências:

```
ansible-playbook playbooks/00-validate-dependencies.yml
```

Somente crie o snapshot depois que essas validações estiverem corretas.

7.4 Registrar a versão do projeto utilizada no baseline

Antes de criar o checkpoint, registre a versão atual do repositório.

No bastion:

```
cd ~/rhcsa-ex200-v10-lab
```

Execute:

```
git status
```

Depois:

```
git log -1 --oneline --decorate
```

E:

```
git describe --tags --always
```

Uma árvore limpa é o estado preferido:

```
nothing to commit, working tree clean
```

Anote o commit ou tag utilizado.

Isso permite relacionar:

```
versão do código
+
versão da documentação
+
baseline das VMs
```

7.5 Por que o bastion não é o alvo principal do snapshot

Os exercícios automatizados desta versão atuam em:

```
servera
serverb
```

O bastion funciona como plano de controle.

Ele contém:

```
repositório Git
CLI lab
estado ativo em .state/
chaves SSH
Ansible
```

Reverter o bastion para um snapshot antigo pode também reverter:

- commits;
- atualizações do repositório;
- documentação;
- arquivos de estado do CLI;
- known_hosts;
- configurações SSH.

Por isso, a política recomendada é:

```
bastion  → estável, preservado e atualizado por Git
servera  → descartável, protegido por snapshot
serverb  → descartável, protegido por snapshot
```

Um snapshot do bastion pode ser criado como proteção adicional, mas não é necessário para o fluxo normal deste projeto.

7.6 Ir para o host KVM

A partir deste ponto, os comandos devem ser executados no **host KVM/libvirt**.

Não exponha acesso administrativo ao libvirt para o bastion apenas para automatizar esta etapa.

Confirme o host:

```
hostnamectl
```

Confira as VMs:

```
sudo virsh -c qemu:///system list --all
```

Devem existir:

```
bastion
servera
serverb
```

7.7 Inspecionar a topologia de discos antes do snapshot

Confira servera:

```
sudo virsh -c qemu:///system domblklist servera
```

Confira serverb:

```
sudo virsh -c qemu:///system domblklist serverb
```

A topologia esperada é:

```
servera
vda  30 GiB
vdb   5 GiB
vdc   5 GiB
vdd   3 GiB
vde   2 GiB
```

e:

```
serverb
vda  30 GiB
vdb   5 GiB
vdc   3 GiB
```

Todos os discos graváveis utilizados por este procedimento devem ser QCOW2.

7.8 Confirmar o formato dos discos

Obtenha os caminhos dos discos:

```
sudo virsh -c qemu:///system domblklist servera --details
sudo virsh -c qemu:///system domblklist serverb --details
```

Para cada arquivo de disco, consulte:

```
sudo qemu-img info /CAMINHO/DO/DISCO.qcow2
```

Confirme:

```
file format: qcow2
```

Não aplique este procedimento sem adaptação se sua VM utilizar:

- discos RAW;
- LVM block devices;
- passthrough de disco;
- volumes de storage externos;
- backing chains que você não compreende;
- storage compartilhado.

O exemplo desta documentação assume discos QCOW2 locais.

7.9 Verificar snapshots existentes

Antes de criar novos snapshots:

```
sudo virsh -c qemu:///system snapshot-list servera
sudo virsh -c qemu:///system snapshot-list serverb
```

O nome adotado por este guia é:

```
baseline-pre-labs
```

Não crie outro snapshot com o mesmo nome se ele já existir.

Se houver um baseline anterior, primeiro decida se ele ainda precisa ser preservado.

7.10 Desligar os nós gerenciados de forma limpa

O snapshot de baseline será criado com os nós desligados.

Isso reduz a quantidade de estado transitório gravado no checkpoint e simplifica a recuperação.

No host KVM:

```
sudo virsh -c qemu:///system shutdown servera
sudo virsh -c qemu:///system shutdown serverb
```

Aguarde:

```
sudo virsh -c qemu:///system list --all
```

O estado esperado é:

```
servera   shut off
serverb   shut off
```

Não utilize:

```
virsh destroy
```

como procedimento normal de desligamento.

`destroy` equivale a interromper abruptamente a alimentação da VM e deve ser reservado para situações em que o desligamento normal não é possível.

7.11 Confirmar que os nós realmente desligaram

Uma verificação objetiva:

```
sudo virsh -c qemu:///system domstate servera
```

Esperado:

```
shut off
```

Depois:

```
sudo virsh -c qemu:///system domstate serverb
```

Esperado:

```
shut off
```

Não prossiga se alguma VM ainda estiver em execução.

7.12 Criar o snapshot de servera

Execute:

```
sudo virsh -c qemu:///system snapshot-create-as \
  servera \
  baseline-pre-labs \
  --description "Baseline RHCSA limpo antes dos laboratórios"
```

Depois valide:

```
sudo virsh -c qemu:///system snapshot-info \
  servera \
  baseline-pre-labs
```

Consulte também:

```
sudo virsh -c qemu:///system snapshot-list servera
```

O snapshot:

```
baseline-pre-labs
```

deve aparecer na lista.

7.13 Criar o snapshot de serverb

Execute:

```
sudo virsh -c qemu:///system snapshot-create-as \
  serverb \
  baseline-pre-labs \
  --description "Baseline RHCSA limpo antes dos laboratórios"
```

Valide:

```
sudo virsh -c qemu:///system snapshot-info \
  serverb \
  baseline-pre-labs
```

E:

```
sudo virsh -c qemu:///system snapshot-list serverb
```

7.14 Inspecionar o XML dos snapshots

Confira servera:

```
sudo virsh -c qemu:///system snapshot-dumpxml \
  servera \
  baseline-pre-labs
```

Depois serverb:

```
sudo virsh -c qemu:///system snapshot-dumpxml \
  serverb \
  baseline-pre-labs
```

Essa inspeção é importante porque a VM possui múltiplos discos.

Não considere o baseline validado apenas porque o nome do snapshot apareceu na lista.

Confirme que o snapshot representa a topologia esperada da VM.

7.15 Opcional: snapshot do bastion

O snapshot do bastion é opcional.

Use-o somente se desejar um checkpoint completo da estação de controle e entender que uma restauração posterior poderá reverter o repositório Git e os arquivos locais.

Primeiro confirme:

```
cd ~/rhcsa-ex200-v10-lab
git status
```

no bastion.

Depois, no host KVM, desligue:

```
sudo virsh -c qemu:///system shutdown bastion
```

Aguarde:

```
sudo virsh -c qemu:///system domstate bastion
```

Quando estiver:

```
shut off
```

crie:

```
sudo virsh -c qemu:///system snapshot-create-as \
  bastion \
  baseline-pre-labs \
  --description "Baseline opcional do plano de controle"
```

Valide:

```
sudo virsh -c qemu:///system snapshot-info \
  bastion \
  baseline-pre-labs
```

Se você optar por **não** criar o snapshot do bastion, mantenha-o protegido por:

- Git;
- backups independentes quando necessário;
- disciplina para não realizar exercícios diretamente nele.

7.16 Reiniciar as VMs

Inicie servera:

```
sudo virsh -c qemu:///system start servera
```

Inicie serverb:

```
sudo virsh -c qemu:///system start serverb
```

Se o bastion também foi desligado:

```
sudo virsh -c qemu:///system start bastion
```

Confira:

```
sudo virsh -c qemu:///system list
```

7.17 Aguardar o sistema operacional ficar disponível

O estado running do libvirt não significa que o SSH já está pronto.

Aguarde alguns segundos.

No bastion, teste:

```
ssh -o BatchMode=yes \
  -o ConnectTimeout=5 \
  servera \
  hostname -f
```

Depois:

```
ssh -o BatchMode=yes \
  -o ConnectTimeout=5 \
  serverb \
  hostname -f
```

Resultados esperados:

```
servera.lab.test
```

e:

```
serverb.lab.test
```

7.18 Revalidar o ambiente depois do snapshot

No bastion:

```
cd ~/rhcsa-ex200-v10-lab
```

Execute:

```
lab status
```

O ambiente deve voltar a:

```
Environment: READY
```

Confirme que não existe exercício ativo:

```
Active      none
```

Entre no diretório Ansible:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Teste:

```
ansible all -m ansible.builtin.ping
```

Todos os hosts devem responder com sucesso.

Restauração

7.19 Quando utilizar o snapshot

O snapshot não deve substituir lab finish no uso diário.

Primeiro tente:

```
lab finish
```

Considere restaurar o snapshot quando:

- a VM não inicializa;
- o exercício alterou o armazenamento além da capacidade do cleanup;
- houve uma modificação manual fora do cenário;
- a limpeza falha repetidamente;
- você deseja voltar ao estado inicial do laboratório para uma nova sequência de estudos.

Para exercícios que envolvem simultaneamente `servera` e `serverb`, considere restaurar os dois nós para preservar a consistência do cenário.

7.20 Preservar dados antes de restaurar

Uma restauração descarta alterações posteriores ao snapshot na VM restaurada.

Antes de reverter, verifique se existe algo que precisa ser preservado.

Em especial:

```
arquivos pessoais
anotações
scripts
configurações criadas manualmente
evidências de troubleshooting
```

O laboratório é descartável, mas isso não significa que dados pessoais devam ser descartados sem intenção.

7.21 Desligar antes da restauração

No host KVM:

```
sudo virsh -c qemu:///system shutdown servera
sudo virsh -c qemu:///system shutdown serverb
```

Aguarde:

```
sudo virsh -c qemu:///system list --all
```

Se apenas uma VM será restaurada, desligue apenas essa VM.

Não execute uma restauração de snapshot sem confirmar o estado da máquina.

7.22 Restaurar servera

Confira primeiro:

```
sudo virsh -c qemu:///system snapshot-list servera
```

Restaure:

```
sudo virsh -c qemu:///system snapshot-revert \
  servera \
  baseline-pre-labs
```

Confira:

```
sudo virsh -c qemu:///system domstate servera
```

Se permanecer desligada, inicie:

```
sudo virsh -c qemu:///system start servera
```

7.23 Restaurar serverb

Confira:

```
sudo virsh -c qemu:///system snapshot-list serverb
```

Restaure:

```
sudo virsh -c qemu:///system snapshot-revert \
  serverb \
  baseline-pre-labs
```

Depois:

```
sudo virsh -c qemu:///system domstate serverb
```

Se necessário:

```
sudo virsh -c qemu:///system start serverb
```

7.24 Restaurar os dois nós

Quando o cenário envolveu ambos:

```
sudo virsh -c qemu:///system snapshot-revert \
  servera \
  baseline-pre-labs

sudo virsh -c qemu:///system snapshot-revert \
  serverb \
  baseline-pre-labs
```

Depois:

```
sudo virsh -c qemu:///system start servera
sudo virsh -c qemu:///system start serverb
```

7.25 Revalidar SSH após restauração

No bastion:

```
ssh -o BatchMode=yes servera hostname -f

ssh -o BatchMode=yes serverb hostname -f
```

Se aparecer aviso de host key alterada, não remova a entrada automaticamente.

Confirme a fingerprint diretamente pelo console da VM.

Um snapshot criado corretamente deve restaurar também a identidade SSH existente no momento em que o baseline foi criado.

7.26 Revalidar o Ansible após restauração

No bastion:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

Execute:

```
ansible all -m ansible.builtin.ping
```

Depois:

```
ansible all \
  -m ansible.builtin.command \
  -a 'hostname -f'
```

Resultados esperados:

```
bastion.lab.test
servera.lab.test
serverb.lab.test
```

7.27 Sincronizar o estado do CLI depois de uma recuperação manual

O estado de exercício ativo é armazenado no bastion, em:

```
.state/active.json
```

Esse diretório é runtime e está ignorado pelo Git.

No uso normal, **não manipule esse arquivo manualmente**.

Primeiro consulte:

```
lab status
```

Se não houver laboratório ativo

Nenhuma ação adicional é necessária.

Se ainda houver um laboratório ativo após restaurar as VMs

Isso pode acontecer quando um lab finish falhou e as VMs foram recuperadas manualmente por snapshot.

Primeiro tente novamente:

```
lab finish
```

Se o cleanup agora terminar com sucesso, o próprio CLI limpará o estado.

Somente em uma recuperação excepcional, quando:

- as VMs já foram comprovadamente restauradas ao baseline;
- não existe mais cenário ativo nos nós;
- lab finish não consegue concluir por um problema externo ao estado das VMs;

pode ser necessário remover manualmente o estado runtime:

```
rm -f ~/rhcsa-ex200-v10-lab/.state/active.json
```

Depois:

```
lab status
```

Esse procedimento é uma recuperação administrativa, não o fluxo normal do laboratório.

7.28 Revalidar o baseline completo após restauração

Para uma recuperação importante, execute novamente:

```
cd ~/rhcsa-ex200-v10-lab/ansible
ansible-playbook playbooks/validate.yml
```

Depois:

```
ansible-playbook \
  playbooks/00-validate-dependencies.yml
```

Volte:

```
cd ~/rhcsa-ex200-v10-lab
```

Confira:

```
lab status
```

Só retome os exercícios quando o ambiente estiver novamente consistente.

Manutenção dos snapshots

7.29 Não confundir snapshot com backup

Snapshots internos ficam associados aos discos e ao armazenamento do host.

Eles não protegem contra:

- perda do SSD do hospedeiro;
- corrupção do arquivo QCOW2;
- exclusão acidental da VM;
- perda do diretório `/var/lib/libvirt`;
- falha completa do host.

Para proteger o ambiente contra esses eventos, utilize backup independente.

7.30 Antes de copiar um QCOW2

Não copie simplesmente um arquivo de disco que está sendo gravado por uma VM em execução.

Para produzir uma cópia consistente:

11. desligue a VM; ou
12. utilize um procedimento de backup compatível com libvirt/QEMU.

Para este projeto local, o caminho mais simples é desligar a VM antes da cópia.

7.31 Inspecionar snapshots periodicamente

No host KVM:

```
sudo virsh -c qemu:///system snapshot-list servera
sudo virsh -c qemu:///system snapshot-list serverb
```

Snapshots acumulados aumentam a complexidade de manutenção e podem aumentar o consumo de armazenamento.

Mantenha apenas checkpoints que tenham uma finalidade conhecida.

7.32 Verificar espaço em disco

No host:

```
df -h /var/lib/libvirt
```

Confira também o tamanho dos arquivos:

```
sudo du -sh /var/lib/libvirt/images/*
```

E informações do QCOW2:

```
sudo qemu-img info \
  /var/lib/libvirt/images/servera.qcow2

sudo qemu-img info \
  /var/lib/libvirt/images/serverb.qcow2
```

Não espere o filesystem do host ficar sem espaço antes de limpar snapshots antigos ou aumentar a capacidade.

7.33 Remover um snapshot antigo

Antes de remover qualquer snapshot, confirme:

```
sudo virsh -c qemu:///system snapshot-list servera
```

Depois, se você tiver certeza de que o snapshot não será mais necessário:

```
sudo virsh -c qemu:///system snapshot-delete \
  servera \
  NOME_DO_SNAPSHOT
```

Para serverb:

```
sudo virsh -c qemu:///system snapshot-delete \
  serverb \
  NOME_DO_SNAPSHOT
```

Nunca remova snapshots apenas para “limpar a lista” sem compreender sua função.

7.34 Atualizar o baseline após uma nova versão do projeto

Uma atualização do Git não atualiza automaticamente os snapshots.

Quando uma nova versão do projeto exigir mudanças no baseline:

13. atualize o repositório;
14. aplique as mudanças de infraestrutura documentadas;
15. execute novamente as validações da etapa 06;
16. confirme que nenhum laboratório está ativo;
17. crie um novo snapshot com nome diferente.

Exemplo:

```
baseline-v1.8.4
baseline-v1.9.0
```

Isso permite distinguir claramente a versão do ambiente.

Não substitua silenciosamente um baseline antigo sem registrar a mudança.

7.35 Estratégia de nomes recomendada

Para um ambiente pessoal simples:

```
baseline-pre-labs
```

é suficiente.

Para ambientes mantidos por mais tempo:

```
baseline-v1.8.4
baseline-v1.9.0
```

é mais informativo.

O nome do snapshot deve permitir responder:

```
"Que versão do ambiente estou restaurando?"
```

7.36 Problemas comuns

snapshot-create-as informa que o snapshot já existe

Confira:

```
sudo virsh -c qemu:///system snapshot-list servera
```

Não reutilize o mesmo nome sem decidir o que fazer com o snapshot existente.

Crie um novo nome ou remova conscientemente o antigo.

O snapshot falha porque um disco não suporta a operação

Confira:

```
sudo virsh -c qemu:///system domblklist servera --details
```

Depois:

```
sudo qemu-img info /CAMINHO/DO/DISCO
```

Este roteiro pressupõe discos QCOW2 locais.

Não converta ou remova discos apenas para forçar o snapshot sem compreender o impacto.

A VM não desliga com ***virsh shutdown***

Confira:

```
sudo virsh -c qemu:///system domstate servera
```

Aguarde o sistema operacional responder ao evento ACPI.

Se necessário, entre pelo console e execute:

```
sudo systemctl poweroff
```

Use `virsh destroy` apenas como último recurso.

A VM foi restaurada, mas o Ansible continua falhando

Teste:

```
ssh -o BatchMode=yes servera hostname -f
```

Depois:

```
cd ~/rhcsa-ex200-v10-lab/ansible
ansible servera -m ansible.builtin.ping
```

Confira:

- hostname;
- endereço IP;
- SSH host key;
- usuário `student`;
- `sudo -n`;
- Python.

---

lab status mostra exercício ativo depois do snapshot revert

O snapshot de `servera` ou `serverb` não modifica o estado armazenado no `bastion`.

Primeiro tente:

```
lab finish
```

Somente depois de confirmar que os nós já estão no baseline considere a recuperação manual do estado runtime descrita nesta página.

O repositório do bastion voltou para uma versão antiga

Isso pode ocorrer se você optou por criar e restaurar um snapshot do `bastion`.

Confira:

```
cd ~/rhcsa-ex200-v10-lab
git status
git log -1 --oneline --decorate
```

Atualize o Git conscientemente antes de continuar.

Esse é um dos motivos pelos quais o snapshot do `bastion` é opcional.

7.37 Critério para concluir a etapa

Antes de iniciar o primeiro exercício, confirme:

```
[ ] nenhum laboratório está ativo

[ ] lab status mostra Environment: READY
[ ] ansible all -m ping funciona
[ ] validate.yml passa
[ ] validação de dependências passa

[ ] versão do Git utilizada no baseline foi identificada

[ ] servera utiliza discos QCOW2 compatíveis com o plano de snapshot
[ ] serverb utiliza discos QCOW2 compatíveis com o plano de snapshot

[ ] servera possui snapshot de baseline
[ ] serverb possui snapshot de baseline

[ ] snapshots foram inspecionados e validados

[ ] servera voltou a iniciar normalmente
[ ] serverb voltou a iniciar normalmente

[ ] SSH continua funcional
[ ] Ansible continua funcional

[ ] bastion permanece estável e em estado conhecido
```

O ambiente está agora protegido por dois níveis complementares:

```
lab finish
└── limpeza normal de cada exercício

snapshot
└── recuperação do nó quando a limpeza normal não é suficiente
```

Continue para:

08 — Primeiro exercício

Índice · Home · Anterior · Próximo

12. Primeiro exercício

Índice · Home · Anterior

Onde executar: comandos `lab` no `bastion`; resolução manual no host indicado pelo enunciado.

Esta etapa encerra a instalação do ambiente e apresenta o fluxo normal de uso do laboratório.

O primeiro exercício utilizado neste roteiro é:

```
obj01-01 — Operações com arquivos
```

Na versão atual do projeto, esse exercício possui:

Campo	Valor
Objetivo	Objective 01 — Essential Tools
Alvo	servera
Dificuldade	1/5
Tempo estimado	10 minutos
Compatibilidade local	RHEL 9.8
Referência	RHEL 10
Nível de compatibilidade	exact
Política de reset	ansible
Status	ready

O objetivo desta página não é ensinar a resposta do exercício.

O estudante deve utilizar o enunciado, a documentação do sistema e seus próprios conhecimentos para chegar ao estado final solicitado.

8.1 Fluxo normal de um laboratório

A sequência recomendada é:

```
lab status
   ↓
lab list
   ↓
lab show ID
   ↓
lab start ID
   ↓
resolver manualmente no alvo
   ↓
lab grade ID
   ↓
corrigir, se necessário
   ↓
lab grade ID
   ↓
lab finish ID
   ↓
lab status
```

O CLI separa três responsabilidades:

```
start   → prepara o cenário
grade   → avalia o estado final
finish  → limpa o cenário
```

Essas três operações não devem ser confundidas.

8.2 Confirmar que o ambiente está pronto

No bastion:

```
cd ~/rhcsa-ex200-v10-lab
```

Execute:

```
lab status
```

Antes de iniciar o primeiro exercício, confirme:

```
Control Plane
  bastion      ...   ONLINE

Managed Nodes
  servera      ...   ONLINE
  serverb      ...   ONLINE
```

As ferramentas devem aparecer como disponíveis:

```
ansible    OK
ssh        OK
git        OK
python3    OK
```

O estado do laboratório deve indicar:

```
Active      none
```

E o resultado final:

```
Environment: READY
```

Se o ambiente estiver DEGRADED, não inicie um novo exercício antes de corrigir o problema.

8.3 Consultar o catálogo

Execute:

```
lab list
```

O comando organiza os exercícios por objetivo e exibe:

```
ID
LAB
LEVEL
TIME
STATUS
```

Somente exercícios com status:

```
ready
```

podem ser iniciados.

Um item com outro status pode existir no catálogo sem estar disponível para execução.

Filtrar por objetivo

Para listar apenas os laboratórios do Objective 01:

```
lab list --objective obj01
```

Também é possível filtrar por parte do nome do objetivo:

```
lab list --objective essential
```

8.4 Conhecer o exercício antes de iniciá-lo

Execute:

```
lab show obj01-01
```

O CLI apresenta:

```
ID
Objective
Title
Target
Difficulty
Time
Compatibility
Reset policy
Status
Description
```

Para `obj01-01`, confirme especialmente:

```
Target      servera
Status      ready
```

Leia o enunciado completo antes de executar `lab start`.

Enunciado em inglês

Os laboratórios que possuem tradução também podem ser consultados com:

```
lab show obj01-01 --lang en
```

O idioma do enunciado não altera o cenário nem os critérios de avaliação.

8.5 O que o primeiro exercício pratica

O exercício `obj01-01` trabalha operações fundamentais com arquivos e diretórios.

O cenário é preparado em:

```
/home/student/rhcsa-lab/obj01-01
```

no host:

```
servera
```

O diretório inicial contém arquivos fornecidos pelo próprio laboratório.

O estudante deverá organizar esse conteúdo conforme o enunciado.

As competências envolvidas incluem operações como:

```
criação de diretórios
cópia de arquivos
movimentação de arquivos
renomeação
criação de arquivos vazios
```

```
remoção seletiva
preservação do conteúdo fornecido
```

Esta página não fornece os comandos de solução.

8.6 Iniciar o exercício

No bastion:

```
lab start obj01-01
```

O CLI executa o setup.yml correspondente ao exercício.

Se a preparação terminar corretamente, a saída inclui:

```
LAB READY
```

e apresenta novamente:

```
Objective
Exercise
Target
Time
Tarefa
```

O comando também informa o host ao qual o estudante deve se conectar.

Para este exercício:

```
servera
```

8.7 O que acontece durante **lab start**

Para obj01-01, a preparação automatizada ocorre em servera.

O projeto:

18. remove dados antigos desse exercício, se existirem;
19. cria:

```
/home/student/rhcsa-lab/obj01-01/input
```

20. adiciona arquivos de prática;
21. registra obj01-01 como o laboratório ativo.

O estado ativo é mantido no bastion.

Enquanto houver um laboratório ativo, o CLI impede que outro seja iniciado.

Essa proteção evita que dois cenários automatizados disputem o mesmo ambiente.

8.8 Confirmar o laboratório ativo

Depois de iniciar:

```
lab status
```

A seção:

```
Lab State
```

deve apresentar:

```
Active      obj01-01
```

Se já houver uma pontuação registrada, também poderá aparecer:

```
Last score  NN%
```

8.9 Conectar ao alvo

No bastion:

```
ssh servera
```

Confirme:

```
hostname -f
```

Resultado esperado:

```
servera.lab.test
```

Confira o usuário:

```
id -un
```

Resultado esperado:

```
student
```

8.10 Ir para o diretório do exercício

No servera:

```
cd /home/student/rhcsa-lab/obj01-01
```

Confira:

```
pwd
```

Resultado esperado:

```
/home/student/rhcsa-lab/obj01-01
```

Inspecione o cenário antes de alterá-lo:

```
find . -maxdepth 2 -type f -o -type d
```

Ou:

```
ls -la
```

e:

```
ls -la input
```

Essa inspeção inicial é uma prática importante.

Antes de modificar um sistema em uma prova prática ou ambiente real, entenda o estado de partida.

8.11 Ler o enunciado novamente, se necessário

O enunciado pode ser consultado no bastion com:

```
lab show obj01-01
```

Se você estiver trabalhando em outra sessão SSH, mantenha o enunciado aberto no terminal do bastion.

O exercício solicita um **estado final**.

O grader não exige que o estudante utilize um comando específico para chegar a esse estado.

Isso é intencional e aproxima a prática de uma avaliação baseada em resultado.

8.12 Resolver manualmente

A partir deste ponto, resolva o exercício manualmente em servera.

Não execute playbooks de solução e não consulte solution.md durante a tentativa normal.

Use, quando necessário:

```
man
info
--help
documentação instalada no sistema
```

Exemplos de consulta:

```
man cp
man mv
man mkdir
man touch
man rm
```

O objetivo é desenvolver velocidade e segurança com as ferramentas de administração Linux.

8.13 Não alterar o cenário fora do solicitado

Durante o exercício:

- trabalhe somente nos arquivos e diretórios relacionados ao cenário;
- não modifique o setup.yml;
- não modifique o grade.py;
- não modifique o finish.yml;
- não altere o inventário Ansible para fazer o grader passar;
- não execute a solução de referência como substituto da prática.

O grader avalia o sistema, não o texto do seu comando.

8.14 Sair de servera antes da avaliação

Quando considerar a tarefa concluída:

```
exit
```

Você deve retornar ao shell de:

```
student@bastion
```

Confirme, se desejar:

```
hostname -f
```

Resultado esperado:

```
bastion.lab.test
```

8.15 Avaliar o exercício

No bastion:

```
lab grade obj01-01
```

Como obj01-01 já está ativo, também é possível utilizar:

```
lab grade
```

Para documentação e troubleshooting, utilizar o ID explicitamente costuma deixar a operação mais clara.

8.16 Como interpretar a avaliação

O grader apresenta cada critério com:

```
[PASS]
```

ou:

```
[FAIL]
```

Ao final:

```
Score: NN%
```

Quando todos os critérios são atendidos:

```
Score: 100%
Resultado: PASS
```

Quando ainda existem pendências:

```
Resultado: INCOMPLETE
```

Um resultado incompleto não encerra o exercício.

8.17 O que o grader verifica em **obj01-01**

Sem revelar uma sequência de comandos de solução, o grader verifica o estado final correspondente ao enunciado, incluindo:

- existência da estrutura de diretórios solicitada;
- arquivos que deveriam ser copiados presentes na origem e no destino;
- arquivos que deveriam ser movidos ausentes da origem e presentes no destino;
- arquivo de notas movido e renomeado corretamente;
- arquivos vazios solicitados realmente vazios;

- arquivos temporários removidos;
- conteúdo dos arquivos fornecidos preservado.

Portanto, “parece certo” não é suficiente.

O estado final precisa corresponder aos critérios do exercício.

8.18 Corrigir uma tentativa incompleta

Se o resultado não for 100%, leia as linhas `[FAIL]`.

O grader pode apresentar uma dica curta para o critério pendente.

Volte ao alvo:

```
ssh servera
```

Entre novamente:

```
cd /home/student/rhcsa-lab/obj01-01
```

Corrija apenas o que estiver incorreto.

Depois:

```
exit
```

Avalie novamente:

```
lab grade obj01-01
```

Esse ciclo pode ser repetido:

```
resolver
  ↓
grade
  ↓
corrigir
  ↓
grade
```

até atingir o estado desejado.

8.19 Não usar **lab start** novamente para corrigir uma tentativa

Depois que o exercício está ativo, não execute novamente:

```
lab start obj01-01
```

O CLI já possui um laboratório ativo e impedirá o início de outro cenário.

Além disso, a função do `start` é preparar o estado inicial, não corrigir uma resposta.

Para corrigir:

```
volte ao host
ajuste manualmente
execute grade novamente
```

8.20 Registrar mentalmente o motivo de cada erro

A pontuação é útil, mas o objetivo do laboratório não é apenas chegar a 100%.

Ao encontrar um [FAIL], identifique:

```
o que eu interpretei errado?
qual comando ou conceito faltou?
eu validei o resultado antes de chamar o grader?
```

Esse processo aumenta a retenção e reduz erros repetidos em exercícios posteriores.

8.21 Finalizar o exercício

Depois de concluir a prática e quando não precisar mais preservar o cenário:

```
lab finish obj01-01
```

Como o exercício está ativo, também é possível:

```
lab finish
```

O CLI executa:

```
finish.yml
```

do exercício.

Em obj01-01, a limpeza remove:

```
/home/student/rhcsa-lab/obj01-01
```

de servera.

Quando a limpeza termina corretamente:

```
LAB FINISHED
```

O estado ativo é então removido.

8.22 Não executar **finish** antes de terminar a prática

lab finish é uma operação de limpeza.

Se você deseja continuar investigando ou corrigindo o exercício, não finalize o cenário.

Use:

```
lab grade
```

quantas vezes forem necessárias durante a tentativa.

Execute:

```
lab finish
```

somente quando quiser encerrar e limpar aquele exercício.

8.23 Confirmar a limpeza

Depois:

```
lab status
```

A seção:

```
Lab State
```

deve voltar para:

```
Active     none
```

Confira também a saúde do ambiente:

```
Environment: READY
```

Opcionalmente, valide no servera:

```
ssh servera \
  'test ! -e /home/student/rhcsa-lab/obj01-01 && echo "obj01-01 limpo"'
```

Resultado esperado:

```
obj01-01 limpo
```

8.24 O que acontece se **lab finish** falhar

Se o playbook finish.yml falhar, o CLI:

- informa a falha;
- não remove silenciosamente o estado ativo.

Isso permite diagnosticar a limpeza antes de prosseguir.

Consulte:

```
lab status
```

Se o exercício continuar ativo, investigue a causa.

Não apague .state/active.json apenas para esconder uma falha de cleanup.

Se a VM tiver sido profundamente alterada e a limpeza não puder ser concluída, utilize o procedimento de recuperação por snapshot documentado em:

07 — Baseline e snapshots

8.25 Um exercício por vez

O projeto trabalha com um único laboratório ativo por vez.

Se tentar iniciar outro enquanto obj01-01 estiver ativo, o CLI recusará a operação.

O fluxo correto é:

```
lab start obj01-01
...
lab finish obj01-01
lab start obj01-02
```

Essa regra reduz interferência entre cenários.

8.26 Status **ready** e outros estados do catálogo

O comando:

```
lab list
```

é a referência para a disponibilidade atual.

Um exercício com:

```
ready
```

possui o conjunto necessário para execução automatizada, incluindo:

```
setup.yml
finish.yml
grade.py
prompt.pt.md
```

Além dos demais metadados e arquivos utilizados pelo projeto.

Se um item não estiver em `ready`, não tente forçar sua execução alterando os metadados.

8.27 Como estudar com cada laboratório

Uma rotina eficiente é:

```
1. Leia o título.
2. Defina mentalmente quais comandos/conceitos serão necessários.
3. Inicie o cenário.
4. Leia o enunciado completo.
5. Inspecione o estado inicial.
6. Resolva sem consultar a solução.
7. Valide manualmente seu próprio resultado.
8. Execute o grader.
9. Corrija o que faltar.
10. Só então finalize o cenário.
```

A etapa 7 é especialmente importante.

No exame real, você não terá um grader indicando cada erro durante a execução.

Aprender a validar o próprio trabalho faz parte da preparação.

8.28 Use documentação local antes de procurar respostas prontas

Durante a primeira tentativa, priorize:

```
man COMANDO
COMANDO --help
apropos PALAVRA
info COMANDO
```

Quando aplicável:

```
rpm -qd PACOTE
```

e documentação instalada em:

```
/usr/share/doc
```

Isso treina uma habilidade valiosa para administração Linux e para avaliações práticas.

8.29 Soluções de referência

Os diretórios dos laboratórios podem conter:

```
solution.md
```

Esse arquivo existe para manutenção, estudo posterior e validação do projeto.

Para uma tentativa realista:

```
não consulte solution.md antes de resolver
```

Uma forma produtiva de utilizá-lo é:

```
tentativa própria
   ↓
grade
   ↓
correção própria
   ↓
100%
   ↓
comparar com a solução de referência
```

Assim, a solução serve para comparar abordagens em vez de substituir o raciocínio.

8.30 Compatibilidade RHEL 9.8 e RHEL 10

O `lab show obj01-01` informa:

```
Local      RHEL 9.8
Target      RHEL 10
Level      exact
```

Para esse exercício, o projeto classifica a prática local como compatível de forma direta com o objetivo correspondente usado como referência.

Outros laboratórios podem possuir classificações diferentes.

Sempre consulte:

```
lab show ID
```

antes de iniciar uma prática nova.

8.31 O projeto não simula uma prova oficial

Este laboratório é um projeto independente de treinamento.

Ele não reproduz:

- questões reais do EX200;
- ambiente oficial de exame;
- conteúdo proprietário da Red Hat;
- laboratórios da Red Hat Learning Subscription.

Os cenários são autorais e utilizam competências de administração Linux/RHEL como referência de estudo.

O objetivo é desenvolver capacidade prática, repetição e troubleshooting.

8.32 Primeira sessão de estudo recomendada

Para uma primeira sessão, utilize esta sequência:

```
cd ~/rhcsa-ex200-v10-lab
lab status
lab list --objective obj01
lab show obj01-01
lab start obj01-01
```

Depois resolva manualmente em servera.

Ao terminar:

```
lab grade obj01-01
```

Se necessário, corrija e repita o grade.

Quando concluir:

```
lab finish obj01-01
```

Por fim:

```
lab status
```

8.33 Antes de iniciar o segundo exercício

Confirme:

```
[ ] primeiro exercício foi iniciado corretamente
[ ] resolução foi feita manualmente no host indicado
[ ] grader foi utilizado somente após a tentativa
[ ] critérios pendentes foram corrigidos conscientemente
[ ] exercício foi finalizado
[ ] Lab State voltou para Active none
[ ] Environment voltou para READY
```

Depois consulte:

```
lab list --objective obj01
```

e avance para o próximo item ready do Objective 01.

8.34 Critério para concluir a instalação do laboratório

A instalação completa pode ser considerada funcional quando você consegue executar, de ponta a ponta:

```
lab status
   ↓
lab list
   ↓
lab show obj01-01
   ↓
lab start obj01-01
   ↓
acesso ao servera
   ↓
resolução manual
   ↓
lab grade obj01-01
```

```
   ↓
lab finish obj01-01
   ↓
lab status
```

O estado final esperado é:

```
Active      none
Environment READY
```

A partir deste ponto, a infraestrutura deixa de ser o foco principal.

O foco passa a ser:

```
prática
repetição
velocidade
validação
troubleshooting
```

Esse é o ciclo de estudo para o qual o projeto foi construído.

Índice · Home · Anterior

PARTE III

Operação e rotina de estudos

RHCSA EX200 v10 Lab

13. CLI lab e fluxo operacional

O comando lab é a interface operacional do ambiente. Ele descobre o catálogo, controla o estado do exercício ativo, aciona playbooks de preparação/limpeza e executa graders. O fluxo normal mantém apenas um laboratório ativo por vez.

Comando	Finalidade
lab status	Verifica nós, ferramentas, plataforma e estado do laboratório ativo.
lab list	Lista o catálogo e permite filtrar por objetivo.
lab show <id>	Mostra metadados e enunciado do laboratório.
lab start <id>	Prepara o cenário e registra o exercício como ativo.
lab grade <id>	Executa o grader e apresenta a pontuação.
lab finish <id>	Executa o cleanup e encerra o estado ativo.

```
lab status
lab list
lab show obj01-01
lab start obj01-01
ssh student@servera
lab grade obj01-01
lab finish obj01-01
```

13.1 Fluxo de uma sessão

- Confirme Environment: READY com lab status.
- Escolha um item do catálogo com lab list ou lab list --objective ....
- Leia o enunciado com lab show antes de iniciar.
- Execute lab start e resolva manualmente no host alvo.
- Use documentação local do sistema antes de consultar soluções.
- Execute lab grade; corrija o estado se necessário e rode o grader novamente.
- Somente ao terminar, execute lab finish e confirme a limpeza.

14. Estratégia de estudo e avaliação por estado final

Os graders foram desenhados para observar o estado real do sistema. Em vez de exigir uma sequência textual de comandos, podem verificar existência, conteúdo, permissões, ownership, serviços, configuração persistente, montagem, política SELinux, regras de firewall e outros efeitos observáveis.

> ***Observação:*** *As soluções de referência em tests/reference-solutions/ existem para QA da engenharia do laboratório. O fluxo normal do estudante é resolver manualmente antes de consultar qualquer solução.*

14.1 Rotina recomendada

- Faça os objetivos em ordem até construir fluência operacional.
- Refaça labs com pontuação inferior a 100% após um intervalo, sem consultar solução.
- Nos desafios integrados, trabalhe a partir do estado final solicitado e documente mentalmente a evidência de cada requisito.
- Repita os dois drills manuais de boot/recovery em console e, quando possível, também em um ambiente RHEL 10.
- Use snapshots como recuperação out-of-band, não como substituto de `lab finish`.

PARTE IV

Catálogo completo de laboratórios

RHCSA EX200 v10 Lab

15. Catálogo completo dos 100 itens

O catálogo abaixo foi extraído dos arquivos `lab.yml` presentes no repositório desta edição. Ele registra ID, título, alvo, dificuldade, duração, compatibilidade e status operacional.

Objective 01 — Essential Tools

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj01-01	Operações com arquivos	servera	1	10 min	exact	ready
obj01-02	Redirecionamento e pipes	servera	2	12 min	exact	ready
obj01-03	grep e expressões regulares	servera	2	12 min	exact	ready
obj01-04	SSH e troca de usuários	servera, serverb	2	12 min	exact	ready
obj01-05	Arquivos e compressão	servera	2	12 min	exact	ready
obj01-06	Edição de arquivos de texto	servera	2	10 min	exact	ready
obj01-07	Hard links e symbolic links	servera	2	10 min	exact	ready
obj01-08	Permissões de arquivos	servera	2	12 min	exact	ready
obj01-09	Documentação do sistema	servera	2	10 min	exact	ready
obj01-10	Desafio integrado Essential Tools	servera, serverb	3	25 min	exact	ready

Objective 02 — Users & Groups

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj02-01	Identidade e bases locais	servera	1	10 min	exact	ready
obj02-02	Criar, modificar e remover usuários	servera	2	14 min	exact	ready
obj02-03	Criar e modificar grupos	servera	2	12 min	exact	ready
obj02-04	Grupos primários e suplementares	servera	2	14 min	exact	ready
obj02-05	Senhas e password aging	servera	2	15 min	exact	ready
obj02-06	Sudo e sudoers.d	servera	3	15 min	exact	ready
obj02-07	Lock, expiração e nologin	servera	2	14 min	exact	ready
obj02-08	Desafio integrado Users & Groups	serverb	3	25 min	exact	ready

Objective 03 — RPM, DNF & Repositories

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj03-01	Consultar a base RPM instalada	servera	1	12 min	exact	ready
obj03-02	Inspecionar e extrair um arquivo RPM	servera	2	18 min	exact	ready
obj03-03	Descobrir software com DNF	servera	2	15 min	exact	ready
obj03-04	Instalar e remover pacotes com DNF	servera	2	15 min	exact	ready
obj03-05	Configurar um repositório DNF	servera	2	15 min	exact	ready
obj03-06	Atualizar pacote por repositório de errata	servera	3	18 min	exact	ready
obj03-07	Auditar transações com DNF history	servera	2	18 min	exact	ready
obj03-08	Desafio integrado de gerenciamento de software	serverb	3	28 min	exact	ready

Objective 04 — Simple Shell Scripts

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj04-01	Parâmetros posicionais e validação de entrada	servera	1	15 min	exact	ready
obj04-02	Condições com test e colchetes	servera	2	16 min	exact	ready
obj04-03	Decisões a partir do status de comandos	servera	2	16 min	exact	ready
obj04-04	Processar saída de comandos em um script	servera	2	18 min	exact	ready
obj04-05	Loop sobre argumentos da linha de comando	servera	2	16 min	exact	ready
obj04-06	Loop sobre entrada de arquivo	servera	2	18 min	exact	ready
obj04-07	Relatório de usuários com loop e saída de comandos	servera	3	22 min	exact	ready
obj04-08	Desafio integrado de shell scripting	serverb	3	30 min	exact	ready

Objective 05 — Processes, Scheduling, Tuning & systemd

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj05-01	Identificar processos, PIDs e estados	servera	1	15 min	exact	ready

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj05-02	Controlar jobs em foreground e background	servera	2	20 min	exact	ready
obj05-03	Controlar processos com sinais	servera	2	18 min	exact	ready
obj05-04	Identificar e encerrar processos intensivos	servera	3	22 min	exact	ready
obj05-05	Ajustar prioridade com nice e renice	servera	2	20 min	exact	ready
obj05-06	Gerenciar perfis de ajuste com TuneD	servera	2	18 min	exact	ready
obj05-07	Identificar units e estados do systemd	servera	2	18 min	exact	ready
obj05-08	Start, restart e reload de serviço	servera	3	22 min	exact	ready
obj05-09	Enable, disable, mask e dependências	servera	3	22 min	exact	ready
obj05-10	Desafio integrado de processos e systemd	serverb	4	35 min	exact	ready

Objective 06 — NetworkManager, IP Networking & Name Resolution

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj06-01	Inventariar dispositivos e conexões de rede	servera	1	15 min	concept-and-state	ready
obj06-02	Criar perfil IPv4 estático com nmcli	servera	2	20 min	concept-and-state	ready
obj06-03	Modificar perfil, gateway e IPv4 secundário	servera	2	22 min	concept-and-state	ready
obj06-04	Configurar perfil dual-stack IPv4 e IPv6	servera	3	25 min	concept-and-state	ready
obj06-05	Editar perfil NetworkManager em formato keyfile	servera	3	25 min	concept-and-state	ready
obj06-06	Configurar hostname e resolução local	servera	2	22 min	concept-and-state	ready
obj06-07	Configurar DNS e search domain em perfil NetworkManager	servera	2	20 min	concept-and-state	ready
obj06-08	Desafio integrado de NetworkManager e resolução	serverb	4	35 min	concept-and-state	ready

Objective 07 — Scheduling, Temporary Files, Logging & Time

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj07-01	Gerenciar tarefa recorrente com systemd timer	servera	2	22 min	concept-and-state	ready
obj07-02	Gerenciar arquivos temporários com systemd-tmpfiles	servera	2	20 min	concept-and-state	ready
obj07-03	Agendar tarefa recorrente em /etc/cron.d	servera	2	18 min	concept-and-state	ready

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj07-04	Roteamento de eventos com rsyslog	servera	2	20 min	concept-and-state	ready
obj07-05	Filtrar eventos com journalctl	servera	2	20 min	concept-and-state	ready
obj07-06	Configurar journal persistente	servera	3	24 min	concept-and-state	ready
obj07-07	Configurar fuso horário e verificar Chrony	servera	2	20 min	concept-and-state	ready
obj07-08	Desafio integrado de scheduling e logs	serverb	4	35 min	concept-and-state	ready

Objective 08 — Partitions, Filesystems, Swap & LVM

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj08-01	Inventariar dispositivos de bloco	servera	1	15 min	concept-and-state	ready
obj08-02	Criar GPT e partição	servera	2	18 min	concept-and-state	ready
obj08-03	Criar XFS e montar temporariamente	servera	2	18 min	concept-and-state	ready
obj08-04	Montar XFS persistentemente por UUID	servera	3	22 min	concept-and-state	ready
obj08-05	Criar swap persistente em partição	servera	3	20 min	concept-and-state	ready
obj08-06	Desafio integrado de Basic Storage	servera	4	35 min	concept-and-state	ready
obj08-07	Criar PV, VG e LV	servera	3	28 min	concept-and-state	ready
obj08-08	Criar XFS em LV e persistir mount	servera	3	25 min	concept-and-state	ready
obj08-09	Estender LV e filesystem XFS	servera	3	24 min	concept-and-state	ready
obj08-10	Adicionar PV e estender VG	servera	3	28 min	concept-and-state	ready
obj08-11	Criar swap em LVM	servera	3	28 min	concept-and-state	ready
obj08-12	Desafio integrado de Storage e LVM	servera	5	45 min	concept-and-state	ready

Objective 09 — Boot, GRUB & Recovery

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj09-01	Inventário de boot, kernel e GRUB	servera	2	18 min	concept-and-state	ready
obj09-02	Argumentos persistentes do kernel com grubby	servera	3	22 min	exact	ready
obj09-03	Gerenciar o target padrão do systemd	servera	3	18 min	exact	ready

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj09-04	Rescue e emergency pelo console	servera	4	30 min	manual-console-rhel10-aligned	manual-only
obj09-05	Diagnosticar e corrigir falha de fstab	servera	4	28 min	concept-and-state	ready
obj09-06	Recuperação de acesso root pelo boot loader	servera	5	40 min	manual-console-rhel10-aligned	manual-only
obj09-07	Desafio integrado de boot e recuperação	servera	4	35 min	concept-and-state	ready

Objective 10 — SELinux & firewalld

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj10-01	Inventário de SELinux e contextos	servera	2	18 min	concept-and-state	ready
obj10-02	Restaurar SELinux enforcing atual e persistente	servera	3	20 min	exact	ready
obj10-03	Contexto SELinux persistente com semanage e restorecon	servera	3	24 min	exact	ready
obj10-04	Boolean SELinux persistente	servera	3	22 min	exact	ready
obj10-05	Diagnóstico de AVC e correção de contexto	servera	4	32 min	concept-and-state	ready
obj10-06	Zona, origem, serviço e porta no firewalld	servera	3	25 min	exact	ready
obj10-07	Rotular porta não padrão para httpd	servera	4	28 min	exact	ready
obj10-08	Persistência de porta na zona ativa	servera	3	22 min	exact	ready
obj10-09	Desafio integrado SELinux e firewalld	servera	5	40 min	concept-and-state	ready

Objective 11 — NFS & AutoFS

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj11-01	Descobrir, montar e desmontar NFS	servera	2	20 min	concept-and-state	ready
obj11-02	Montagem NFS persistente no fstab	servera	3	24 min	concept-and-state	ready
obj11-03	Mapa direto do AutoFS	servera	3	28 min	concept-and-state	ready
obj11-04	Mapa indireto com wildcard no AutoFS	servera	4	30 min	concept-and-state	ready
obj11-05	Automount NFS via systemd e fstab	servera	4	30 min	concept-and-state	ready
obj11-06	Desafio integrado NFS e AutoFS	servera	5	40 min	concept-and-state	ready

Objective 12 — Comprehensive EX200 Challenges

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj12-01	Desafio final de usuários, permissões, ACL e shell	servera	5	35 min	concept-and-state	ready
obj12-02	Desafio final de serviços, agendamento e logs	servera	5	35 min	concept-and-state	ready
obj12-03	Desafio final de partição, LVM, XFS e swap	servera	5	45 min	concept-and-state	ready
obj12-04	Desafio final de boot e manutenção persistente	servera	5	35 min	concept-and-state	ready
obj12-05	Desafio final de SELinux, firewalld e Apache	servera	5	40 min	concept-and-state	ready
obj12-06	Simulado final integrado	servera	5	45 min	concept-and-state	ready

15.1 Totais operacionais

Métrica	Quantidade
Itens de aprendizagem	100
Labs ready	98
Drills manual-only	2
Reset policy ansible	85
Reset policy guarded-storage-reset	13
Reset policy manual-console	2
Compatibilidade exact	52
Compatibilidade concept-and-state	46
Compatibilidade manual-console-rhel10-aligned	2

16. Boot, recuperação manual e segurança de armazenamento

16.1 Dois exercícios deliberadamente manuais

O Objective 09 possui sete itens. Cinco são automatizados e dois são manual-only: obj09-04 (rescue/emergency pelo console) e obj09-06 (recuperação de acesso root pelo boot loader). A escolha evita conceder ao bastion controle privilegiado out-of-band do hipervisor apenas para automatizar interação com GRUB.

Os drills devem ser praticados pelo console gráfico/libvirt. O repositório também registra a diferença de contexto entre o runtime local RHEL 9.8 e o alvo RHEL 10 para recuperação de boot.

16.2 Contrato de segurança do Objective 08

O Objective 08 é intencionalmente destrutivo apenas em discos de scratch. Os dispositivos do sistema nunca são alvos autorizados.

Categoria	**Dispositivos / objetos**
Protegidos	/dev/vda, /dev/vda1, /dev/vda2, rhel_servera/root, rhel_servera/swap
Scratch autorizado em servera	/dev/vdb (5 GiB), /dev/vdc (5 GiB), /dev/vdd (3 GiB), /dev/vde (2 GiB)

- O preflight valida host, tipo de dispositivo e faixa de tamanho antes de um lab destrutivo.
- O reset normal é limitado pela allowlist de scratch disks.
- Labs destrutivos salvam /etc/fstab antes do trabalho e restauram a cópia no finish.
- Snapshots permanecem como recuperação out-of-band para falhas anormais.

PARTE V

Engenharia, QA e manutenção

RHCSA EX200 v10 Lab

17. Estrutura do repositório e contrato de um lab

```
rhcsa-ex200-v10-lab/
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   ├── playbooks/
│   └── vars/
├── bin/lab
├── bootstrap/
├── docs/
│   ├── setup/
│   ├── releases/
│   └── development/
├── labctl/
├── labs/
├── tests/
│   ├── reference-solutions/
│   ├── integration-reference.sh
│   └── release-gate.sh
├── README.md
├── SUPPORT.md
└── LICENSE
```

17.1 Contrato de um laboratório ready

Cada laboratório do catálogo possui um manifesto `lab.yml` e, quando automatizado, os artefatos necessários para preparar, avaliar e limpar o cenário. O catálogo é descoberto pelo CLI a partir desses manifests.

Artefato	Função
lab.yml	Metadados: ID, título, objetivo, alvo, dificuldade, duração, compatibilidade, reset policy e status.
prompt.pt.md / prompt.en.md	Enunciado do estudante.
setup.yml	Preparação automatizada do cenário.
grade.py	Validação do estado final.
finish.yml	Cleanup e restauração do cenário.
solution.md	Referência de solução; não faz parte do fluxo normal do estudante.

17.2 Separação de responsabilidades

Camada	Responsabilidade
Ansible	Preparação, dependências, estado inicial e cleanup.
Python / labctl	Catálogo, orquestração, estado ativo, CLI e graders.
Estudante	Resolução manual do cenário no sistema alvo.
Host KVM	Controle out-of-band, snapshots e recuperação.

18. Release gate, integração de referência e manutenção

18.1 Gate estático

```
bash tests/release-gate.sh
```

O gate valida sintaxe Python, metadados YAML, contrato dos labs `ready`, sintaxe Ansible, smoke test do CLI e verificações específicas de segurança/portabilidade implementadas no conjunto de testes do repositório.

18.2 Integração de referência

```
bash tests/integration-reference.sh
```

O runner de referência executa o ciclo `lab start` → solução de referência → `lab grade` exigindo 100% → `lab finish` para os laboratórios automatizados selecionados. A existência desse runner não transforma a solução automatizada em método de estudo; sua finalidade é QA.

```
bash tests/integration-reference.sh obj01-04 obj01-05 obj01-06 obj01-07
```

18.3 Checklist de release

- Executar `bash tests/release-gate.sh`.
- Executar a integração de referência pertinente à alteração.
- Confirmar que o fluxo manual do estudante permanece separado das soluções de QA.
- Revisar segurança de storage, rede, SSH, SELinux e cleanup quando a mudança tocar essas áreas.
- Atualizar documentação e release notes quando o comportamento público mudar.

18.4 Fluxo Git recomendado para manutenção documental

```
git switch -c docs/minha-alteracao
# editar e validar
git diff --check
git add <arquivos>
git diff --cached --check
git commit -m "docs: describe change"
git push -u origin docs/minha-alteracao
```

Para alterações maiores, o fluxo por branch e Pull Request facilita revisão antes do merge em `main`. Em correções pequenas e controladas, o mantenedor pode adotar fluxo mais simples conforme sua política de contribuição.

19. Releases e roadmap

Versão	Nota de release
v1.2.0	v1.2.0 — Objective 06 Networking
v1.2.1	v1.2.1 — OBJ06 routing contract hotfix
v1.2.2	v1.2.2 — OBJ06 IPv6 grader hotfix
v1.2.3	v1.2.3 — OBJ06 keyfile grader permission hotfix
v1.3.0	v1.3.0 — Objective 07 Scheduling, Logging & Time
v1.3.1	v1.3.1 — OBJ07 tmpfiles grader permission hotfix
v1.3.2	v1.3.2 — OBJ07 tmpfiles age-semantics hotfix
v1.4.0	v1.4.0 — Objective 08 Storage
v1.4.1	v1.4.1 — OBJ08 safety-bootstrap hotfix
v1.4.2	v1.4.2 — OBJ08 swap identity hotfix
v1.4.3	v1.4.3 — OBJ08 protected-disk release-gate hotfix
v1.4.4	v1.4.4 — OBJ08 bounded LVM setup hotfix
v1.4.5	v1.4.5 — OBJ08 bounded reset and recovery hotfix
v1.4.6	v1.4.6 — interrupted LVM recovery hotfix
v1.4.7	v1.4.7 — timeout containment hotfix
v1.4.8	v1.4.8 — stale LVM signature hotfix
v1.4.9	v1.4.9 — release-gate grep matcher hotfix
v1.4.10	v1.4.10 — LVM swap cleanup hotfix
v1.4.12	v1.4.12 — Objective 03 local repository permission hotfix
v1.4.13	v1.4.13 — Objective 05 non-interactive pause hotfix
v1.5.0	v1.5.0 — Objective 09 Boot, GRUB & Recovery
v1.5.1	v1.5.1 — OBJ09 reference-solver stdin hotfix
v1.5.2	v1.5.2 — OBJ08-02 GPT partition grader portability hotfix
v1.6.0	v1.6.0 — Objective 10 SELinux & firewalld
v1.6.1	v1.6.1 — OBJ10-05 bounded AVC analysis hotfix
v1.7.0	v1.7.0 — Objective 11 NFS & AutoFS
v1.8.0	v1.8.0 — Objective 12 Comprehensive EX200 Challenges
v1.8.1	v1.8.1 — OBJ12-02 grader privileged-log hotfix
v1.8.2	v1.8.2 — OBJ12-03 non-interactive XFS growth hotfix
v1.8.3	v1.8.3 — OBJ12-03 evidence grader hotfix
v1.8.4	v1.8.4 — OBJ12-06 protected report evidence hotfix

19.1 Estado do roadmap

O currículo está concluído em 12 objetivos, com 98 labs automatizados e 2 drills manuais. O roadmap do repositório registra como gate de engenharia a regressão sequencial de referência dos 98 labs automatizados após a validação do Objective 12.

Observação: *A presença do runner não deve ser interpretada como evidência automática de que uma regressão completa foi executada após toda alteração documental ou funcional. O histórico de execução deve ser tratado separadamente do código que implementa o runner.*

PARTE VI

Referência e publicação

RHCSA EX200 v10 Lab

20. Referência rápida

20.1 Comandos do estudante

```
lab status
lab list
lab list --objective 08
lab show obj08-01
lab start obj08-01
ssh student@servera
lab grade obj08-01
lab finish obj08-01
```

20.2 Comandos de diagnóstico do ambiente

```
virsh list --all
virsh net-list --all
virsh pool-list --all
ansible all -m ansible.builtin.ping
ansible all -b -m ansible.builtin.command -a 'id'
lsblk
findmnt --verify
getenforce
firewall-cmd --get-active-zones
systemctl --failed
```

20.3 Valores esperados de rede

Elemento	Valor
Rede	rhcsa-lab
CIDR	192.168.100.0/24
Gateway	192.168.100.1
bastion	192.168.100.10
servera	192.168.100.11
serverb	192.168.100.12
Domínio	lab.test

20.4 Downloads e repositório

Repositório do projeto

Baixar imagem-base do laboratório

> ***Observação:*** *A distribuição pública da imagem-base deve observar os termos aplicáveis ao Red Hat Enterprise Linux, conforme registrado na documentação do projeto.*

21. Autoria, apoio, licença e marcas

21.1 Autor e perfis profissionais

Eduardo Andrade

LinkedIn — linkedin.com/in/eduardoandradej

Credly — credly.com/users/eduardoandradej

21.2 Apoie o projeto

O RHCSA EX200 v10 Lab é desenvolvido e mantido de forma independente e disponibilizado gratuitamente. O apoio financeiro é inteiramente opcional e não condiciona acesso ao código, laboratórios, documentação, funcionalidades ou licença.

PIX	**Dados**
Chave	5a41a19c-a773-4fc5-bd8d-821ebbe37c23
Favorecido	Eduardo Andrade

21.3 Independência e marcas

- O projeto não é afiliado à Red Hat.
- O projeto não é patrocinado pela Red Hat.
- O projeto não comercializa conteúdo oficial da Red Hat.
- O projeto não distribui questões reais de certificação.
- O projeto não substitui treinamento oficial, documentação oficial ou Red Hat Learning Subscription.

Red Hat, Red Hat Enterprise Linux, RHEL, RHCSA e demais marcas mencionadas pertencem aos seus respectivos titulares.

21.4 Licença

Código e documentação do repositório são disponibilizados sob a Apache License 2.0. O Apêndice A reproduz fielmente o conteúdo do arquivo `LICENSE` incluído no repositório, que também informa o endereço oficial para consulta dos termos completos.

Copyright 2026 Eduardo Andrade

Apêndice A — Arquivo LICENSE (Apache License 2.0)

Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/

Copyright 2026 Eduardo Andrade

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

RHCSA EX200 v10 Lab

Automação para preparar e avaliar.
Administração Linux para ser praticada de verdade.